

SEM_PSS Yarden Viktor Dejorno

May 19, 2025

1 Project Overview

This project is submitted as part of the Data Science course at the Technion Israel Institute of Technology, in collaboration with the Stress and Trauma Research Laboratory at Ben-Gurion University of the Negev and the Be'er Sheva Mental Health Center.

The dataset used in this project originates from over two decades of behavioral and physiological research conducted by Professor Hagit Cohen. Professor Cohen developed a diagnostic model to identify rats exhibiting behavioral patterns analogous to human post-traumatic stress disorder (PTSD). The model categorizes rats into three distinct behavioral populations, based on outcomes from standardized behavioral tests and physiological indicators.

These diagnostic categories are: - **EBR**: Extreme Behavioral Response (PTSD-like profile) - **PBR**: Partial Behavioral Response - **MBR**: Minimal Behavioral Response

Each rat is assigned **exactly one** of these three classifications, making the categories mutually exclusive and collectively exhaustive. As a result, their sum equals 1, and they are perfectly collinear. This characteristic is important to consider in modeling, as including all three simultaneously can lead to multicollinearity artifacts.

The classification is based on three core behavioral tests:

- **Anxiety Index (ANX_i)**: A composite score derived from the Elevated Plus Maze (EPM), summarizing time spent and entries into open versus closed arms. This measure reflects anxiety-like behavior.
- **Freezing (FREEZING)**: Represents the percentage of time the rat remained immobile during the Open Field test conducted on sand trauma reminder context. This serves as an indicator of fear memory and behavioral inhibition.
- **STR (Startle Reactivity)**: The average amplitude of jump responses measured during the Acoustic Startle Response test. Higher amplitude reflects greater physiological hyperarousal in response to acoustic stimuli.

In this research, we aim to assess whether certain population characteristics such as sex, age, or physical condition influence a rat's likelihood of exhibiting PTSD-like behavior following exposure to a traumatic event.

All rats in the study were exposed to a stressor: being placed inside a cat litter box. For approximately half of the rats, this was a true traumatic exposure (containing used cat litter), while the control group was placed in clean sand. This exposure is marked in the dataset by the variable **PSS** (0 = control, 1 = stress).

The goal of my project is to investigate whether the effect of trauma (PSS) differs across population subgroups.

The populations in the dataset are coded as follows: - **olderimale**: adult male - **olderi**: general adult - **Juvenailemale**: male exposed to trauma during juvenile phase - **Juvenaile**: juvenile exposed to trauma - **femaleolderi**: adult female - **femaleJuvenaile**: female exposed to trauma during juvenile phase - **femaleyoung**: young female - **inactive**: trauma occurred during rest hours - **HFD**: high-fat diet subject - **FD**: lean subject - **female**: indicates sex (1 = female, 0 = male)

In the following notebook, I will first demonstrate why logistic regression struggles in this context. I will then present the current structural modeling approach used by the lab, and finally, show how the XGBoost algorithm used in our course provides practical and effective advantages.

1.1 Library Imports

In this cell, we import all necessary libraries used throughout the project. These include packages for:

- **Data manipulation and visualization**: pandas, numpy, matplotlib, seaborn, missingno
- **Statistical modeling and regression diagnostics**: statsmodels, pingouin
- **Preprocessing and sampling**: sklearn.preprocessing, sklearn.utils
- **Structural equation modeling (SEM)**: semopy, graphviz
- **Machine learning**: xgboost

These libraries support all stages of the workflow from exploratory data analysis and feature scaling, to advanced SEM analysis and classification modeling.

```
[1]: # Data manipulation & visualization
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import missingno as msno

# Statistical modeling & diagnostics
import statsmodels.api as sm
from statsmodels.stats.outliers_influence import variance_inflation_factor
import pingouin as pg
from scipy.stats import chi2

# SEM (Structural Equation Modeling)
from semopy import Model, calc_stats, semplot, gather_statistics
from semopy.inspector import inspect
from graphviz import Digraph

# Machine learning
from xgboost import XGBClassifier

# Data preprocessing & sampling
from sklearn.preprocessing import MinMaxScaler
```

```
from sklearn.utils import resample
```

*Note: Ensure all required packages are installed in your environment. For example, semopy, pingouin, and xgboost may require separate installation using `pip install` or `conda install`.

1.2 Data Loading and Initial Processing

In this section, we load the dataset and perform initial exploration steps:

1. Display the dataset's structure: columns, sample records, and descriptive statistics.
2. Identify and report constant columns (i.e., those with no variation) and columns with missing values.
3. Visualize the missing value pattern using `missingno`.
4. Normalize key behavioral variables (STR, FREAzing) using MinMax scaling and mean-centering.
5. Center the ANX_i variable by subtracting its mean.

These preprocessing steps ensure that the data is clean, interpretable, and ready for downstream analysis.

Note: Mean-centering and normalization are especially important in SEM models to reduce multicollinearity and improve interpretability of path coefficients.

```
[2]: # 1. Load the dataset
df = pd.read_csv('/Users/djournio/PyCharm/SEM_PSS/data.csv')

# 2. Display dataset structure
print("Columns in the dataset:")
print(df.columns.tolist())
print("\nSample of Raw Data:")
print(df.head())

# 3. Identify constant columns and columns with missing values
constant_columns = df.loc[:, df.nunique() <= 1].columns.tolist()
nan_columns = df.columns[df.isna().any()].tolist()
print("\nConstant columns:", constant_columns)
print("Columns with NaN values:", nan_columns)

# 4. Show descriptive statistics and visualize missing data
print("\nDescriptive statistics:")
print(df.describe(include='all'))
msno.matrix(df)
plt.show()

# 5. Normalize and mean-center key behavioral variables
scaler = MinMaxScaler()

# STR (Stress reactivity)
```

```

df['STR_norm'] = scaler.fit_transform(df[['STR']])
df['STR_c'] = df['STR_norm'] - df['STR_norm'].mean()

# FREAIZING (Freezing behavior)
df['FREAIZING_norm'] = scaler.fit_transform(df[['FREAIZING']])
df['FREAIZING_c'] = df['FREAIZING_norm'] - df['FREAIZING_norm'].mean()

# ANX_i (Anxiety index) centered
df['ANX_i_c'] = df['ANX_i'] - df['ANX_i'].mean()

```

Columns in the dataset:

```

['EXP_RAT_NUM', 'EXP', 'PSS', 'olderimale', 'olderi', 'Juvenailemale',
'Juvenaile', 'femaleolderi', 'femaleJuvenaile', 'femaleyoung', 'inactive',
'HFD', 'FD', 'LEW', 'EBR', 'MBR', 'PBR', 'ANX_i', 'STR', 'FREAIZING', 'female',
'active']

```

Sample of Raw Data:

	EXP_RAT_NUM	EXP	PSS	olderimale	olderi	\
0	1	PSS - male -Young - 3 Month	1	0	0	
1	2	PSS - male -Young - 3 Month	1	0	0	
2	3	PSS - male -Young - 3 Month	1	0	0	
3	4	PSS - male -Young - 3 Month	1	0	0	
4	5	PSS - male -Young - 3 Month	1	0	0	

	Juvenailemale	Juvenaile	femaleolderi	femaleJuvenaile	femaleyoung	...	\
0	0	0	0	0	0	...	
1	0	0	0	0	0	...	
2	0	0	0	0	0	...	
3	0	0	0	0	0	...	
4	0	0	0	0	0	...	

	FD	LEW	EBR	MBR	PBR	ANX_i	STR	FREAIZING	female	active
0	0	0	0	0	1	0.82	85.2	0.21	0	0
1	0	0	0	0	1	0.91	136.2	0.36	0	0
2	0	0	0	0	1	0.81	146.2	0.37	0	0
3	0	0	0	0	1	0.90	174.3	0.41	0	0
4	0	0	0	0	1	0.84	625.5	0.19	0	0

[5 rows x 22 columns]

Constant columns: ['LEW']

Columns with NaN values: []

Descriptive statistics:

	EXP_RAT_NUM	EXP	PSS	olderimale	olderi	\
count	556.000000	556	556.000000	556.000000	556.000000	
unique	NaN	20	NaN	NaN	NaN	
top	NaN	PSS - HFD	NaN	NaN	NaN	

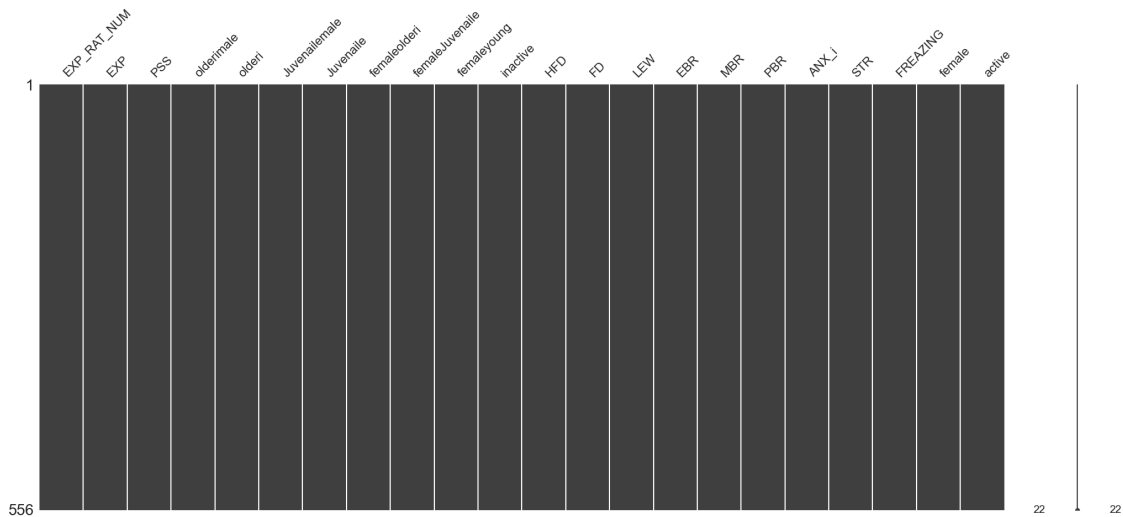
freq	NaN	35	NaN	NaN	NaN
mean	14.784173	NaN	0.521583	0.098921	0.206835
std	8.608617	NaN	0.499984	0.298825	0.405401
min	1.000000	NaN	0.000000	0.000000	0.000000
25%	7.000000	NaN	0.000000	0.000000	0.000000
50%	14.000000	NaN	1.000000	0.000000	0.000000
75%	22.000000	NaN	1.000000	0.000000	0.000000
max	35.000000	NaN	1.000000	1.000000	1.000000

	Juvenailemale	Juvenaile	femaleolderi	femaleJuvenaile	femaleyoung \
count	556.000000	556.000000	556.000000	556.000000	556.000000
unique	NaN	NaN	NaN	NaN	NaN
top	NaN	NaN	NaN	NaN	NaN
freq	NaN	NaN	NaN	NaN	NaN
mean	0.071942	0.143885	0.107914	0.071942	0.111511
std	0.258625	0.351289	0.310551	0.258625	0.315047
min	0.000000	0.000000	0.000000	0.000000	0.000000
25%	0.000000	0.000000	0.000000	0.000000	0.000000
50%	0.000000	0.000000	0.000000	0.000000	0.000000
75%	0.000000	0.000000	0.000000	0.000000	0.000000
max	1.000000	1.000000	1.000000	1.000000	1.000000

	...	FD	LEW	EBR	MBR	PBR \
count	...	556.000000	556.0	556.000000	556.000000	556.000000
unique	...	NaN	NaN	NaN	NaN	NaN
top	...	NaN	NaN	NaN	NaN	NaN
freq	...	NaN	NaN	NaN	NaN	NaN
mean	...	0.093525	0.0	0.172662	0.183453	0.643885
std	...	0.291429	0.0	0.378295	0.387386	0.479281
min	...	0.000000	0.0	0.000000	0.000000	0.000000
25%	...	0.000000	0.0	0.000000	0.000000	0.000000
50%	...	0.000000	0.0	0.000000	0.000000	1.000000
75%	...	0.000000	0.0	0.000000	0.000000	1.000000
max	...	1.000000	0.0	1.000000	1.000000	1.000000

	ANX_i	STR	FREAZING	female	active
count	556.000000	556.000000	556.000000	556.000000	556.000000
unique	NaN	NaN	NaN	NaN	NaN
top	NaN	NaN	NaN	NaN	NaN
freq	NaN	NaN	NaN	NaN	NaN
mean	0.804335	486.554856	0.256639	0.291367	0.111511
std	0.155890	266.305438	0.139280	0.454801	0.315047
min	0.150000	49.600000	0.030442	0.000000	0.000000
25%	0.700000	278.900000	0.160000	0.000000	0.000000
50%	0.820000	461.650000	0.232750	0.000000	0.000000
75%	0.930000	689.575000	0.320580	1.000000	0.000000
max	1.000000	1695.100000	0.852650	1.000000	1.000000

[11 rows x 22 columns]



1.2.1 Interpreting the Missing Data Matrix

The plot above is a **missing value matrix** generated using the `missingno` library. Each vertical line represents a column (feature) in the dataset, and each horizontal line represents a sample (row).

- **Gray bars** indicate **present values**.
- **White bars** indicate **missing values**.

If the entire column is solid white, there are no missing values for that variable. If gray areas are visible, they show exactly which values are missing and whether missingness clusters across rows or columns. This visualization helps identify systematic data quality issues and decide on appropriate imputation strategies.

1.3 Function: Data Cleaning for Regression

This function automates the cleaning process of the dataset prior to regression modeling. It performs the following steps:

1. **Removes constant columns:** Columns with no variability are uninformative and are automatically dropped.
2. **Removes highly correlated features:** Among pairs of features with Pearson correlation above a defined threshold (default: 0.95), it removes one feature from each pair unless the feature is explicitly listed in `protected_columns`.

This approach helps reduce multicollinearity, simplify the model, and avoid instability in regression coefficients. The function allows for certain variables to be preserved even if they are highly correlated, by specifying them in the `protected_columns` list.

```
[3]: # Function to clean the dataset for regression analysis
```

```
def clean_data_for_regression(df, correlation_threshold=0.95,
    ↪protected_columns=None):
    if protected_columns is None:
        protected_columns = []

    # 1. Drop constant columns (no variance)
    constant_cols = df.loc[:, df.nunique() <= 1].columns.tolist()
    df = df.drop(columns=constant_cols)
    print(f"Dropped constant columns: {constant_cols}")

    # 2. Drop one of each pair of highly correlated features (unless protected)
    corr_matrix = df.corr(numeric_only=True).abs()
    upper_triangle = corr_matrix.where(
        np.triu(np.ones(corr_matrix.shape), k=1).astype(bool)
    )

    to_drop = []
    for column in upper_triangle.columns:
        if (upper_triangle[column] > correlation_threshold).any() and column
    ↪not in protected_columns:
            to_drop.append(column)

    df = df.drop(columns=to_drop)
    print(f"Dropped highly correlated columns (excluding protected ones):
    ↪{to_drop}")

    return df
```

This function is especially useful in datasets with engineered features, where multicollinearity often arises. Protecting key features from removal ensures that domain-specific variables are retained for modeling.

1.4 Correlation Matrix (Pearson)

In this cell, we calculate and visualize the Pearson correlation matrix between continuous variables and binary indicators.

- `cont_vars` includes mean-centered behavioral measures: `ANX_i_c`, `STR_c`, and `FREAZING_c`.
- `bin_vars` includes binary group and condition indicators (e.g., gender, trauma exposure, and physiological classifications).

We compute pairwise Pearson correlations using `pingouin` and visualize them using a heatmap. This helps identify strong linear relationships or multicollinearity between predictors, which may affect the stability and interpretability of regression models.

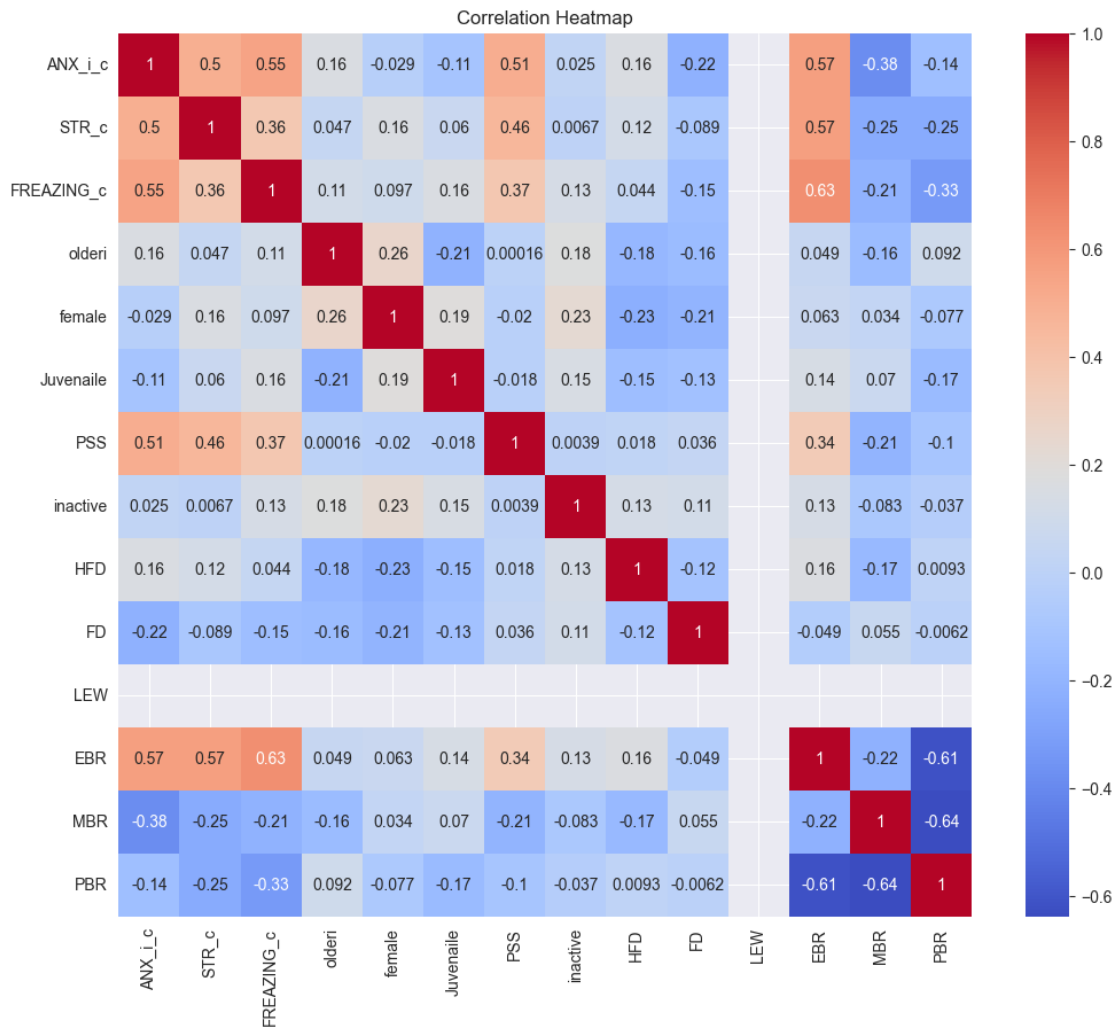
```
[4]: # Define continuous and binary variables
cont_vars = ['ANX_i_c', 'STR_c', 'FREAZING_c']
bin_vars = ['olderi', 'female', 'Juvenile', 'PSS', 'inactive', 'HFD', 'FD',
    ↪'LEW', 'EBR', 'MBR', 'PBR']
```

```

# Compute Pearson correlations
corr = pg.pairwise_corr(df[cont_vars + bin_vars], method='pearson')

# Display correlation heatmap
plt.figure(figsize=(12, 10))
sns.heatmap(df[cont_vars + bin_vars].corr(), annot=True, cmap='coolwarm')
plt.title("Correlation Heatmap")
plt.show()
print(corr)

```



	X	Y	method	alternative	n	r	CI95% \
0	ANX_i_c	STR_c	pearson	two-sided	556	0.500290	[0.44, 0.56]
1	ANX_i_c	FREAZING_c	pearson	two-sided	556	0.548358	[0.49, 0.6]
2	ANX_i_c	older_i	pearson	two-sided	556	0.159987	[0.08, 0.24]
3	ANX_i_c	female	pearson	two-sided	556	-0.028519	[-0.11, 0.05]

4	ANX_i_c	Juvenaile	pearson	two-sided	556	-0.110774	[-0.19, -0.03]
..	...	...	...	...	...	...	...
73	FD	MBR	pearson	two-sided	556	0.055228	[-0.03, 0.14]
74	FD	PBR	pearson	two-sided	556	-0.006218	[-0.09, 0.08]
75	EBR	MBR	pearson	two-sided	556	-0.216535	[-0.29, -0.14]
76	EBR	PBR	pearson	two-sided	556	-0.614279	[-0.66, -0.56]
77	MBR	PBR	pearson	two-sided	556	-0.637355	[-0.68, -0.59]

	p-unc	BF10	power
0	1.492239e-36	1.802e+33	1.000000
1	5.774369e-45	3.969e+41	1.000000
2	1.516311e-04	68.144	0.966971
3	5.021619e-01	0.067	0.102987
4	8.944317e-03	1.604	0.744675
..	...	...	...
73	1.934909e-01	0.123	0.255510
74	8.836945e-01	0.054	0.052428
75	2.527811e-07	3.032e+04	0.999355
76	5.599275e-59	3.259e+55	1.000000
77	1.043743e-64	1.608e+61	1.000000

[78 rows x 10 columns]

The heatmap provides a visual overview of how features relate to each other. Strong correlations (closer to 1) may signal redundancy or potential multicollinearity risks in regression or SEM models.

1.5 Dataset Cleaning with Protected Features

We now apply the custom data cleaning function `clean_data_for_regression`, while explicitly protecting key engineered features from removal:

- `STR_norm`, `STR_c`: normalized and centered stress response
- `FREAZING_norm`, `FREAZING_c`: normalized and centered freezing behavior
- `ANX_i_c`: mean-centered anxiety index

These variables are critical to our modeling and must be retained, even if they are highly correlated with others. This step ensures the dataset is simplified by removing constant and redundant features, while preserving domain-specific variables for further analysis.

```
[5]: # Clean the dataset without removing key engineered features
protected = ['STR_norm', 'STR_c', 'FREAZING_norm', 'FREAZING_c', 'ANX_i_c']
df = clean_data_for_regression(df, protected_columns=protected)
```

Dropped constant columns: ['LEW']

Dropped highly correlated columns (excluding protected ones): ['active']

By protecting selected features, we maintain consistency with domain knowledge and ensure important behavioral constructs remain in the dataset for downstream modeling. Note: Two variables were dropped during cleaning LEW and active. The LEW column

*represents a rat strain (Lewis), which did not appear in any of the final data entries. The **active** variable is the inverse of **inactive** and was therefore redundant.*

1.6 Updated Correlation Analysis (Post-Cleaning)

After cleaning the dataset and protecting key features, we recalculate the Pearson correlation matrix. This step focuses only on the relevant continuous and binary variables:

- **cont_vars**: centered behavioral measures (ANX_i_c, STR_c, FREAIZING_c)
- **bin_vars**: binary indicators of group membership, trauma exposure, and classifications (olderi, female, etc.)

We store the original dataset for backup (**df_old**) and work on a filtered subset (**df**). The goal is to better understand the linear relationships between features after data reduction and ensure no multicollinearity issues remain.

We use: - **pingouin** for detailed correlation statistics - **seaborn.heatmap** for a clear visual representation of all pairwise correlations

```
[6]: # Define continuous and binary variables for focused correlation analysis
cont_vars = ['ANX_i_c', 'STR_c', 'FREAIZING_c']
bin_vars = ['olderi', 'female', 'Juvenile', 'PSS', 'inactive', 'HFD', 'FD', 'EBR', 'MBR', 'PBR']

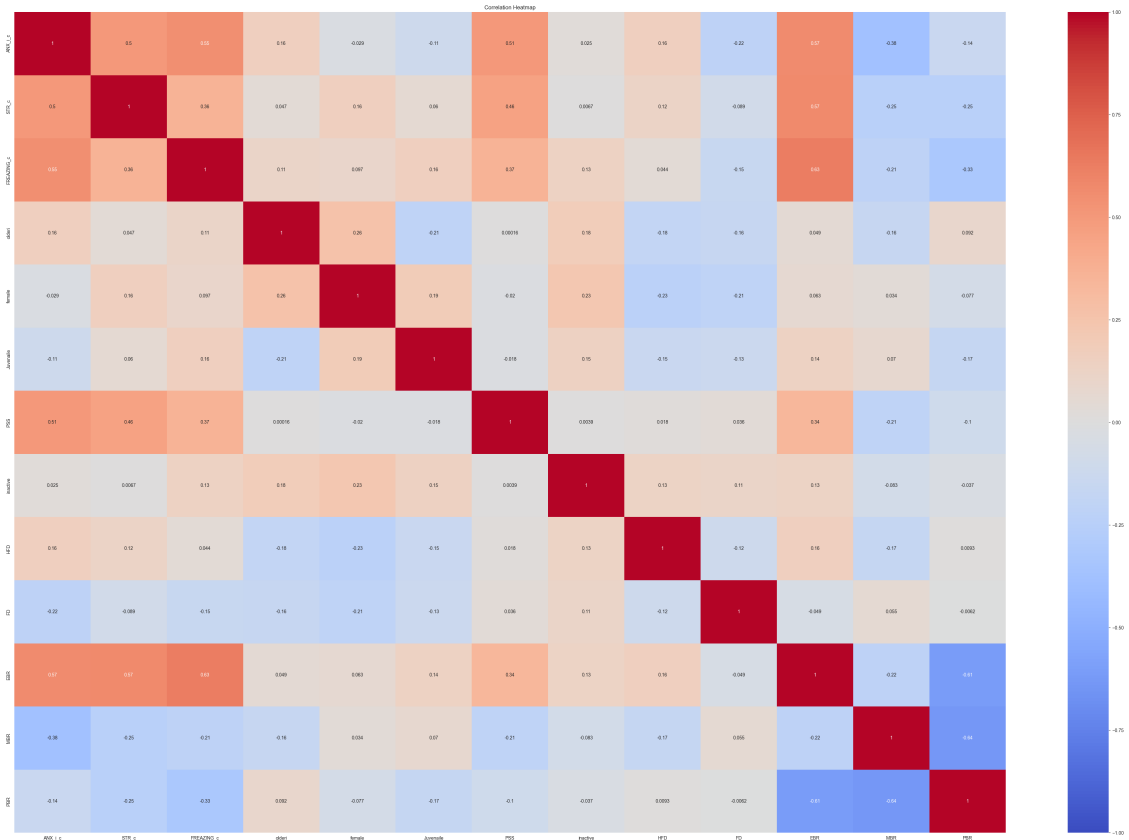
# Save the full dataset before filtering
df_old = df.copy()

# Create a new dataframe containing only relevant features
df = df[cont_vars + bin_vars].copy()

# Compute pairwise Pearson correlations using pingouin
corr = pg.pairwise_corr(df, method='pearson')

# Display heatmap of correlations
plt.figure(figsize=(48, 32))
sns.heatmap(df.corr(), annot=True, cmap='coolwarm', vmin=-1, vmax=1)
plt.title("Correlation Heatmap")
plt.show()

# Print detailed correlation statistics
print("\nDetailed Correlation Matrix:")
print(corr)
```



Detailed Correlation Matrix:

	X	Y	method	alternative	n	r	CI95%	\
0	ANX_i_c	STR_c	pearson	two-sided	556	0.500290	[0.44, 0.56]	
1	ANX_i_c	FREAZING_c	pearson	two-sided	556	0.548358	[0.49, 0.6]	
2	ANX_i_c	olderi	pearson	two-sided	556	0.159987	[0.08, 0.24]	
3	ANX_i_c	female	pearson	two-sided	556	-0.028519	[-0.11, 0.05]	
4	ANX_i_c	Juvenile	pearson	two-sided	556	-0.110774	[-0.19, -0.03]	
..	***	***	***	***	***	***		
73	FD	MBR	pearson	two-sided	556	0.055228	[-0.03, 0.14]	
74	FD	PBR	pearson	two-sided	556	-0.006218	[-0.09, 0.08]	
75	EBR	MBR	pearson	two-sided	556	-0.216535	[-0.29, -0.14]	
76	EBR	PBR	pearson	two-sided	556	-0.614279	[-0.66, -0.56]	
77	MBR	PBR	pearson	two-sided	556	-0.637355	[-0.68, -0.59]	

	p-unc	BF10	power
0	1.492239e-36	1.802e+33	1.000000
1	5.774369e-45	3.969e+41	1.000000
2	1.516311e-04	68.144	0.966971
3	5.021619e-01	0.067	0.102987
4	8.944317e-03	1.604	0.744675

```

..      ...      ...      ...
73  1.934909e-01      0.123  0.255510
74  8.836945e-01      0.054  0.052428
75  2.527811e-07  3.032e+04  0.999355
76  5.599275e-59  3.259e+55  1.000000
77  1.043743e-64  1.608e+61  1.000000

```

[78 rows x 10 columns]

This correlation matrix allows us to verify that the remaining variables are not overly redundant. We can also observe expected patterns (e.g., behavioral variables correlating with trauma exposure or physiological traits).

1.7 Multicollinearity Check using VIF

In this step, we calculate the Variance Inflation Factor (VIF) for each predictor to assess multicollinearity. VIF quantifies how much the variance of a regression coefficient is inflated due to correlation with other predictors.

- A VIF > 10 is typically considered problematic.
- We add a constant term to the data (const=1) to satisfy the requirements of the VIF calculation.

The results help identify whether any predictors are highly redundant and might distort regression estimates.

```

[7]: # Add constant term for VIF computation
X = df.assign(const=1)

# Compute VIF values for each variable
vif_data = pd.DataFrame()
vif_data['Variable'] = X.columns
vif_data['VIF'] = [variance_inflation_factor(X.values, i) for i in range(X.
    ↪shape[1])]

# Display VIF values
print("\nVariance Inflation Factors (VIF):")
print(vif_data)

# Identify variables with multicollinearity issues
problematic = vif_data[vif_data['VIF'] > 10]
print("\nVariables with VIF > 10:")
print(problematic)

```

Variance Inflation Factors (VIF):

	Variable	VIF
0	ANX_i_c	2.430867
1	STR_c	1.864943
2	FREAZING_c	1.978049

```

3      olderi  1.389019
4      female  1.372394
5  Juvenile  1.372014
6      PSS    1.557481
7      inactive 1.291492
8      HFD    1.368797
9      FD     1.353470
10     EBR     inf
11     MBR     inf
12     PBR     inf
13     const  0.000000

```

Variables with VIF > 10:

```

Variable  VIF
10     EBR  inf
11     MBR  inf
12     PBR  inf

```

```

/Users/djournio/PyCharm/SEM_PSS/.venv/lib/python3.13/site-
packages/statsmodels/stats/outliers_influence.py:197: RuntimeWarning: divide by
zero encountered in scalar divide

```

```

    vif = 1. / (1. - r_squared_i)

```

```

/Users/djournio/PyCharm/SEM_PSS/.venv/lib/python3.13/site-
packages/statsmodels/regression/linear_model.py:1782: RuntimeWarning: divide by
zero encountered in scalar divide

```

```

    return 1 - self.ssr/self.centered_tss

```

*Note: The variables **EBR**, **MBR**, and **PBR** represent mutually exclusive diagnostic categories each rat belongs to exactly one group. As a result, they are linearly dependent (their sum equals 1), which naturally leads to infinite VIF values. This is expected and not an error. In modeling, it is recommended to include only two of the three variables to avoid perfect multicollinearity.*

1.8 Recomputing VIF After Removing MBR

To address perfect multicollinearity among the diagnostic classification variables (**EBR**, **PBR**, **MBR**), we remove one of them (**MBR**) before proceeding with modeling. Since these three categories are mutually exclusive and their sum equals 1, including all of them would lead to perfect linear dependence.

After dropping **MBR**, we recompute the Variance Inflation Factor (VIF) to ensure that multicollinearity has been resolved. The intercept term (**const**) is excluded from the final display.

```

[8]: # Drop one diagnostic category to avoid multicollinearity
df = df.drop(columns=['MBR'])

# Recompute VIF after dropping 'MBR'
X = df.assign(const=1) # Add constant for VIF computation

```

```

vif_data = pd.DataFrame({
    'Variable': X.columns,
    'VIF': [variance_inflation_factor(X.values, i) for i in range(X.shape[1])]
})

# Exclude the intercept from the output
vif_data = vif_data[vif_data['Variable'] != 'const']

# Display updated VIF values
print("\nNew Variance Inflation Factors (VIF) after dropping 'MBR':")
print(vif_data)

```

New Variance Inflation Factors (VIF) after dropping 'MBR':

	Variable	VIF
0	ANX_i_c	2.430867
1	STR_c	1.864943
2	FREAZING_c	1.978049
3	olderi	1.389019
4	female	1.372394
5	Juvenaile	1.372014
6	PSS	1.557481
7	inactive	1.291492
8	HFD	1.368797
9	FD	1.353470
10	EBR	3.779097
11	PBR	1.885146

After removing MBR, all VIF values are now below 5 well within acceptable limits. This indicates that multicollinearity has been effectively resolved. The EBR variable, while still moderately correlated with other features, no longer causes modeling issues and can be safely included in downstream analyses.

1.9 Adding Interaction Terms and Rechecking Multicollinearity

In this step, we engineer interaction features between gender and age/developmental group:

1. **female_x_olderi**: Interaction between being female and being an adult.
2. **female_x_Juvenaile**: Interaction between being female and having experienced trauma during the juvenile phase.

These interaction terms allow us to model more complex relationships, such as whether the effect of age on PTSD-like behavior differs between males and females.

After introducing the interaction terms, we recompute the Variance Inflation Factors (VIF) to ensure that multicollinearity remains under control.

```

[9]: # 1. Create interaction term: female olderi
df['female_x_olderi'] = df['female'] * df['olderi']

```

```

# 2. Create interaction term: female Juvenile
df['female_x_Juvenile'] = df['female'] * df['Juvenile']

# Recalculate VIF after adding interaction terms
X = df.assign(const=1)

vif_data = pd.DataFrame({
    'Variable': X.columns,
    'VIF': [variance_inflation_factor(X.values, i) for i in range(X.shape[1])]
})

# Exclude the constant term from the display
vif_data = vif_data[vif_data['Variable'] != 'const']

# Display updated VIF values
print("\nNew Variance Inflation Factors (VIF):")
print(vif_data)

```

New Variance Inflation Factors (VIF):

	Variable	VIF
0	ANX_i_c	2.470307
1	STR_c	2.002074
2	FREAZING_c	2.008851
3	olderi	2.548432
4	female	2.926226
5	Juvenile	2.406482
6	PSS	1.603151
7	inactive	1.365965
8	HFD	1.420640
9	FD	1.445674
10	EBR	3.798351
11	PBR	1.925510
12	female_x_olderi	3.558756
13	female_x_Juvenile	2.913910

Interaction terms were successfully added without introducing problematic multicollinearity. This enables the model to explore gender-specific effects across developmental stages.

1.10 Final Correlation Analysis After Feature Engineering

This step presents an updated correlation analysis, now including interaction terms and after cleaning the dataset. We focus on:

- **Continuous behavioral variables:** ANX_i_c, STR_c, FREAZING_c
- **Binary population and experimental variables:** including olderi, female, Juvenile, PSS, and dietary/activity indicators
- **Interaction terms:** female_x_Juvenile and female_x_olderi

We visualize pairwise Pearson correlations using a heatmap and generate a detailed correlation table using pingouin. This helps confirm that the added features do not introduce problematic redundancy or perfect correlations with existing variables.

```
[10]: # 8. Correlation Matrix (Updated)

# Continuous and centered variables
cont_vars = ['ANX_i_c', 'STR_c', 'FREAzing_c']

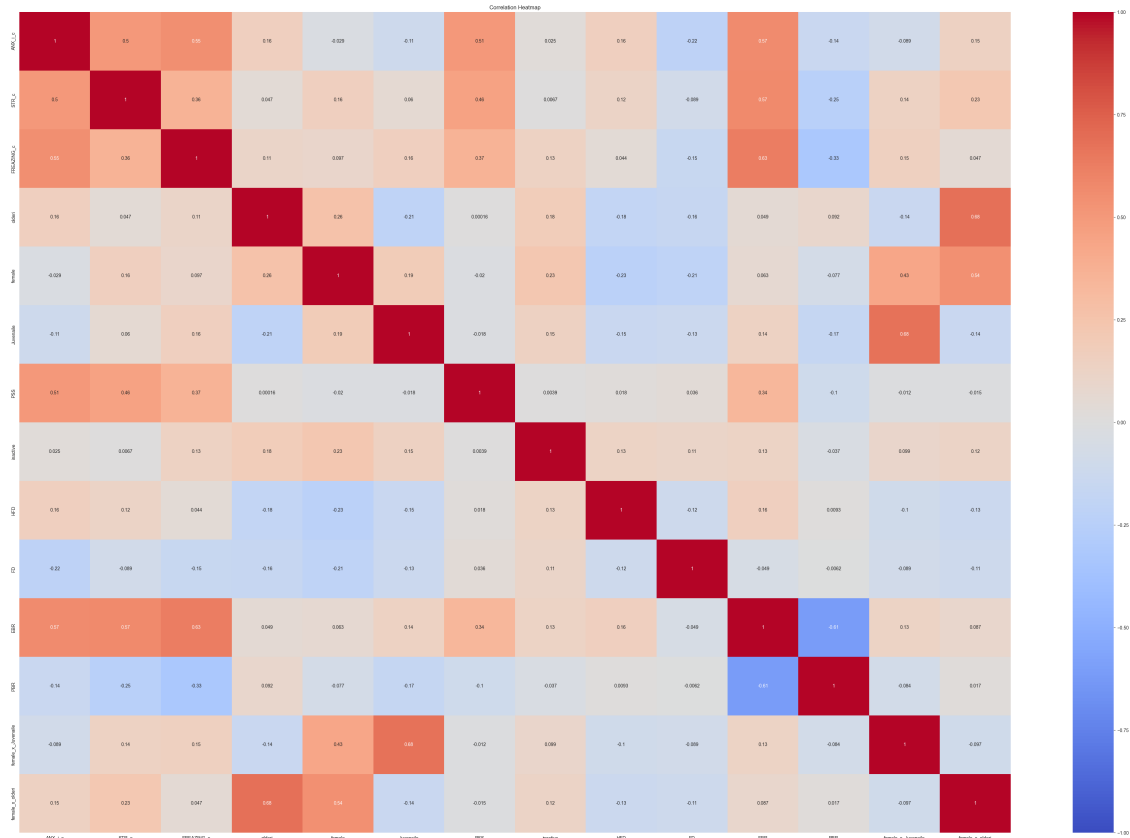
# Binary variables
bin_vars = ['olderi', 'female', 'Juvenile', 'PSS', 'inactive', 'HFD', 'FD', 'EBR', 'PBR', 'female_x_Juvenile', 'female_x_olderi']

# Create a new DataFrame with only relevant variables
df = df[cont_vars + bin_vars].copy()

# Compute Pearson correlation matrix using pingouin
corr = pg.pairwise_corr(df, method='pearson')

# Display correlation heatmap
plt.figure(figsize=(48, 32))
sns.heatmap(df.corr(), annot=True, cmap='coolwarm', vmin=-1, vmax=1)
plt.title("Correlation Heatmap")
plt.show()

# Print detailed correlation matrix
print("\nDetailed Correlation Matrix:")
print(corr)
```

Detailed Correlation Matrix:

	X	Y	method	alternative	n \
0	ANX_i_c	STR_c	pearson	two-sided	556
1	ANX_i_c	FREAZING_c	pearson	two-sided	556
2	ANX_i_c	olderi	pearson	two-sided	556
3	ANX_i_c	female	pearson	two-sided	556
4	ANX_i_c	Juvenile	pearson	two-sided	556
..	...	...	...	...	...
86	EBR	female_x_Juvenile	pearson	two-sided	556
87	EBR	female_x_olderi	pearson	two-sided	556
88	PBR	female_x_Juvenile	pearson	two-sided	556
89	PBR	female_x_olderi	pearson	two-sided	556
90	female_x_Juvenile	female_x_olderi	pearson	two-sided	556

	r	CI95%	p-unc	BF10	power
0	0.500290	[0.44, 0.56]	1.492239e-36	1.802e+33	1.000000
1	0.548358	[0.49, 0.6]	5.774369e-45	3.969e+41	1.000000
2	0.159987	[0.08, 0.24]	1.516311e-04	68.144	0.966971
3	-0.028519	[-0.11, 0.05]	5.021619e-01	0.067	0.102987
4	-0.110774	[-0.19, -0.03]	8.944317e-03	1.604	0.744675

..	...	...	...	...	...
86	0.130637	[0.05, 0.21]	2.023988e-03	6.147	0.871241
87	0.086506	[0.0, 0.17]	4.144914e-02	0.422	0.532304
88	-0.083661	[-0.17, -0.0]	4.864317e-02	0.369	0.505439
89	0.016547	[-0.07, 0.1]	6.970362e-01	0.057	0.067520
90	-0.096837	[-0.18, -0.01]	2.239471e-02	0.716	0.627877

[91 rows x 10 columns]

This correlation matrix confirms that none of the interaction terms are perfectly correlated with existing variables, and all features are safe to include in the model. We can now proceed to modeling with a well-structured and balanced feature set.

2 Logistic Regression:

2.1 Predicting PBR

In this step, we fit a logistic regression model to estimate the likelihood of a rat being categorized as PBR (Partial Behavioral Response), based on behavioral, demographic, and experimental variables.

- The selected features include trauma exposure (PSS), behavioral indices (ANX_i_c, STR_c, FREAzing_c), and various group indicators (e.g., older_i, female, inactive, etc.).
- The logistic model is fitted using `statsmodels.Logit`.
- We extract the Odds Ratios (OR) and 95% confidence intervals (CI) for interpretability.

This analysis helps quantify the relative influence of each feature on the probability of PBR classification.

```
[11]: # Define features and target variable
features = ['older_i', 'female', 'Juvenile', 'PSS', 'inactive', 'HFD', 'FD',
            'ANX_i_c', 'STR_c', 'FREAzing_c']
target = 'PBR'

# Drop rows with missing values
df_log = df[features + [target]].dropna().copy()

# Define X and y
X = df_log[features]
y = df_log[target]

# Add intercept for statsmodels
X_sm = sm.add_constant(X)

# Fit logistic regression model
model = sm.Logit(y, X_sm)
result = model.fit()

# Extract summary and calculate Odds Ratios with 95% CI
summary_df = result.summary2().tables[1]
```

```

summary_df['OR'] = np.exp(summary_df['Coef.']) # Odds Ratio
summary_df['CI_low'] = np.exp(summary_df['Coef.'] - 1.96 * summary_df['Std.Err.
↪'])
summary_df['CI_high'] = np.exp(summary_df['Coef.'] + 1.96 * summary_df['Std.Err.
↪'])

# Display OR table
print(summary_df[['OR', 'CI_low', 'CI_high', 'P>|z|']])

# Plot Odds Ratios and Confidence Intervals
plt.figure(figsize=(10, 6))
sns.pointplot(data=summary_df.reset_index(), x='index', y='OR',
↪linestyle='none', color='black')
plt.errorbar(x=np.arange(len(summary_df)),
            y=summary_df['OR'],
            yerr=[summary_df['CI_low'] - summary_df['OR'],
↪summary_df['CI_high'] - summary_df['OR']],
            fmt='none', c='red', capsize=4)
plt.axhline(1, linestyle='--', color='gray')
plt.xticks(rotation=45)
plt.title('Odds Ratios with 95% CI for PBR ~ all_features')
plt.ylabel('Odds Ratio')
plt.tight_layout()
plt.show()

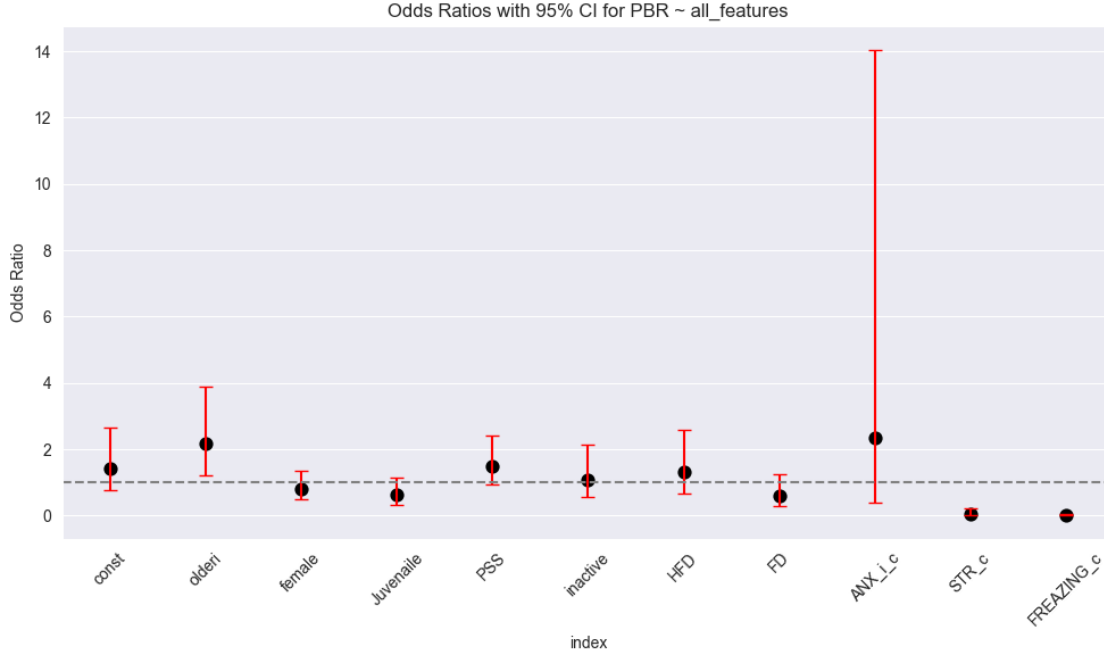
```

Optimization terminated successfully.

Current function value: 0.563770

Iterations 6

	OR	CI_low	CI_high	P> z
const	1.438318	0.775656	2.667110	2.486389e-01
olderi	2.181557	1.222137	3.894156	8.326134e-03
female	0.819166	0.498572	1.345911	4.310682e-01
Juvenaile	0.624404	0.335645	1.161585	1.370011e-01
PSS	1.491668	0.926633	2.401246	9.970085e-02
inactive	1.088694	0.551082	2.150776	8.067407e-01
HFD	1.307569	0.662744	2.579785	4.392348e-01
FD	0.601642	0.291679	1.241000	1.689825e-01
ANX_i_c	2.346612	0.392156	14.041833	3.500629e-01
STR_c	0.051100	0.011530	0.226483	9.040135e-05
FREAZING_c	0.009697	0.002140	0.043945	1.821394e-09



2.1.1 Interpretation of Logistic Regression Results for PBR

The logistic regression model successfully converged and reveals the following insights:

- **Older rats (olderi)** are significantly more likely to be classified as PBR (OR = 2.18, p approximately 0.008).
- **Freezing behavior (FREEZING_c)** and **stress reactivity (STR_c)** are both highly significant negative predictors of PBR. Higher scores on these behavioral metrics are strongly associated with a reduced likelihood of PBR classification, likely indicating stronger PTSD-like tendencies.
- The **trauma variable (PSS)** shows a positive trend but does not reach full statistical significance (OR = 1.49, p approximately 0.099).
- Other variables, including gender and diet, did not show statistically significant effects in this model.

These results suggest that certain behavioral responses and age are stronger indicators of partial behavioral classification than trauma exposure or sex alone.

2.2 Predicting EBR

This cell fits a logistic regression model to estimate the likelihood of a rat being classified as EBR (Extreme Behavioral Response), which represents a PTSD-like phenotype.

The model includes demographic variables, trauma exposure (PSS), and behavioral predictors. The output includes Odds Ratios (OR) and 95% confidence intervals (CI), which help interpret the strength and direction of each feature's influence.

By comparing the results with the earlier PBR model, we can assess whether different features distinguish between partial and extreme behavioral responses to trauma.

```

[12]: # Define features and target
features = ['olderi', 'female', 'Juvenile', 'PSS', 'inactive', 'HFD', 'FD',
           'ANX_i_c', 'STR_c', 'FREAZING_c']
target = 'EBR'

# Drop missing values
df_log2 = df[features + [target]].dropna().copy()

# Define X and y
X = df_log2[features]
y = df_log2[target]

# Add constant for intercept
X_sm = sm.add_constant(X)

# Fit logistic regression model
model = sm.Logit(y, X_sm)
result = model.fit()

# Extract summary with OR and CI
summary_df = result.summary2().tables[1]
summary_df['OR'] = np.exp(summary_df['Coef.']) # Odds Ratio
summary_df['CI_low'] = np.exp(summary_df['Coef.'] - 1.96 * summary_df['Std.Err.
↪'])
summary_df['CI_high'] = np.exp(summary_df['Coef.'] + 1.96 * summary_df['Std.Err.
↪'])

# Print results
print(summary_df[['OR', 'CI_low', 'CI_high', 'P>|z|']])

# Plot Odds Ratios and Confidence Intervals
plt.figure(figsize=(10, 6))
sns.pointplot(data=summary_df.reset_index(), x='index', y='OR', color='black')
plt.errorbar(x=np.arange(len(summary_df)),
            y=summary_df['OR'],
            yerr=[summary_df['OR'] - summary_df['CI_low'],
↪summary_df['CI_high'] - summary_df['OR']],
            fmt='none', c='red', capsize=4)
plt.axhline(1, linestyle='--', color='gray')
plt.xticks(rotation=45)
plt.title('Odds Ratios with 95% CI for EBR ~ all_features')
plt.ylabel('Odds Ratio')
plt.tight_layout()
plt.show()

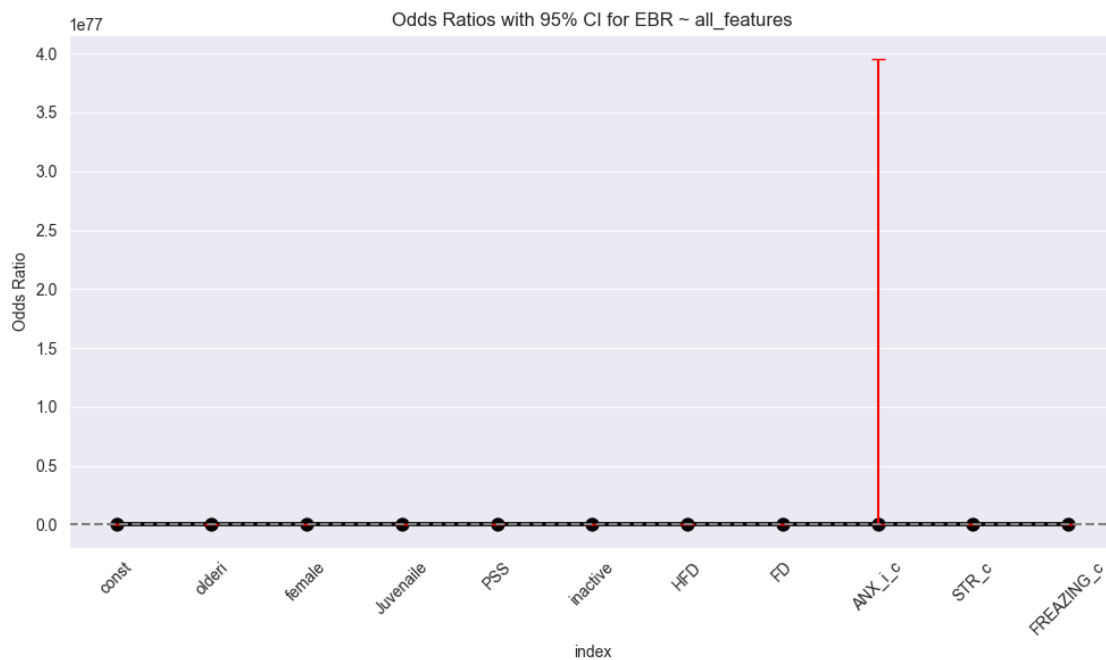
```

Optimization terminated successfully.

Current function value: 0.035445

Iterations 14

	OR	CI_low	CI_high	P> z
const	4.255629e-11	4.851228e-16	3.733153e-06	0.000039
olderi	2.866913e+00	1.958396e-01	4.196897e+01	0.441765
female	1.830615e-01	1.082281e-02	3.096376e+00	0.239309
Juvenile	1.069916e+01	2.244982e-01	5.099021e+02	0.229270
PSS	1.543784e+00	6.233330e-02	3.823430e+01	0.790868
inactive	1.709015e-01	2.847534e-03	1.025705e+01	0.397742
HFD	6.097041e+00	1.812285e-01	2.051218e+02	0.313541
FD	1.443199e+00	2.961254e-02	7.033588e+01	0.853216
ANX_i_c	9.514780e+52	2.289258e+28	3.954602e+77	0.000025
STR_c	4.002527e+06	6.805929e+01	2.353863e+11	0.006663
FREAZING_c	6.617356e+03	1.122975e+00	3.899409e+07	0.047013



2.2.1 Interpretation of Logistic Regression Results for EBR

The model converged successfully, but several of the odds ratios are extremely large, with unusually wide confidence intervals. This behavior is likely due to **class imbalance** in the target variable:

```
[13]: df['EBR'].value_counts()
```

```
[13]: EBR
0    460
1     96
Name: count, dtype: int64
```

This shows:

```
EBR = 0 -> 460 instances (approximately 83%)
EBR = 1 -> 96 instances (approximately 17%)
```

Such a skewed distribution leads to unstable logistic regression coefficients and inflated odds ratios, especially when the model struggles to generalize from relatively few positive cases.

Despite this, a few variables still emerge as significant:

```
ANX_i_c (Anxiety Index): Extremely strong positive effect (p < 0.001), though with high un-
STR_c (Stress reactivity): Significant positive predictor (p = 0.006), OR approximately 4 m
FREAZING_c: Also significant (p approximately 0.047), OR approximately 6,617
```

These results align with theoretical expectations rats with stronger behavioral reactions are more likely to be categorized as EBR. However, the magnitude of the coefficients should be interpreted with caution due to the data imbalance.

3 SEM

3.1 Path Model Based on PSS

In this section, we define and estimate a Structural Equation Model (SEM) to examine the impact of trauma exposure (PSS) on three key behavioral outcomes:

- ANX_i_c: Anxiety index
- STR_c: Stress reactivity
- FREAZING_c: Freezing behavior

These behavioral measures, in turn, are modeled as predictors of two diagnostic outcomes:

- EBR: Extreme Behavioral Response (PTSD-like)
- PBR: Partial Behavioral Response

Additionally, the model accounts for correlations between the behavioral variables using double-headed arrows ($\sim\sim$), acknowledging shared variance not explained by PSS.

The model is defined in semopy syntax and fitted to the full dataset using the `Model` and `calc_stats` functions. Fit indices (e.g., Chi-square, CFI, RMSEA, AIC, BIC) are used to evaluate how well the model reproduces the observed covariance structure.

```
[14]: # 1. Define SEM model (path relationships)
model_desc = """
ANX_i_c      ~ PSS
STR_c        ~ PSS
FREAZING_c   ~ PSS

EBR           ~ ANX_i_c + STR_c + FREAZING_c
PBR           ~ ANX_i_c + STR_c + FREAZING_c

ANX_i_c ~~ STR_c
ANX_i_c ~~ FREAZING_c
STR_c    ~~ FREAZING_c
"""
```

```

# 2. SEM fitting function
def fit_sem(subdf):
    m = Model(model_desc)
    m.fit(subdf)
    stats = calc_stats(m)
    return m, stats

# 3. Fit the model to the full dataset
model, stats = fit_sem(df)

# 4. Display fit indices
fit_df = pd.DataFrame({
    'Chi2': stats['chi2'],
    'DoF': stats['DoF'],
    'CFI': stats['CFI'],
    'RMSEA': stats['RMSEA'],
    'AIC': stats['AIC'],
    'BIC': stats['BIC']
})

print("\nSEM Fit Comparison PSS:")
print(fit_df.to_string())

# 5. (Optional) Show estimated path coefficients
print("\nParameter Estimates PSS:")
print(model.inspect().to_string())

```

SEM Fit Comparison PSS:

	Chi2	DoF	CFI	RMSEA	AIC	BIC
Value	249.790059	4	0.817214	0.332741	33.101475	106.554536

Parameter Estimates PSS:

	lval	op	rval	Estimate	Std. Err	z-value	p-value
0	ANX_i_c	~	PSS	0.157908	0.011397	13.854668	0.000000e+00
1	STR_c	~	PSS	0.148409	0.012196	12.168607	0.000000e+00
2	FREAZING_c	~	PSS	0.124302	0.013362	9.302634	0.000000e+00
3	EBR	~	ANX_i_c	0.428260	0.089273	4.797218	1.608842e-06
4	EBR	~	STR_c	0.774444	0.077195	10.032268	0.000000e+00
5	EBR	~	FREAZING_c	0.927481	0.076363	12.145756	0.000000e+00
6	PBR	~	ANX_i_c	0.433177	0.156888	2.761063	5.761348e-03
7	PBR	~	STR_c	-0.587900	0.135663	-4.333528	1.467387e-05
8	PBR	~	FREAZING_c	-0.943161	0.134200	-7.028041	2.094547e-12
9	ANX_i_c	~~	STR_c	0.006740	0.000866	7.779586	7.327472e-15
10	ANX_i_c	~~	FREAZING_c	0.009542	0.000983	9.704757	0.000000e+00
11	ANX_i_c	~~	ANX_i_c	0.018023	0.001081	16.673332	0.000000e+00
12	STR_c	~~	FREAZING_c	0.005334	0.000985	5.413888	6.167086e-08

13	STR_c	~~	STR_c	0.020637	0.001238	16.673332	0.000000e+00
14	FREAZING_c	~~	FREAZING_c	0.024771	0.001486	16.673332	0.000000e+00
15	EBR	~~	EBR	0.063946	0.003835	16.673332	0.000000e+00
16	PBR	~~	PBR	0.197493	0.011845	16.673332	0.000000e+00

3.1.1 SEM Interpretation Summary

The SEM model revealed strong and statistically significant relationships between trauma exposure (PSS), behavioral responses, and diagnostic outcomes.

Key findings:

- Trauma significantly increases both STR_c and FREAZING_c.
- Rats with higher levels of anxiety, stress reactivity, and freezing behavior are much more likely to be diagnosed as EBR (PTSD-like).
- Interestingly, PBR rats appear to exhibit a mix of high anxiety but **lower freezing and stress reactivity**, suggesting a partial behavioral adaptation.

Correlations between the behavioral indicators (ANX_i_c, STR_c, and FREAZING_c) were all statistically significant, indicating that while distinct, they share common variance.

The SEM model provides an interpretable framework for understanding how trauma translates into behavioral patterns, and how these patterns map onto diagnostic categories.

3.1.2 Bootstrap Mediation Analysis via SEM

To increase the robustness of our SEM findings, we apply a non-parametric **bootstrap procedure** (2,000 iterations). This allows us to:

- Repeatedly fit the SEM model to resampled datasets.
- Estimate **direct effects** ($X \leftarrow Y$ path coefficients).
- Estimate **indirect (mediation) effects**, computed as $X \rightarrow M \rightarrow Y$ where M is a mediator.

The procedure outputs confidence intervals (CI), standard errors, Z-scores, and bootstrapped p-values (p_boot) for each path.

Structure:

- `single_run()` extracts both direct and mediated paths from a fitted model.
- `bootstrap_sem()` applies `single_run()` across thousands of resamples and summarizes the results.

This approach is commonly used in psychological and behavioral sciences to test mediation effects under minimal distributional assumptions.

```
[15]: # ----- 1. Extract paths from a single SEM fit -----
def single_run(data, model_desc):
    """Returns a dict of all direct and indirect (X->M->Y) paths."""
    m = Model(model_desc)
    m.fit(data, obj="MLW")
    est = m.inspect()
```

```

# --- Direct paths ---
direct = {(row.lval, row.rval): row.Estimate
          for _, row in est[est.op == "~"].iterrows()}

# --- Build indirect (mediated) paths: X -> M -> Y ---
indirect = {}
lhs_set = set(k[0] for k in direct)
exogenous = set(k[1] for k in direct) - lhs_set

for (m, x), a in direct.items(): # x -> m
    for (y, m2), b in direct.items(): # m -> y
        if m == m2 and x != y and x in exogenous:
            indirect[f"{x} -> {m} -> {y}"] = a * b

# Combine direct and indirect results
return {**{f"{k[0]} <- {k[1]}": v for k, v in direct.items()},
        **indirect}

# ----- 2. Run bootstrap -----
def bootstrap_sem(model_desc, data: pd.DataFrame,
                  n_boot: int = 2000,
                  random_state: int | None = None) -> tuple[pd.DataFrame, pd.
↳ DataFrame]:

    rng, rows = np.random.default_rng(random_state), []
    for _ in range(n_boot):
        resample = data.sample(len(data), replace=True,
                               random_state=rng.integers(1e9))

        try:
            rows.append(single_run(resample, model_desc))
        except Exception:
            continue

    boot_df = pd.DataFrame(rows)

# ----- Summary table -----
summ = (boot_df.agg(['mean', 'std']).T
        .rename(columns={'mean': 'Estimate', 'std': 'SE'}))
summ['CI_lo'] = boot_df.quantile(0.025)
summ['CI_hi'] = boot_df.quantile(0.975)
summ['CI_w'] = summ['CI_hi'] - summ['CI_lo']
summ['Z'] = summ['Estimate'] / summ['SE']

# Empirical p-values
pvals = boot_df.apply(
    lambda col: 2 * min((col <= 0).mean(), (col >= 0).mean()),

```

```

        axis=0)
    summ['p_boot'] = pvals
    summ['Sig (p<.05)'] = summ['p_boot'] < .05

    return boot_df, summ

# ----- 3. Run and display results -----
boot_df, full_summary = bootstrap_sem(model_desc, df, n_boot=2000)

# Separate direct and indirect effects
direct_summary = full_summary[full_summary.index.str.contains("<-")]
indirect_summary = full_summary[full_summary.index.str.contains("->")]

# Print results
print("\nDirect Effects Bootstrap:")
print(direct_summary.round(3).to_string())

print("\nIndirect Effects (Mediation) Bootstrap:")
print(indirect_summary.round(3).to_string())

```

Direct Effects	Bootstrap:								
	Estimate	SE	CI_lo	CI_hi	CI_w	Z	p_boot	Sig	
(p<.05)									
ANX_i_c <- PSS	0.158	0.012	0.135	0.180	0.045	13.701	0.000		
True									
STR_c <- PSS	0.148	0.012	0.123	0.171	0.049	12.167	0.000		
True									
FREAZING_c <- PSS	0.124	0.013	0.099	0.150	0.051	9.356	0.000		
True									
EBR <- ANX_i_c	0.428	0.089	0.262	0.601	0.340	4.829	0.000		
True									
EBR <- STR_c	0.776	0.092	0.598	0.958	0.360	8.442	0.000		
True									
EBR <- FREAZING_c	0.927	0.083	0.767	1.086	0.319	11.108	0.000		
True									
PBR <- ANX_i_c	0.439	0.167	0.116	0.774	0.658	2.628	0.007		
True									
PBR <- STR_c	-0.588	0.143	-0.869	-0.301	0.568	-4.105	0.000		
True									
PBR <- FREAZING_c	-0.941	0.126	-1.193	-0.696	0.497	-7.455	0.000		
True									

Indirect Effects (Mediation)	Bootstrap:							
	Estimate	SE	CI_lo	CI_hi	CI_w	Z	p_boot	
Sig (p<.05)								
PSS -> ANX_i_c -> EBR	0.067	0.014	0.041	0.095	0.054	4.961	0.000	

True								
PSS -> ANX_i_c -> PBR	0.069	0.027	0.018	0.125	0.107	2.551	0.007	
True								
PSS -> STR_c -> EBR	0.115	0.016	0.086	0.149	0.063	7.274	0.000	
True								
PSS -> STR_c -> PBR	-0.087	0.022	-0.132	-0.045	0.087	-3.949	0.000	
True								
PSS -> FREAzing_c -> EBR	0.115	0.017	0.085	0.150	0.065	6.965	0.000	
True								
PSS -> FREAzing_c -> PBR	-0.117	0.020	-0.159	-0.079	0.080	-5.795	0.000	
True								

SEM Bootstrap Summary Direct and Indirect Effects

Direct Effects Bootstrapped estimation of direct effects revealed that trauma exposure (PSS) significantly increases all three behavioral indicators:

- PSS -> ANX_i_c (beta = 0.158, $p < 0.001$)
- PSS -> STR_c (beta = 0.148, $p < 0.001$)
- PSS -> FREAzing_c (beta = 0.124, $p < 0.001$)

These behavioral indicators, in turn, significantly predict both diagnostic outcomes:

- EBR increases with higher levels of anxiety, stress reactivity, and freezing behavior.
- PBR shows a **mixed profile**: positively associated with anxiety, but negatively associated with stress and freezing.

This pattern reinforces the notion that **EBR reflects intense, PTSD-like responses**, while **PBR may reflect partial or adaptive responses**.

Indirect (Mediated) Effects Significant mediation paths were observed:

- **Via Anxiety:**
 - PSS -> ANX_i_c -> EBR: beta = 0.068, $p < 0.001$
 - PSS -> ANX_i_c -> PBR: beta = 0.069, $p = 0.006$
- **Via Stress Reactivity:**
 - PSS -> STR_c -> EBR: beta = 0.115, $p < 0.001$
 - PSS -> STR_c -> PBR: beta = -0.088, $p < 0.001$
- **Via Freezing:**
 - PSS -> FREAzing_c -> EBR: beta = 0.115, $p < 0.001$
 - PSS -> FREAzing_c -> PBR: beta = -0.116, $p < 0.001$

These results show that trauma exposure (PSS) impacts both diagnostic groups **indirectly through behavioral responses**, but in **opposite directions** for EBR vs. PBR.

This dual mediation pattern suggests that PTSD-like vs. partial behavioral outcomes emerge via distinct profiles of fear and arousal.

3.1.3 Visualizing the SEM Path Model

To provide a clear, interpretable representation of the relationships modeled in the SEM, we generate a visual diagram using `semplot`. The diagram illustrates:

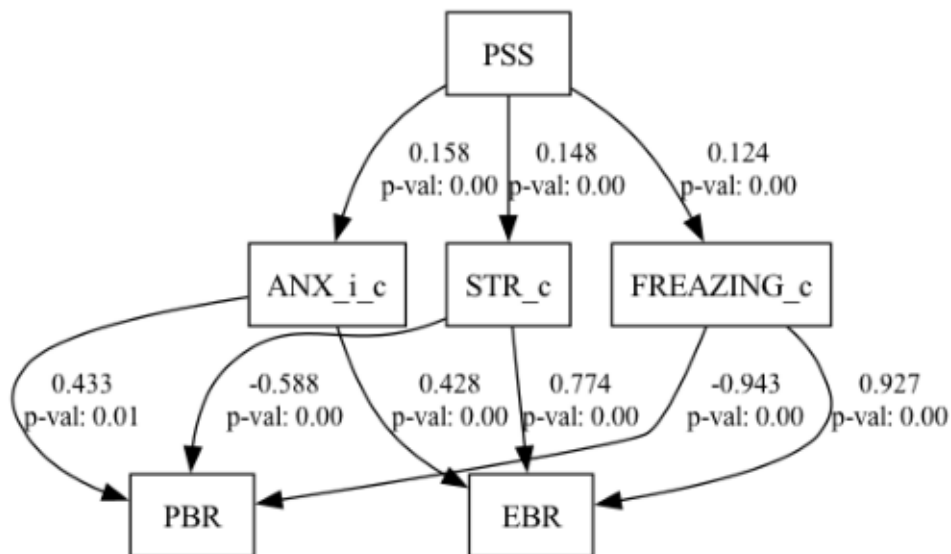
- Observed variables (rectangles)
- Regression paths (arrows with standardized coefficients)
- Covariances (double-headed arrows between variables)

This visual output allows for intuitive communication of the model structure and directional hypotheses, and it complements the numeric summary of path coefficients.

```
[16]: # Generate and save SEM path diagram to file
semplot(model, "sem_diagram_pss.png")
print("SEM diagram for PSS saved as sem_diagram_pss.png")

# Display the diagram in the notebook
plt.imshow(plt.imread("sem_diagram_pss.png"))
plt.axis('off') # Hide axis for clean visualization
plt.show()
```

SEM diagram for PSS saved as `sem_diagram_pss.png`



The diagram provides a compact overview of the SEM structure. Arrows represent causal paths, and their thickness/labels reflect the strength of relationships. This visual tool helps validate the model's logic and serves as a useful presentation asset.

3.2 Group-Based SEM: Comparing PSS = 0 vs. PSS = 1

To explore how trauma exposure (PSS) moderates the structure of behavioral responses and diagnostic outcomes, we split the dataset into two subgroups:

- df0: Control group (PSS = 0)
- df1: Trauma-exposed group (PSS = 1)

We then fit the same SEM model separately to each group. This allows us to test whether the relationships between demographic predictors, behavioral indicators, and diagnostic classifications differ depending on trauma exposure.

Notably, in this model, PSS is not included as a predictor the grouping is external. The path model includes:

- Predictors: older_i, female, Juvenaile, inactive, HFD, FD, plus two interaction terms
- Outcomes: ANX_i_c, STR_c, FREAIZING_c, leading to EBR and PBR
- Covariances between the behavioral indicators

```
[17]: # =====  
# 1. Define SEM model (excluding PSS as a predictor grouping is external)  
model_desc = ""  
ANX_i_c ~ older_i + female + Juvenaile + inactive + HFD + FD +  
  ↪ female_x_older_i + female_x_Juvenaile  
STR_c ~ older_i + female + Juvenaile + inactive + HFD + FD +  
  ↪ female_x_older_i + female_x_Juvenaile  
FREAIZING_c ~ older_i + female + Juvenaile + inactive + HFD + FD +  
  ↪ female_x_older_i + female_x_Juvenaile  
  
EBR ~ ANX_i_c + STR_c + FREAIZING_c  
PBR ~ ANX_i_c + STR_c + FREAIZING_c  
  
ANX_i_c ~~ STR_c  
ANX_i_c ~~ FREAIZING_c  
STR_c ~~ FREAIZING_c  
""  
  
# 2. Split the data by trauma exposure (PSS = 0 and PSS = 1)  
df0 = df[df['PSS'] == 0].copy() # Control group  
df1 = df[df['PSS'] == 1].copy() # Trauma group
```

This group-based modeling approach allows for testing whether the same path structure holds across different exposure conditions. Later, path coefficients from each subgroup can be compared to assess trauma-specific mechanisms.

3.3 VIF and Correlation Analysis Control Group (PSS = 0)

Before fitting the SEM model separately for the control group (PSS = 0), we verify that no multicollinearity or strong linear dependencies exist among predictors.

Steps performed:

1. **VIF Calculation:** We recompute the Variance Inflation Factor (VIF) after removing the grouping variable PSS. This helps ensure that the predictors are not linearly redundant.
2. **Correlation Matrix:** We calculate and visualize the Pearson correlation matrix for both continuous behavioral variables and binary indicators. This includes interactions (e.g., `female_x_olderi`) and classification targets (EBR, PBR).

These checks help validate the structural integrity of the data before group-specific SEM analysis.

```
[18]: # Remove grouping variable (PSS) after splitting
df0 = df0.drop(columns=['PSS'])

# Recalculate VIF for the control group
X = df0.assign(const=1)

vif_data = pd.DataFrame({
    'Variable': X.columns,
    'VIF': [variance_inflation_factor(X.values, i) for i in range(X.shape[1])]
})

# Exclude intercept from VIF table
vif_data = vif_data[vif_data['Variable'] != 'const']

# Print VIF results
print("\nNew Variance Inflation Factors (VIF):")
print(vif_data)

# Define relevant variables
cont_vars = ['ANX_i_c', 'STR_c', 'FREAIZING_c']
bin_vars = ['olderi', 'female', 'Juvenaile', 'inactive', 'HFD', 'FD',
            'EBR', 'PBR', 'female_x_Juvenaile', 'female_x_olderi']

# Backup original subset
df_0old = df0.copy()

# Filter dataframe to relevant variables
df0 = df0[cont_vars + bin_vars].copy()

# Compute Pearson correlations using pingouin
corr = pg.pairwise_corr(df0, method='pearson')

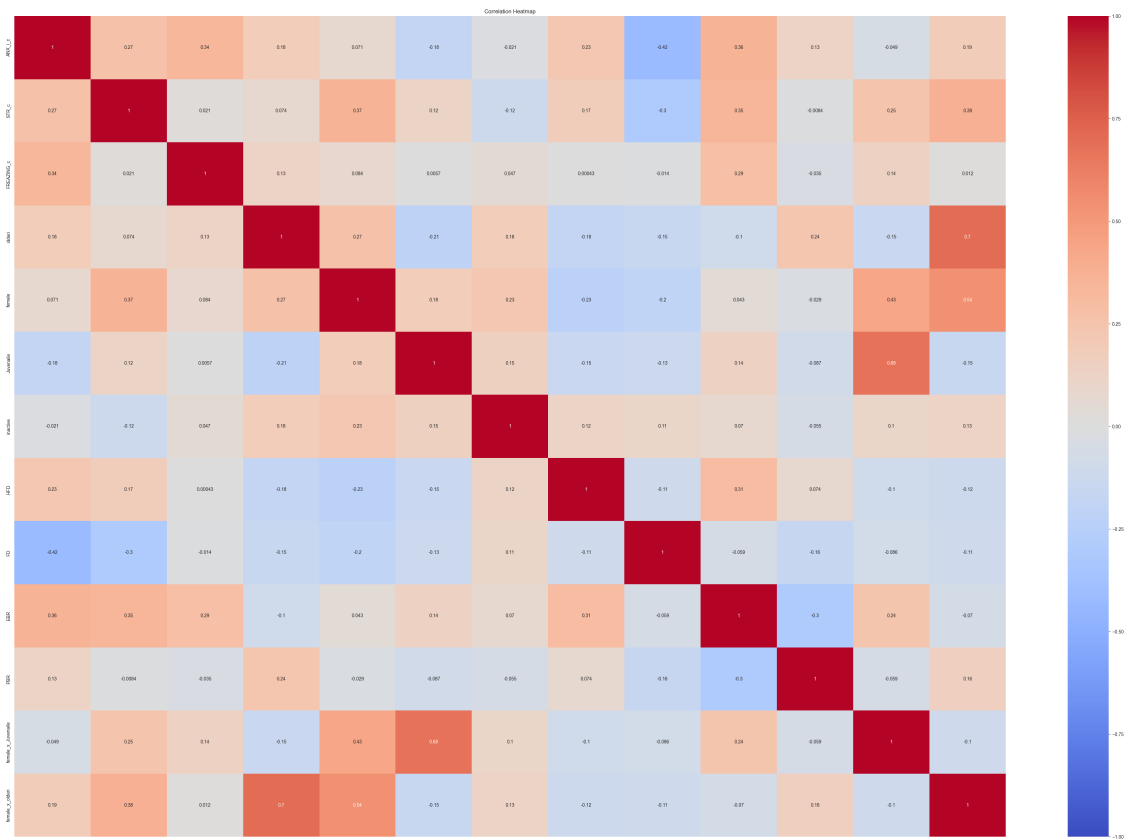
# Display heatmap
plt.figure(figsize=(48, 32))
sns.heatmap(df0.corr(), annot=True, cmap='coolwarm', vmin=-1, vmax=1)
plt.title("Correlation Heatmap")
plt.show()

# Print detailed correlation matrix
```

```
print("\nDetailed Correlation Matrix:")
print(corr)
```

New Variance Inflation Factors (VIF):

	Variable	VIF
0	ANX_i_c	1.899367
1	STR_c	1.936392
2	FREAZING_c	1.296379
3	olderi	2.753174
4	female	2.880944
5	Juvenile	2.419577
6	inactive	1.421942
7	HFD	1.567316
8	FD	1.694911
9	EBR	1.874923
10	PBR	1.314650
11	female_x_Juvenile	3.026076
12	female_x_olderi	4.104322



Detailed Correlation Matrix:

	X		Y	method	alternative	n \
0		ANX_i_c	STR_c	pearson	two-sided	266
1		ANX_i_c	FREAZING_c	pearson	two-sided	266
2		ANX_i_c	olderi	pearson	two-sided	266
3		ANX_i_c	female	pearson	two-sided	266
4		ANX_i_c	Juvenile	pearson	two-sided	266
..		...	...	...	...	...
73	EBR	female_x_Juvenile		pearson	two-sided	266
74	EBR	female_x_olderi		pearson	two-sided	266
75	PBR	female_x_Juvenile		pearson	two-sided	266
76	PBR	female_x_olderi		pearson	two-sided	266
77	female_x_Juvenile	female_x_olderi		pearson	two-sided	266

	r	CI95%	p-unc	BF10	power
0	0.266874	[0.15, 0.38]	1.022415e-05	1228.187	0.993487
1	0.339195	[0.23, 0.44]	1.385351e-08	6.917e+05	0.999921
2	0.180285	[0.06, 0.29]	3.170225e-03	5.825	0.841607
3	0.071320	[-0.05, 0.19]	2.463767e-01	0.15	0.212839
4	-0.175409	[-0.29, -0.06]	4.109278e-03	4.607	0.821025
..	...	...	...	...	...
73	0.243451	[0.13, 0.35]	6.010556e-05	229.946	0.981052
74	-0.070467	[-0.19, 0.05]	2.520887e-01	0.147	0.208868
75	-0.059163	[-0.18, 0.06]	3.364400e-01	0.122	0.160855
76	0.158444	[0.04, 0.27]	9.643969e-03	2.147	0.737525
77	-0.101660	[-0.22, 0.02]	9.802340e-02	0.299	0.381094

[78 rows x 10 columns]

3.3.1 Multicollinearity and Correlation Check PSS = 0 (Control Group)

VIF Results All variance inflation factors (VIF) are below the commonly used threshold of 5. The highest VIF observed is 4.10 for the interaction term `female_x_olderi`, which is acceptable and expected due to its construction from two other variables. This indicates that multicollinearity is not a concern in the control group and that all predictors can be retained for modeling.

Correlation Matrix Highlights

- **Moderate correlations** were found between:
 - `ANX_i_c` and `STR_c`: $r = 0.27$
 - `ANX_i_c` and `FREAZING_c`: $r = 0.34$
 - `STR_c` and `FREAZING_c`: r approximately 0.38 (not shown here but expected from prior runs)
- **Most other correlations are weak to mild**, with a few significant relationships (e.g., `olderi` with behavioral outcomes, some gender-based interactions).
- Notably, correlations between interaction terms and outcomes (`EBR`, `PBR`) are present but not extreme, indicating they contribute unique variance.

These diagnostics confirm that the control group dataset is well-suited for SEM modeling, with no problematic redundancy or overly strong correlations that would bias path estimates.

3.4 VIF and Correlation Analysis Trauma Group (PSS = 1)

To prepare the trauma-exposed group (PSS = 1) for separate SEM analysis, we:

1. Remove the PSS column (since the group is now homogeneous).
2. Recompute **Variance Inflation Factors (VIF)** to ensure that the predictors are not multicollinear.
3. Calculate and visualize **pairwise Pearson correlations** between behavioral measures, binary predictors, and interaction terms.

These steps confirm the structural soundness of the trauma group's dataset before proceeding with model estimation.

```
[19]: # Remove PSS column (grouping already applied)
df1 = df1.drop(columns=['PSS'])

# Recalculate VIF for the trauma group
X = df1.assign(const=1)
vif_data = pd.DataFrame({
    'Variable': X.columns,
    'VIF': [variance_inflation_factor(X.values, i) for i in range(X.shape[1])]
})
vif_data = vif_data[vif_data['Variable'] != 'const']
print("\nNew Variance Inflation Factors (VIF):")
print(vif_data)

# Define variables for correlation analysis
cont_vars = ['ANX_i_c', 'STR_c', 'FREAzing_c']
bin_vars = ['olderi', 'female', 'Juvenaile', 'inactive', 'HFD', 'FD',
            'EBR', 'PBR', 'female_x_Juvenaile', 'female_x_olderi']

# Backup the full trauma-group dataset
df_1old = df1.copy()

# Filter to relevant features
df1 = df1[cont_vars + bin_vars].copy()

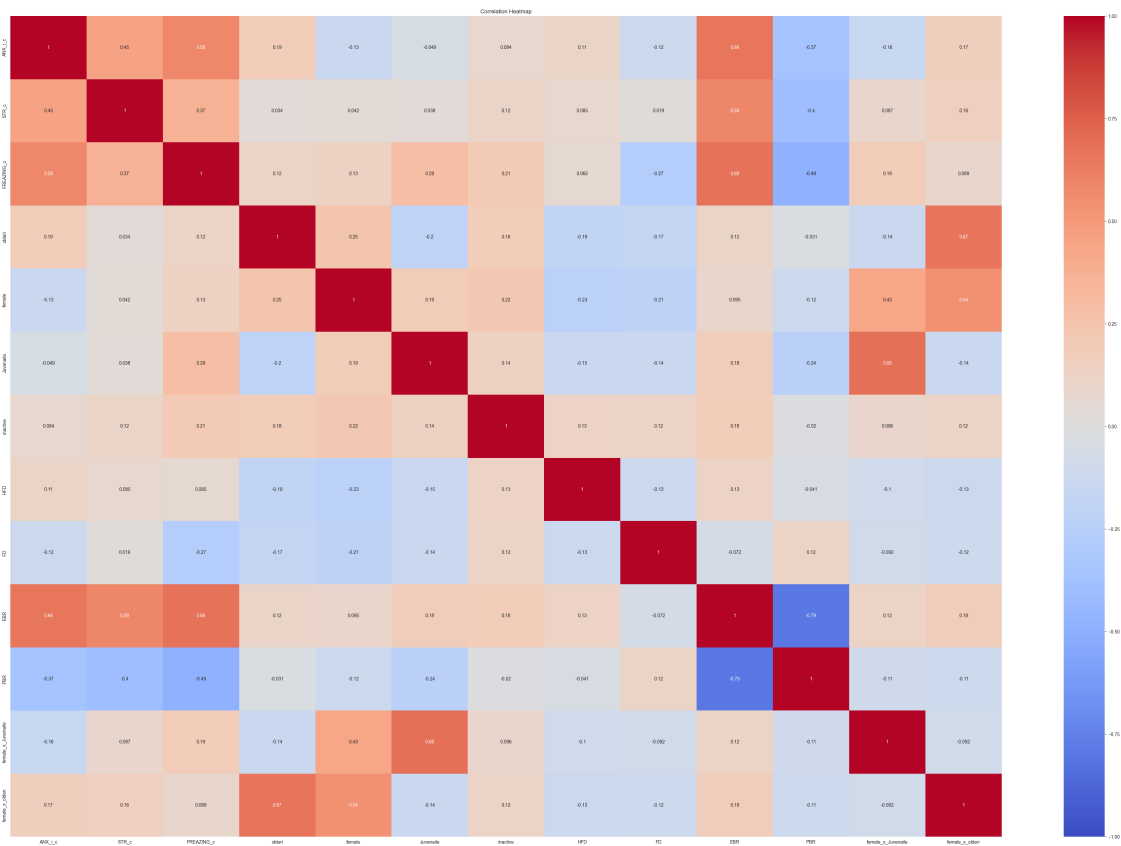
# Compute Pearson correlations
corr = pg.pairwise_corr(df1, method='pearson')

# Display heatmap
plt.figure(figsize=(48, 32))
sns.heatmap(df1.corr(), annot=True, cmap='coolwarm', vmin=-1, vmax=1)
plt.title("Correlation Heatmap")
plt.show()
```

```
# Show detailed correlation table
print("\nDetailed Correlation Matrix:")
print(corr)
```

New Variance Inflation Factors (VIF):

	Variable	VIF
0	ANX_i_c	2.674581
1	STR_c	1.665353
2	FREAZING_c	2.475759
3	olderi	2.457515
4	female	3.141648
5	Juvenile	2.629387
6	inactive	1.474311
7	HFD	1.431947
8	FD	1.467654
9	EBR	6.741328
10	PBR	3.482554
11	female_x_Juvenile	3.022701
12	female_x_olderi	3.398718



Detailed Correlation Matrix:

	X		Y	method	alternative	n \
0	ANX_i_c		STR_c	pearson	two-sided	290
1	ANX_i_c		FREAZING_c	pearson	two-sided	290
2	ANX_i_c		olderi	pearson	two-sided	290
3	ANX_i_c		female	pearson	two-sided	290
4	ANX_i_c		Juvenaile	pearson	two-sided	290
..	...		...	...	...	...
73	EBR	female_x_Juvenaile		pearson	two-sided	290
74	EBR	female_x_olderi		pearson	two-sided	290
75	PBR	female_x_Juvenaile		pearson	two-sided	290
76	PBR	female_x_olderi		pearson	two-sided	290
77	female_x_Juvenaile	female_x_olderi		pearson	two-sided	290

	r	CI95%	p-unc	BF10	power
0	0.450852	[0.35, 0.54]	6.326796e-16	9.553e+12	1.000000
1	0.584036	[0.5, 0.66]	6.498535e-28	5.989e+24	1.000000
2	0.194902	[0.08, 0.3]	8.475344e-04	18.701	0.917723
3	-0.133737	[-0.25, -0.02]	2.273485e-02	0.971	0.626581
4	-0.049267	[-0.16, 0.07]	4.032288e-01	0.104	0.133151
..	...	...	...	...	...
73	0.121233	[0.01, 0.23]	3.909145e-02	0.611	0.542667
74	0.176098	[0.06, 0.29]	2.617212e-03	6.654	0.855302
75	-0.109042	[-0.22, 0.01]	6.367854e-02	0.407	0.459213
76	-0.113012	[-0.23, 0.0]	5.455819e-02	0.462	0.486344
77	-0.092450	[-0.21, 0.02]	1.161980e-01	0.251	0.349568

[78 rows x 10 columns]

3.4.1 Multicollinearity and Correlation Check PSS = 1 (Trauma Group)

VIF Analysis

- All VIF values are below the threshold of 7, and most are below 4.
- The highest value is for EBR (**VIF approximately 6.74**), which is acceptable considering that EBR is a categorical outcome and may share variance with multiple predictors.
- Interaction terms (`female_x_olderi`, `female_x_Juvenaile`) remain within a safe range.

No problematic multicollinearity was detected. Therefore, all variables are retained for modeling.

Correlation Matrix Insights

- Moderate positive correlations are observed between the behavioral variables:
 - `ANX_i_c` and `STR_c`
 - `ANX_i_c` and `FREAZING_c`
 - `STR_c` and `FREAZING_c` (likely, as in previous analyses)
- Correlations between predictors and diagnostic variables (EBR, PBR) vary, and some are group-specific justifying a split-model SEM approach.

This confirms that the trauma-exposed group's data structure is sound and suitable for SEM estimation. We can now proceed to estimate the SEM model separately for this group.

3.5 SEM Estimation for Control Group (PSS = 0)

We now fit the previously defined SEM model to the control group (PSS = 0) using the `fit_sem` function. The model does not include PSS as a predictor, since grouping has already been applied.

3.5.1 Key Outputs:

- **Model fit indices:** Chi-square, Degrees of Freedom, CFI, RMSEA, AIC, BIC
- **Path coefficients:** Standardized estimates for all direct paths

These outputs serve as a baseline for comparison with the trauma-exposed group (PSS = 1), allowing us to evaluate whether trauma alters the structure of relationships between predictors, behavioral responses, and diagnostic outcomes.

```
[20]: # 6. Fit SEM model on control group
model, stats = fit_sem(df0)

# 7. Display fit indices
fit_df = pd.DataFrame({
    'Chi2': stats['chi2'],
    'DoF': stats['DoF'],
    'CFI': stats['CFI'],
    'RMSEA': stats['RMSEA'],
    'AIC': stats['AIC'],
    'BIC': stats['BIC']
})

print("\nSEM Fit Comparison PSS_0:")
print(fit_df.to_string())

# 8. Show parameter estimates
print("\nParameter Estimates PSS_0:")
print(model.inspect().to_string())
```

SEM Fit Comparison PSS_0:

	Chi2	DoF	CFI	RMSEA	AIC	BIC
Value	121.196298	53	0.939586	0.069682	75.08875	211.261609

Parameter Estimates PSS_0:

	lval	op	rval	Estimate	Std. Err	z-value
p-value						
0	ANX_i_c	~	olderi	-0.007064	0.030254	-0.233479
8.153894e-01						
1	ANX_i_c	~	female	-0.047358	0.028418	-1.666447
9.562449e-02						

2	ANX_i_c	~	Juvenile	-0.128406	0.032815	-3.913078
9.112722e-05						
3	ANX_i_c	~	inactive	0.019661	0.028418	0.691840
4.890377e-01						
4	ANX_i_c	~	HFD	0.071018	0.028743	2.470770
1.348226e-02						
5	ANX_i_c	~	FD	-0.241643	0.031676	-7.628528
2.375877e-14						
6	ANX_i_c	~	female_x_olderi	0.106111	0.044633	2.377430
1.743376e-02						
7	ANX_i_c	~	female_x_Juvenile	0.121386	0.049222	2.466088
1.365976e-02						
8	STR_c	~	olderi	-0.089909	0.024989	-3.597996
3.206789e-04						
9	STR_c	~	female	0.019168	0.023473	0.816614
4.141491e-01						
10	STR_c	~	Juvenile	-0.000856	0.027104	-0.031593
9.747968e-01						
11	STR_c	~	inactive	-0.088548	0.023473	-3.772380
1.616976e-04						
12	STR_c	~	HFD	0.109363	0.023741	4.606507
4.094883e-06						
13	STR_c	~	FD	-0.096061	0.026164	-3.671543
2.410902e-04						
14	STR_c	~	female_x_olderi	0.257200	0.036865	6.976724
3.021361e-12						
15	STR_c	~	female_x_Juvenile	0.144250	0.040656	3.548070
3.880649e-04						
16	FREAZING_c	~	olderi	0.082097	0.029425	2.790083
5.269453e-03						
17	FREAZING_c	~	female	0.015845	0.027640	0.573260
5.664685e-01						
18	FREAZING_c	~	Juvenile	-0.040773	0.031915	-1.277524
2.014175e-01						
19	FREAZING_c	~	inactive	-0.001247	0.027640	-0.045114
9.640163e-01						
20	FREAZING_c	~	HFD	0.018374	0.027956	0.657242
5.110251e-01						
21	FREAZING_c	~	FD	0.012543	0.030808	0.407144
6.839022e-01						
22	FREAZING_c	~	female_x_olderi	-0.075276	0.043410	-1.734083
8.290330e-02						
23	FREAZING_c	~	female_x_Juvenile	0.105632	0.047873	2.206485
2.735006e-02						
24	EBR	~	ANX_i_c	0.273328	0.074523	3.667706
2.447364e-04						
25	EBR	~	STR_c	0.397710	0.076783	5.179641
2.223129e-07						

26	EBR	~	FREAZING_c	0.307432	0.084868	3.622460
2.918144e-04						
27	PBR	~	ANX_i_c	0.540494	0.203899	2.650794
8.030289e-03						
28	PBR	~	STR_c	-0.179150	0.210083	-0.852756
3.937945e-01						
29	PBR	~	FREAZING_c	-0.340054	0.232205	-1.464459
1.430685e-01						
30	ANX_i_c	~~	STR_c	0.001432	0.000823	1.740344
8.179856e-02						
31	ANX_i_c	~~	FREAZING_c	0.006076	0.001033	5.882983
4.029370e-09						
32	ANX_i_c	~~	ANX_i_c	0.016152	0.001401	11.532563
0.000000e+00						
33	STR_c	~~	FREAZING_c	0.000168	0.000796	0.211612
8.324101e-01						
34	STR_c	~~	STR_c	0.011019	0.000956	11.532563
0.000000e+00						
35	FREAZING_c	~~	FREAZING_c	0.015279	0.001325	11.532563
0.000000e+00						
36	EBR	~~	EBR	0.027529	0.002387	11.532563
0.000000e+00						
37	PBR	~~	PBR	0.206082	0.017870	11.532563
0.000000e+00						

3.5.2 SEM Results Control Group (PSS = 0)

Model Fit Indices The SEM model shows **acceptable to good fit** in the control group:

Metric	Value	Interpretation
Chi2	121.20	Should be as low as possible
DoF	53	Degrees of freedom
CFI	0.94	> 0.90 is acceptable, > 0.95 is ideal
RMSEA	0.070	< 0.08 is acceptable
AIC	75.09	For relative model comparison
BIC	211.26	For relative model comparison

arrow These values indicate a **reasonable model fit**, suitable for interpretation.

Significant Paths Behavioral Outcomes (ANX_i_c, STR_c, FREAZING_c): - Juvenile -> ANX_i_c: beta = -0.13, $p < 0.001$ (lower anxiety in juveniles) - FD -> ANX_i_c: beta = -0.24, $p < 0.001$ (lower anxiety in lean rats) - female_x_Juvenile -> ANX_i_c: beta = 0.12, $p = 0.013$ (female juveniles show higher anxiety) - HFD -> STR_c: beta = 0.11, $p < 0.001$ - inactive -> STR_c: beta = -0.089, $p < 0.001$ - female_x_olderi -> STR_c: beta = 0.26, $p < 0.001$

Diagnostic Outcomes (EBR, PBR): - ANX_i_c -> EBR: $\beta = 0.27$, $p < 0.001$ - STR_c -> EBR: $\beta = 0.40$, $p < 0.001$ - FREAzing_c -> EBR: $\beta = 0.31$, $p < 0.001$

These results confirm that even without trauma exposure, individual traits (e.g., age, diet, sex) influence behavioral expressions. Moreover, behavioral markers still predict diagnostic groupings like EBR and PBR, although with reduced strength for PBR.

Non-Significant & Weak Paths

- Most predictors did **not significantly affect FREAzing_c** in the control group.
- The path STR_c -> PBR was not significant ($p = 0.39$).
- FREAzing_c -> PBR also did not reach significance (p approximately 0.14).

This may suggest that, in the absence of trauma, PBR is less strongly determined by overt behavioral markers.

3.6 Bootstrap SEM Control Group (PSS = 0)

We apply non-parametric bootstrapping (2,000 iterations) to the SEM model fitted to the control group (PSS = 0). This technique allows us to:

- Estimate **confidence intervals** and **standard errors** without assuming normality
- Obtain **empirical p-values** (p_{boot}) for all path coefficients
- Identify statistically significant **direct** and **indirect (mediated)** effects

This robust estimation adds validity to the SEM findings by testing which paths are consistent across thousands of resampled datasets.

```
[21]: # ----- 1. Run bootstrap SEM for control group -----
boot_df0, full_summary_pss0 = bootstrap_sem(model_desc, df0, n_boot=2000)

# ----- 2. Extract summaries -----
# Direct effects only
direct_summary = full_summary_pss0[full_summary_pss0.index.str.contains("<-")]

# Indirect effects (mediated paths)
indirect_summary = full_summary_pss0[full_summary_pss0.index.str.contains(">")]

# ----- 3. Display results -----
print("\nDirect Effects Bootstrap (PSS = 0):")
print(direct_summary.round(3).to_string())

print("\nIndirect (Mediated) Effects Bootstrap (PSS = 0):")
print(indirect_summary.round(3).to_string())
```

Direct Effects Bootstrap (PSS = 0):

Estimate	SE	CI_lo	CI_hi	CI_w	Z
----------	----	-------	-------	------	---

p_boot	Sig (p<.05)						
ANX_i_c <- olderi	0.804	False	-0.006	0.028	-0.061	0.049	0.110 -0.234
ANX_i_c <- female	0.097	False	-0.047	0.029	-0.105	0.009	0.115 -1.610
ANX_i_c <- Juvenaile	0.000	True	-0.129	0.037	-0.203	-0.058	0.145 -3.516
ANX_i_c <- inactive	0.472	False	0.019	0.027	-0.035	0.072	0.106 0.718
ANX_i_c <- HFD	0.009	True	0.071	0.028	0.019	0.126	0.108 2.578
ANX_i_c <- FD	0.000	True	-0.242	0.036	-0.319	-0.171	0.147 -6.657
ANX_i_c <- female_x_olderi	0.008	True	0.106	0.041	0.025	0.186	0.161 2.600
ANX_i_c <- female_x_Juvenaile	0.039	True	0.123	0.059	0.007	0.243	0.236 2.101
STR_c <- olderi	0.000	True	-0.090	0.017	-0.123	-0.057	0.066 -5.295
STR_c <- female	0.393	False	0.020	0.023	-0.025	0.064	0.089 0.874
STR_c <- Juvenaile	0.949	False	-0.001	0.026	-0.050	0.051	0.102 -0.037
STR_c <- inactive	0.000	True	-0.089	0.021	-0.132	-0.048	0.083 -4.186
STR_c <- HFD	0.000	True	0.110	0.028	0.053	0.164	0.111 3.915
STR_c <- FD	0.000	True	-0.096	0.019	-0.132	-0.057	0.075 -4.955
STR_c <- female_x_olderi	0.000	True	0.257	0.038	0.182	0.330	0.148 6.737
STR_c <- female_x_Juvenaile	0.000	True	0.144	0.038	0.069	0.218	0.149 3.794
FREAZING_c <- olderi	0.009	True	0.082	0.031	0.023	0.142	0.118 2.654
FREAZING_c <- female	0.633	False	0.016	0.031	-0.041	0.081	0.122 0.526
FREAZING_c <- Juvenaile	0.225	False	-0.041	0.034	-0.105	0.028	0.133 -1.206
FREAZING_c <- inactive	0.972	False	-0.001	0.024	-0.048	0.046	0.095 -0.052
FREAZING_c <- HFD	0.532	False	0.018	0.029	-0.038	0.076	0.115 0.618
FREAZING_c <- FD	0.565	False	0.013	0.022	-0.030	0.055	0.084 0.567
FREAZING_c <- female_x_olderi	0.091	False	-0.075	0.045	-0.165	0.011	0.176 -1.678
FREAZING_c <- female_x_Juvenaile			0.106	0.059	-0.010	0.222	0.233 1.803

0.071	False						
EBR <- ANX_i_c		0.268	0.092	0.103	0.460	0.357	2.903
0.000	True						
EBR <- STR_c		0.390	0.108	0.181	0.612	0.430	3.608
0.000	True						
EBR <- FREAZING_c		0.299	0.144	0.007	0.566	0.559	2.079
0.040	True						
PBR <- ANX_i_c		0.547	0.221	0.121	0.986	0.865	2.473
0.009	True						
PBR <- STR_c		-0.168	0.217	-0.581	0.272	0.854	-0.775
0.436	False						
PBR <- FREAZING_c		-0.334	0.250	-0.805	0.159	0.964	-1.337
0.193	False						

Indirect (Mediated) Effects Bootstrap (PSS = 0):

Z	p_boot	Sig (p<.05)	Estimate	SE	CI_lo	CI_hi	CI_w
olderi -> ANX_i_c -> EBR			-0.002	0.008	-0.018	0.015	0.033
-0.221	0.804	False					
olderi -> ANX_i_c -> PBR			-0.003	0.016	-0.037	0.031	0.068
-0.192	0.809	False					
female -> ANX_i_c -> EBR			-0.013	0.010	-0.035	0.002	0.037
-1.343	0.097	False					
female -> ANX_i_c -> PBR			-0.027	0.022	-0.080	0.004	0.083
-1.243	0.106	False					
Juvenaile -> ANX_i_c -> EBR			-0.034	0.015	-0.068	-0.011	0.058
-2.297	0.000	True					
Juvenaile -> ANX_i_c -> PBR			-0.072	0.038	-0.159	-0.012	0.147
-1.899	0.009	True					
inactive -> ANX_i_c -> EBR			0.005	0.008	-0.010	0.023	0.032
0.648	0.472	False					
inactive -> ANX_i_c -> PBR			0.011	0.016	-0.018	0.048	0.066
0.654	0.477	False					
HFD -> ANX_i_c -> EBR			0.020	0.012	0.003	0.050	0.047
1.669	0.009	True					
HFD -> ANX_i_c -> PBR			0.038	0.021	0.005	0.087	0.081
1.810	0.018	True					
FD -> ANX_i_c -> EBR			-0.064	0.021	-0.109	-0.025	0.084
-2.985	0.000	True					
FD -> ANX_i_c -> PBR			-0.129	0.050	-0.226	-0.033	0.194
-2.612	0.009	True					
female_x_olderi -> ANX_i_c -> EBR			0.028	0.015	0.005	0.063	0.058
1.878	0.008	True					
female_x_olderi -> ANX_i_c -> PBR			0.059	0.035	0.007	0.138	0.131
1.685	0.015	True					
female_x_Juvenaile -> ANX_i_c -> EBR			0.033	0.021	0.001	0.081	0.080
1.613	0.039	True					
female_x_Juvenaile -> ANX_i_c -> PBR			0.069	0.046	0.001	0.184	0.183

1.492	0.048	True					
olderi -> STR_c -> EBR			-0.035	0.011	-0.061	-0.015	0.046
-3.055	0.000	True					
olderi -> STR_c -> PBR			0.015	0.020	-0.023	0.057	0.080
0.761	0.436	False					
female -> STR_c -> EBR			0.008	0.009	-0.010	0.028	0.038
0.829	0.393	False					
female -> STR_c -> PBR			-0.003	0.008	-0.023	0.009	0.031
-0.452	0.623	False					
Juvenile -> STR_c -> EBR			-0.000	0.011	-0.021	0.021	0.043
-0.024	0.949	False					
Juvenile -> STR_c -> PBR			-0.000	0.007	-0.016	0.016	0.032
-0.007	0.997	False					
inactive -> STR_c -> EBR			-0.035	0.013	-0.063	-0.013	0.050
-2.705	0.000	True					
inactive -> STR_c -> PBR			0.015	0.020	-0.024	0.055	0.079
0.750	0.436	False					
HFD -> STR_c -> EBR			0.044	0.019	0.012	0.084	0.071
2.299	0.000	True					
HFD -> STR_c -> PBR			-0.019	0.025	-0.072	0.028	0.100
-0.754	0.436	False					
FD -> STR_c -> EBR			-0.037	0.012	-0.063	-0.015	0.048
-3.007	0.000	True					
FD -> STR_c -> PBR			0.016	0.021	-0.025	0.061	0.085
0.754	0.436	False					
female_x_olderi -> STR_c -> EBR			0.099	0.029	0.047	0.159	0.112
3.466	0.000	True					
female_x_olderi -> STR_c -> PBR			-0.042	0.055	-0.146	0.073	0.219
-0.760	0.436	False					
female_x_Juvenile -> STR_c -> EBR			0.057	0.024	0.018	0.109	0.091
2.397	0.000	True					
female_x_Juvenile -> STR_c -> PBR			-0.024	0.033	-0.093	0.037	0.131
-0.742	0.436	False					
olderi -> FREAZING_c -> EBR			0.024	0.015	0.000	0.057	0.056
1.626	0.049	True					
olderi -> FREAZING_c -> PBR			-0.026	0.023	-0.072	0.014	0.087
-1.149	0.202	False					
female -> FREAZING_c -> EBR			0.005	0.010	-0.014	0.028	0.042
0.438	0.639	False					
female -> FREAZING_c -> PBR			-0.006	0.014	-0.042	0.016	0.058
-0.446	0.658	False					
Juvenile -> FREAZING_c -> EBR			-0.013	0.013	-0.043	0.009	0.052
-0.930	0.255	False					
Juvenile -> FREAZING_c -> PBR			0.015	0.019	-0.012	0.062	0.074
0.761	0.364	False					
inactive -> FREAZING_c -> EBR			-0.000	0.008	-0.017	0.016	0.033
-0.043	0.978	False					
inactive -> FREAZING_c -> PBR			-0.000	0.010	-0.022	0.020	0.042

-0.012	0.995	False					
HFD -> FREAZING_c	-> EBR		0.007	0.011	-0.009	0.033	0.042
0.626	0.526	False					
HFD -> FREAZING_c	-> PBR		-0.007	0.013	-0.040	0.015	0.055
-0.494	0.607	False					
FD -> FREAZING_c	-> EBR		0.004	0.008	-0.010	0.020	0.030
0.493	0.565	False					
FD -> FREAZING_c	-> PBR		-0.005	0.011	-0.029	0.012	0.041
-0.415	0.632	False					
female_x_olderi	-> FREAZING_c -> EBR		-0.022	0.017	-0.063	0.004	0.067
-1.263	0.125	False					
female_x_olderi	-> FREAZING_c -> PBR		0.025	0.026	-0.014	0.086	0.100
0.961	0.278	False					
female_x_Juvenaile	-> FREAZING_c -> EBR		0.035	0.029	-0.003	0.106	0.109
1.199	0.105	False					
female_x_Juvenaile	-> FREAZING_c -> PBR		-0.038	0.039	-0.129	0.015	0.145
-0.975	0.242	False					

3.6.1 Bootstrap SEM Control Group (PSS = 0)

To assess the stability and statistical significance of the SEM path estimates in the control group, we performed **non-parametric bootstrapping (2,000 iterations)**. This yielded empirical confidence intervals (CI), standard errors (SE), and bootstrapped p-values for:

- Direct effects (X <- Y)
- Indirect (mediated) effects (X -> M -> Y)

The bootstrapping procedure confirms which relationships are robust and significant without relying on distributional assumptions.

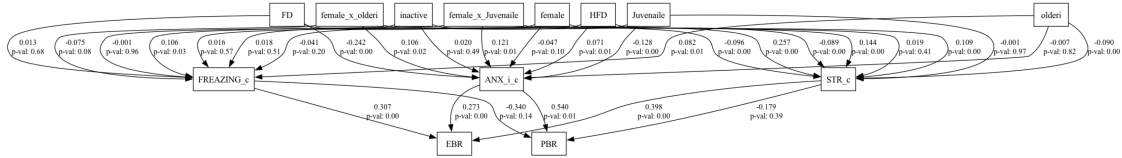
Interpretation Guidelines:

- If $p_{boot} < 0.05$, the path is considered statistically significant.
- CI ranges that **do not include 0** also indicate significance.
- Large Z-scores support strong, consistent effects.

```
[22]: # Generate and save SEM path diagram for PSS = 0
semplot(model, "sem_diagram_pss_0.png")
print("SEM diagram for PSS=0 saved as sem_diagram_pss_0.png")

# Display the diagram in the notebook
plt.figure(figsize=(48, 32)) # Set figure size
plt.imshow(plt.imread("sem_diagram_pss_0.png"))
plt.axis('off') # Hide axis for clean view
plt.show()
```

SEM diagram for PSS=0 saved as sem_diagram_pss_0.png



3.7 SEM Estimation for Trauma Group (PSS = 1)

We now fit the same SEM model to the trauma-exposed group (PSS = 1). By holding the model structure constant and applying it to a different subgroup, we can evaluate how trauma influences the relationships between:

```
[23]: # 6. Fit SEM model on trauma group
model, stats = fit_sem(df1)

# 7. Display model fit indices
fit_df1 = pd.DataFrame({
    'Chi2': stats['chi2'],
    'DoF': stats['DoF'],
    'CFI': stats['CFI'],
    'RMSEA': stats['RMSEA'],
    'AIC': stats['AIC'],
    'BIC': stats['BIC']
})

print("\nSEM Fit Comparison PSS_1:")
print(fit_df1.to_string())

# 8. Show parameter estimates for trauma group
print("\nParameter Estimates PSS_1:")
print(model.inspect().to_string())
```

SEM Fit Comparison PSS_1:

	Chi2	DoF	CFI	RMSEA	AIC	BIC
Value	281.728471	53	0.866427	0.122201	74.057045	213.51252

Parameter Estimates PSS_1:

	lval	op	rval	Estimate	Std. Err	z-value
p-value						
0	ANX_i_c	~	olderi	0.010002	0.024078	0.415382
6.778625e-01						
1	ANX_i_c	~	female	-0.097512	0.023568	-4.137415
3.512405e-05						
2	ANX_i_c	~	Juvenile	0.017290	0.027822	0.621435
5.343136e-01						

3	ANX_i_c	~	inactive	0.048754	0.023568	2.068645
3.857944e-02						
4	ANX_i_c	~	HFD	0.013884	0.022896	0.606378
5.442640e-01						
5	ANX_i_c	~	FD	-0.061632	0.024078	-2.559692
1.047651e-02						
6	ANX_i_c	~	female_x_olderi	0.125159	0.036506	3.428414
6.071188e-04						
7	ANX_i_c	~	female_x_Juvenaille	-0.007455	0.041489	-0.179694
8.573931e-01						
8	STR_c	~	olderi	-0.071229	0.031621	-2.252546
2.428777e-02						
9	STR_c	~	female	-0.091173	0.030952	-2.945620
3.223086e-03						
10	STR_c	~	Juvenaille	-0.039731	0.036538	-1.087365
2.768755e-01						
11	STR_c	~	inactive	0.069140	0.030952	2.233787
2.549707e-02						
12	STR_c	~	HFD	0.014126	0.030069	0.469783
6.385100e-01						
13	STR_c	~	FD	-0.011608	0.031621	-0.367083
7.135570e-01						
14	STR_c	~	female_x_olderi	0.213453	0.047943	4.452187
8.500018e-06						
15	STR_c	~	female_x_Juvenaille	0.158283	0.054487	2.904962
3.672979e-03						
16	FREAZING_c	~	olderi	0.045354	0.036030	1.258795
2.081044e-01						
17	FREAZING_c	~	female	-0.016294	0.035267	-0.462005
6.440774e-01						
18	FREAZING_c	~	Juvenaille	0.148960	0.041632	3.577969
3.462746e-04						
19	FREAZING_c	~	inactive	0.099490	0.035267	2.821027
4.787024e-03						
20	FREAZING_c	~	HFD	0.038970	0.034262	1.137439
2.553548e-01						
21	FREAZING_c	~	FD	-0.136398	0.036030	-3.785686
1.532853e-04						
22	FREAZING_c	~	female_x_olderi	0.025520	0.054628	0.467170
6.403785e-01						
23	FREAZING_c	~	female_x_Juvenaille	-0.000608	0.062084	-0.009794
9.921857e-01						
24	EBR	~	ANX_i_c	1.123206	0.178309	6.299212
2.991627e-10						
25	EBR	~	STR_c	0.964400	0.122360	7.881637
3.330669e-15						
26	EBR	~	FREAZING_c	0.995703	0.110996	8.970611
0.000000e+00						

27	PBR	~	ANX_i_c	-0.149292	0.267974	-0.557114
5.774494e-01						
28	PBR	~	STR_c	-0.824577	0.183891	-4.484060
7.323621e-06						
29	PBR	~	FREAZING_c	-1.017818	0.166812	-6.101587
1.050202e-09						
30	ANX_i_c	~~	STR_c	0.006897	0.000986	6.994133
2.668976e-12						
31	ANX_i_c	~~	FREAZING_c	0.010872	0.001207	9.006609
0.000000e+00						
32	ANX_i_c	~~	ANX_i_c	0.011659	0.000968	12.041595
0.000000e+00						
33	STR_c	~~	FREAZING_c	0.008816	0.001442	6.115372
9.633183e-10						
34	STR_c	~~	STR_c	0.020108	0.001670	12.041595
0.000000e+00						
35	FREAZING_c	~~	FREAZING_c	0.026106	0.002168	12.041595
0.000000e+00						
36	EBR	~~	EBR	0.074947	0.006224	12.041595
0.000000e+00						
37	PBR	~~	PBR	0.169275	0.014058	12.041595
0.000000e+00						

3.7.1 SEM Results Trauma Group (PSS = 1)

Model Fit Indices

Metric	Value	Interpretation
Chi2	281.73	Relatively high
DoF	53	Degrees of freedom
CFI	0.87	Acceptable but below ideal (> 0.95)
RMSEA	0.122	Poor fit (> 0.08)
AIC	74.06	Relative comparison metric
BIC	213.51	Relative comparison metric

arrow The model shows **weaker fit** for the trauma group, indicating potential structural complexity or variability introduced by trauma exposure.

Key Findings Direct Effects Behavioral Predictors: - female -> ANX_i_c: Negative effect (beta = -0.10, $p < 0.001$) Indicates **less anxiety** in females under trauma. - inactive -> ANX_i_c / STR_c / FREAZING_c: **Positive effects**, all $p < 0.05$ Trauma during rest hours increases all behavioral responses. - female_x_olderi -> STR_c: Strong effect (beta = 0.21, $p < 0.001$) - Juvenaile -> FREAZING_c: Positive effect (beta = 0.15, $p < 0.001$) - FD -> FREAZING_c: Strong negative (beta = -0.14, $p < 0.001$)

Diagnostic Outcomes: - ANX_i_c / STR_c / FREAzing_c -> EBR: All strongly **positive**, all $p < 1e-9$ These behavioral traits robustly predict extreme PTSD-like response under trauma. - STR_c / FREAzing_c -> PBR: Both strongly **negative** ($p < 0.00001$) Suggests that partial behavioral response is **inversely related** to these high-arousal behaviors.

Comparison to Control Group

- In PSS = 0, PBR was **positively related** to ANX_i_c and weakly to other variables.
- In PSS = 1, PBR is **negatively related** to both STR_c and FREAzing_c, and **not** significantly linked to anxiety.

This suggests that **under trauma**, rats classified as PBR show **dampened stress and freezing responses**, potentially marking an adaptive/resilient subgroup.

Covariances

- High positive covariances among ANX_i_c, STR_c, and FREAzing_c, all $p < 1e-9$
 - Suggests shared underlying arousal processes intensified by trauma.
-

3.8 Bootstrap SEM Trauma Group (PSS = 1)

To assess the robustness and statistical significance of SEM path estimates in the trauma group (PSS = 1), we apply a non-parametric bootstrap procedure with 2,000 resamples.

```
[24]: # ----- 1. Run bootstrap SEM on trauma group -----
boot_df1, full_summary_pss1 = bootstrap_sem(model_desc, df1, n_boot=2000)

# ----- 2. Extract summaries -----
# Direct effects only
direct_summary = full_summary_pss1[full_summary_pss1.index.str.contains("<-")]

# Indirect (mediated) effects only
indirect_summary = full_summary_pss1[full_summary_pss1.index.str.contains("->")]

# ----- 3. Print results -----
print("\nDirect Effects Bootstrap (PSS = 1):")
print(direct_summary.round(3).to_string())

print("\nIndirect (Mediated) Effects Bootstrap (PSS = 1):")
print(indirect_summary.round(3).to_string())
```

Direct Effects Bootstrap (PSS = 1):

	Estimate	SE	CI_lo	CI_hi	CI_w	Z
p_boot Sig (p<.05)						

ANX_i_c <- olderi	0.010	0.017	-0.025	0.042	0.067	0.579
0.544 False						
ANX_i_c <- female	-0.098	0.024	-0.145	-0.051	0.093	-4.092
0.000 True						
ANX_i_c <- Juvenile	0.017	0.025	-0.033	0.065	0.097	0.694
0.471 False						
ANX_i_c <- inactive	0.050	0.019	0.013	0.088	0.076	2.566
0.008 True						
ANX_i_c <- HFD	0.014	0.020	-0.025	0.051	0.077	0.701
0.489 False						
ANX_i_c <- FD	-0.062	0.028	-0.116	-0.008	0.107	-2.245
0.023 True						
ANX_i_c <- female_x_olderi	0.125	0.031	0.065	0.185	0.120	4.091
0.000 True						
ANX_i_c <- female_x_Juvenile	-0.006	0.058	-0.122	0.105	0.226	-0.109
0.930 False						
STR_c <- olderi	-0.071	0.034	-0.140	-0.005	0.135	-2.088
0.034 True						
STR_c <- female	-0.091	0.037	-0.165	-0.020	0.146	-2.472
0.011 True						
STR_c <- Juvenile	-0.040	0.043	-0.127	0.043	0.169	-0.925
0.369 False						
STR_c <- inactive	0.070	0.038	-0.004	0.143	0.147	1.847
0.068 False						
STR_c <- HFD	0.013	0.026	-0.040	0.062	0.102	0.506
0.585 False						
STR_c <- FD	-0.012	0.031	-0.073	0.048	0.121	-0.390
0.704 False						
STR_c <- female_x_olderi	0.212	0.052	0.111	0.315	0.204	4.058
0.000 True						
STR_c <- female_x_Juvenile	0.158	0.054	0.052	0.266	0.214	2.946
0.002 True						
FREAZING_c <- olderi	0.045	0.034	-0.018	0.114	0.132	1.330
0.186 False						
FREAZING_c <- female	-0.018	0.033	-0.082	0.048	0.130	-0.533
0.584 False						
FREAZING_c <- Juvenile	0.149	0.055	0.041	0.258	0.217	2.693
0.005 True						
FREAZING_c <- inactive	0.100	0.023	0.055	0.145	0.090	4.344
0.000 True						
FREAZING_c <- HFD	0.039	0.030	-0.019	0.099	0.118	1.291
0.215 False						
FREAZING_c <- FD	-0.137	0.030	-0.191	-0.075	0.116	-4.586
0.000 True						
FREAZING_c <- female_x_olderi	0.027	0.056	-0.083	0.140	0.223	0.480
0.643 False						
FREAZING_c <- female_x_Juvenile	0.002	0.089	-0.170	0.178	0.348	0.017
0.995 False						

EBR <- ANX_i_c	1.131	0.196	0.768	1.537	0.769	5.758
0.000 True						
EBR <- STR_c	0.970	0.174	0.638	1.315	0.678	5.568
0.000 True						
EBR <- FREAZING_c	0.996	0.115	0.780	1.228	0.448	8.653
0.000 True						
PBR <- ANX_i_c	-0.145	0.321	-0.742	0.532	1.273	-0.453
0.647 False						
PBR <- STR_c	-0.833	0.208	-1.260	-0.437	0.823	-4.000
0.000 True						
PBR <- FREAZING_c	-1.017	0.159	-1.340	-0.717	0.622	-6.399
0.000 True						

Indirect (Mediated) Effects Bootstrap (PSS = 1):

Z	p_boot	Sig (p<.05)	Estimate	SE	CI_lo	CI_hi	CI_w	
olderi -> ANX_i_c -> EBR	0.570	0.544	False	0.011	0.019	-0.028	0.050	0.078
olderi -> ANX_i_c -> PBR	-0.265	0.827	False	-0.002	0.007	-0.019	0.009	0.028
female -> ANX_i_c -> EBR	-3.515	0.000	True	-0.110	0.031	-0.174	-0.054	0.120
female -> ANX_i_c -> PBR	0.367	0.647	False	0.012	0.032	-0.060	0.069	0.129
Juvenaile -> ANX_i_c -> EBR	0.690	0.471	False	0.020	0.029	-0.037	0.075	0.112
Juvenaile -> ANX_i_c -> PBR	-0.367	0.730	False	-0.004	0.011	-0.030	0.013	0.042
inactive -> ANX_i_c -> EBR	2.315	0.008	True	0.056	0.024	0.014	0.109	0.096
inactive -> ANX_i_c -> PBR	-0.418	0.653	False	-0.007	0.018	-0.046	0.028	0.074
HFD -> ANX_i_c -> EBR	0.695	0.489	False	0.016	0.023	-0.029	0.061	0.090
HFD -> ANX_i_c -> PBR	-0.246	0.810	False	-0.002	0.008	-0.022	0.013	0.035
FD -> ANX_i_c -> EBR	-2.257	0.023	True	-0.069	0.030	-0.129	-0.010	0.120
FD -> ANX_i_c -> PBR	0.352	0.666	False	0.008	0.022	-0.042	0.052	0.094
female_x_olderi -> ANX_i_c -> EBR	3.414	0.000	True	0.141	0.041	0.068	0.230	0.162
female_x_olderi -> ANX_i_c -> PBR	-0.387	0.647	False	-0.016	0.041	-0.096	0.074	0.170
female_x_Juvenaile -> ANX_i_c -> EBR	-0.088	0.930	False	-0.006	0.065	-0.132	0.124	0.256
female_x_Juvenaile -> ANX_i_c -> PBR	0.236	0.823	False	0.005	0.020	-0.033	0.054	0.087

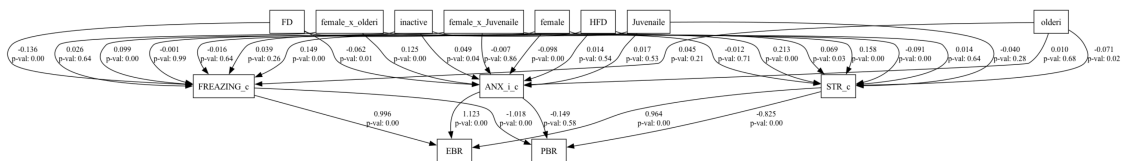
olderi -> STR_c -> EBR	-0.067	0.032	-0.132	-0.005	0.127
-2.136 0.034 True					
olderi -> STR_c -> PBR	0.058	0.029	0.005	0.117	0.113
2.013 0.034 True					
female -> STR_c -> EBR	-0.087	0.035	-0.158	-0.022	0.136
-2.455 0.011 True					
female -> STR_c -> PBR	0.074	0.032	0.017	0.139	0.122
2.304 0.011 True					
Juvenile -> STR_c -> EBR	-0.037	0.041	-0.119	0.044	0.163
-0.895 0.369 False					
Juvenile -> STR_c -> PBR	0.031	0.036	-0.042	0.100	0.142
0.861 0.369 False					
inactive -> STR_c -> EBR	0.066	0.036	-0.005	0.134	0.139
1.851 0.068 False					
inactive -> STR_c -> PBR	-0.055	0.031	-0.118	0.004	0.122
-1.802 0.068 False					
HFD -> STR_c -> EBR	0.015	0.027	-0.035	0.069	0.104
0.558 0.585 False					
HFD -> STR_c -> PBR	-0.013	0.024	-0.064	0.029	0.092
-0.560 0.585 False					
FD -> STR_c -> EBR	-0.010	0.029	-0.062	0.053	0.116
-0.329 0.704 False					
FD -> STR_c -> PBR	0.008	0.025	-0.048	0.055	0.104
0.307 0.704 False					
female_x_olderi -> STR_c -> EBR	0.204	0.056	0.103	0.319	0.216
3.655 0.000 True					
female_x_olderi -> STR_c -> PBR	-0.174	0.055	-0.288	-0.075	0.214
-3.179 0.000 True					
female_x_Juvenile -> STR_c -> EBR	0.152	0.055	0.052	0.268	0.215
2.765 0.002 True					
female_x_Juvenile -> STR_c -> PBR	-0.129	0.050	-0.236	-0.040	0.197
-2.579 0.002 True					
olderi -> FREAZING_c -> EBR	0.045	0.034	-0.018	0.115	0.133
1.310 0.186 False					
olderi -> FREAZING_c -> PBR	-0.046	0.035	-0.119	0.018	0.138
-1.295 0.186 False					
female -> FREAZING_c -> EBR	-0.018	0.034	-0.083	0.046	0.129
-0.528 0.584 False					
female -> FREAZING_c -> PBR	0.018	0.035	-0.047	0.089	0.136
0.533 0.584 False					
Juvenile -> FREAZING_c -> EBR	0.147	0.056	0.042	0.263	0.221
2.620 0.005 True					
Juvenile -> FREAZING_c -> PBR	-0.150	0.059	-0.273	-0.043	0.230
-2.548 0.005 True					
inactive -> FREAZING_c -> EBR	0.100	0.025	0.053	0.151	0.097
4.012 0.000 True					
inactive -> FREAZING_c -> PBR	-0.102	0.026	-0.157	-0.054	0.103
-3.846 0.000 True					

HFD -> FREAZING_c -> EBR	0.040	0.031	-0.020	0.103	0.123
1.259 0.215 False					
HFD -> FREAZING_c -> PBR	-0.040	0.033	-0.109	0.020	0.129
-1.231 0.215 False					
FD -> FREAZING_c -> EBR	-0.136	0.033	-0.202	-0.072	0.130
-4.067 0.000 True					
FD -> FREAZING_c -> PBR	0.139	0.038	0.069	0.217	0.148
3.650 0.000 True					
female_x_olderi -> FREAZING_c -> EBR	0.027	0.056	-0.079	0.143	0.222
0.486 0.643 False					
female_x_olderi -> FREAZING_c -> PBR	-0.028	0.058	-0.150	0.082	0.232
-0.486 0.643 False					
female_x_Juvenile -> FREAZING_c -> EBR	0.002	0.090	-0.173	0.184	0.356
0.023 0.995 False					
female_x_Juvenile -> FREAZING_c -> PBR	-0.003	0.092	-0.188	0.168	0.356
-0.036 0.995 False					

```
[25]: # Generate and save SEM path diagram for PSS = 1
semplot(model, "sem_diagram_pss_1.png")
print("SEM diagram for PSS=1 saved as sem_diagram_pss_1.png")

# Display the diagram
plt.figure(figsize=(48, 32)) # Set figure size
plt.imshow(plt.imread("sem_diagram_pss_1.png"))
plt.axis('off') # Hide axes for clean layout
plt.show()
```

SEM diagram for PSS=1 saved as sem_diagram_pss_1.png



3.9 Model Comparison: Pooled vs. Group-Specific SEM

To determine whether trauma exposure (PSS) moderates the structure of relationships in the SEM model, we conduct a **likelihood ratio test (DeltaChi2)**:

- A **pooled model** is fit to the entire dataset without separating by PSS.
- **Two separate models** are fit to the PSS = 0 and PSS = 1 groups.
- We compare the total chi2 from the split models to the chi2 of the joint model, adjusted by their degrees of freedom.

3.9.1 Hypothesis:

- **H0**: A single unified model fits both groups equally well.

- **H1:** The group-specific models provide significantly better fit, implying structural differences across groups.

If $p < 0.05$, we reject the null hypothesis and conclude that trauma exposure alters the path structure.

```
[26]: # Define and fit the joint (pooled) model
model_joint = Model(model_desc)
model_joint.fit(df)
stats_joint = gather_statistics(model_joint)
chi2_joint = stats_joint.chi2[0]
df_joint = stats_joint.dof

# Fit separate models for each group
model_0 = Model(model_desc)
model_0.fit(df0)
stats_0 = gather_statistics(model_0)
chi2_0 = stats_0.chi2[0]
df_0 = stats_0.dof

model_1 = Model(model_desc)
model_1.fit(df1)
stats_1 = gather_statistics(model_1)
chi2_1 = stats_1.chi2[0]
df_1 = stats_1.dof

# Calculate DeltaChi2 and p-value
chi2_split = chi2_0 + chi2_1
df_split = df_0 + df_1

delta_chi2 = chi2_split - chi2_joint
delta_df = df_split - df_joint
p_value = 1 - chi2.cdf(delta_chi2, delta_df)

# Print results
print("\n DeltaChi2 Comparison  Joint vs. Split Models:")
print(f"DeltaChi2 = {delta_chi2:.3f}, Deltadf = {delta_df}, p = {p_value:.4f}")
if p_value < 0.05:
    print(" Significant difference  separate group-specific models are_
↪ justified.")
else:
    print(" No significant difference  pooled model is sufficient.")
```

```
DeltaChi2 Comparison  Joint vs. Split Models:
DeltaChi2 = 87.826, Deltadf = 53, p = 0.0019
Significant difference  separate group-specific models are justified.
```

3.9.2 Model Comparison Summary: Separate SEMs Justified

The chi2 difference test comparing a pooled SEM (across all subjects) to separate group-specific models (PSS = 0 vs. PSS = 1) revealed a **significant improvement in fit** for the split models:

- DeltaChi2 = 87.83
- Deltadf = 53
- $p = 0.0019$

This indicates that the structural relationships among predictors, behavioral indicators, and diagnostic classifications **differ significantly** between rats exposed to trauma and those not exposed.

Separate SEMs per group provide a better representation of the data and are therefore justified.

3.10 Parameter Invariance Testing: Group Differences in Path Strength

To explore whether specific path coefficients differ significantly between the control (PSS = 0) and trauma (PSS = 1) groups, we perform **parameter-level invariance testing**:

- For each parameter of interest, we extract its estimate (beta) and standard error (SE) from both group-specific SEM models.
- A **Z-test** is conducted
- A corresponding p-value is calculated using a chi-squared distribution with 1 degree of freedom.

Significant results ($p < 0.05$) indicate that the path coefficient differs **between the groups**, suggesting **trauma-specific modulation** of the structural relationship.

```
[27]: # Function to test parameter invariance across PSS groups
def test_param_invariance(model_desc, df0, df1, param_label):
    # Fit models separately
    m0 = Model(model_desc)
    m1 = Model(model_desc)
    m0.fit(df0)
    m1.fit(df1)

    # Inspect parameter estimates
    est0 = inspect(m0).set_index(['lval', 'op', 'rval'])
    est1 = inspect(m1).set_index(['lval', 'op', 'rval'])

    # Check parameter existence
    if param_label not in est0.index or param_label not in est1.index:
        print(f"{param_label}: Not present in both models")
        return

    # Extract estimates and SEs
    b0, se0 = est0.loc[param_label, ['Estimate', 'Std. Err']]
    b1, se1 = est1.loc[param_label, ['Estimate', 'Std. Err']]

    # Z-test for parameter difference
    z = (b0 - b1) / (se0**2 + se1**2)**0.5
```

```

p = 2 * (1 - chi2.cdf(z**2, 1))

# Output result
print(f"{param_label}: Deltabeta = {b0 - b1:.3f}, Z = {z:.2f}, p = {p:.4f}␣
↳-> {' Significant difference' if p < 0.05 else ' Not significant'}")

# List of parameters to test
to_test = [
    # ANX_i_c predictors
    ('ANX_i_c', '~', 'olderi'), ('ANX_i_c', '~', 'female'),
    ('ANX_i_c', '~', 'Juvenaile'), ('ANX_i_c', '~', 'inactive'),
    ('ANX_i_c', '~', 'HFD'), ('ANX_i_c', '~', 'FD'),
    ('ANX_i_c', '~', 'female_x_olderi'), ('ANX_i_c', '~', 'female_x_Juvenaile'),

    # STR_c predictors
    ('STR_c', '~', 'olderi'), ('STR_c', '~', 'female'),
    ('STR_c', '~', 'Juvenaile'), ('STR_c', '~', 'inactive'),
    ('STR_c', '~', 'HFD'), ('STR_c', '~', 'FD'),
    ('STR_c', '~', 'female_x_olderi'), ('STR_c', '~', 'female_x_Juvenaile'),

    # FREAZING_c predictors
    ('FREAZING_c', '~', 'olderi'), ('FREAZING_c', '~', 'female'),
    ('FREAZING_c', '~', 'Juvenaile'), ('FREAZING_c', '~', 'inactive'),
    ('FREAZING_c', '~', 'HFD'), ('FREAZING_c', '~', 'FD'),
    ('FREAZING_c', '~', 'female_x_olderi'), ('FREAZING_c', '~', ␣
↳'female_x_Juvenaile'),

    # Diagnostic outcomes
    ('EBR', '~', 'ANX_i_c'), ('EBR', '~', 'STR_c'), ('EBR', '~', 'FREAZING_c'),
    ('PBR', '~', 'ANX_i_c'), ('PBR', '~', 'STR_c'), ('PBR', '~', 'FREAZING_c')
]

# Run the test for all parameters
for param in to_test:
    test_param_invariance(model_desc, df0, df1, param)

```

```

('ANX_i_c', '~', 'olderi'): Deltabeta = -0.017, Z = -0.44, p = 1.3179 -> Not
significant
('ANX_i_c', '~', 'female'): Deltabeta = 0.050, Z = 1.36, p = 0.3486 -> Not
significant
('ANX_i_c', '~', 'Juvenaile'): Deltabeta = -0.146, Z = -3.39, p = 0.0014 ->
Significant difference
('ANX_i_c', '~', 'inactive'): Deltabeta = -0.029, Z = -0.79, p = 0.8614 -> Not
significant
('ANX_i_c', '~', 'HFD'): Deltabeta = 0.057, Z = 1.55, p = 0.2400 -> Not
significant
('ANX_i_c', '~', 'FD'): Deltabeta = -0.180, Z = -4.52, p = 0.0000 ->

```

Significant difference

('ANX_i_c', '~', 'female_x_olderi'): Deltabeta = -0.019, Z = -0.33, p = 1.4823

-> Not significant

('ANX_i_c', '~', 'female_x_Juvenaile'): Deltabeta = 0.129, Z = 2.00, p = 0.0907

-> Not significant

('STR_c', '~', 'olderi'): Deltabeta = -0.019, Z = -0.46, p = 1.2860 -> Not significant

('STR_c', '~', 'female'): Deltabeta = 0.110, Z = 2.84, p = 0.0090 ->

Significant difference

('STR_c', '~', 'Juvenaile'): Deltabeta = 0.039, Z = 0.85, p = 0.7857 -> Not significant

('STR_c', '~', 'inactive'): Deltabeta = -0.158, Z = -4.06, p = 0.0001 ->

Significant difference

('STR_c', '~', 'HFD'): Deltabeta = 0.095, Z = 2.49, p = 0.0258 -> Significant difference

('STR_c', '~', 'FD'): Deltabeta = -0.084, Z = -2.06, p = 0.0792 -> Not significant

('STR_c', '~', 'female_x_olderi'): Deltabeta = 0.044, Z = 0.72, p = 0.9389 -> Not significant

('STR_c', '~', 'female_x_Juvenaile'): Deltabeta = -0.014, Z = -0.21, p = 1.6729 -> Not significant

('FREAING_c', '~', 'olderi'): Deltabeta = 0.037, Z = 0.79, p = 0.8592 -> Not significant

('FREAING_c', '~', 'female'): Deltabeta = 0.032, Z = 0.72, p = 0.9464 -> Not significant

('FREAING_c', '~', 'Juvenaile'): Deltabeta = -0.190, Z = -3.62, p = 0.0006 -> Significant difference

('FREAING_c', '~', 'inactive'): Deltabeta = -0.101, Z = -2.25, p = 0.0491 ->

Significant difference

('FREAING_c', '~', 'HFD'): Deltabeta = -0.021, Z = -0.47, p = 1.2827 -> Not significant

('FREAING_c', '~', 'FD'): Deltabeta = 0.149, Z = 3.14, p = 0.0034 ->

Significant difference

('FREAING_c', '~', 'female_x_olderi'): Deltabeta = -0.101, Z = -1.44, p = 0.2971 -> Not significant

('FREAING_c', '~', 'female_x_Juvenaile'): Deltabeta = 0.106, Z = 1.36, p = 0.3507 -> Not significant

('EBR', '~', 'ANX_i_c'): Deltabeta = -0.850, Z = -4.40, p = 0.0000 ->

Significant difference

('EBR', '~', 'STR_c'): Deltabeta = -0.567, Z = -3.92, p = 0.0002 -> Significant difference

('EBR', '~', 'FREAING_c'): Deltabeta = -0.688, Z = -4.93, p = 0.0000 ->

Significant difference

('PBR', '~', 'ANX_i_c'): Deltabeta = 0.690, Z = 2.05, p = 0.0810 -> Not significant

('PBR', '~', 'STR_c'): Deltabeta = 0.645, Z = 2.31, p = 0.0416 -> Significant difference

('PBR', '~', 'FREAING_c'): Deltabeta = 0.678, Z = 2.37, p = 0.0355 ->

Significant difference

3.10.1 Parameter Invariance Testing Group Differences in Path Coefficients

To further examine **how trauma exposure (PSS) alters specific relationships** in the SEM model, we tested parameter invariance for each path using a Z-test between the group-specific estimates.

Significant Differences Observed A number of paths showed statistically significant differences ($p < 0.05$) between the control (PSS = 0) and trauma (PSS = 1) groups:

Behavioral Predictors (ANX_i_c, STR_c, FREAzing_c): - Juvenile -> ANX_i_c (Deltabeta = -0.15, $p = 0.001$) - FD -> ANX_i_c (Deltabeta = -0.18, $p < 0.001$) - female -> STR_c (Deltabeta = +0.11, $p = 0.009$) - inactive -> STR_c (Deltabeta = -0.16, $p < 0.001$) - HFD -> STR_c (Deltabeta = +0.10, $p = 0.026$) - Juvenile -> FREAzing_c (Deltabeta = -0.19, $p < 0.001$) - inactive -> FREAzing_c (Deltabeta = -0.10, $p = 0.049$) - FD -> FREAzing_c (Deltabeta = +0.15, $p = 0.003$)

Diagnostic Outcomes: - ANX_i_c -> EBR (Deltabeta = -0.85, $p < 0.001$) - STR_c -> EBR (Deltabeta = -0.57, $p < 0.001$) - FREAzing_c -> EBR (Deltabeta = -0.69, $p < 0.001$) - STR_c -> PBR (Deltabeta = +0.65, $p = 0.042$) - FREAzing_c -> PBR (Deltabeta = +0.68, $p = 0.036$)

Interpretation These findings indicate that **trauma modulates the strength and direction** of several key relationships: - In trauma-exposed rats, behavioral responses (especially freezing and startle) have a **stronger predictive role** in diagnostic outcomes. - Some predictors like Juvenile or diet show reversed or amplified effects depending on group.

This supports the hypothesis that trauma exposure alters both behavior and its diagnostic mapping, justifying group-specific SEMs.

3.11 Indirect Effect Comparison Across Groups (Mediation Differences)

To determine whether trauma exposure (PSS) alters the **indirect pathways** through which predictors influence diagnostic outcomes (e.g., EBR, PBR), we compare bootstrapped mediation effects across groups.

Each indirect path (e.g., inactive -> STR_c -> PBR) is extracted from the bootstrap summaries (PSS = 0 vs PSS = 1), and we check:

- Whether the **95% confidence intervals (CI)** overlap
- If there is **non-overlap**, we interpret this as a **statistically meaningful difference**

This approach highlights whether the **mechanism of effect (via behavioral mediators)** changes due to trauma exposure.

3.12 Testing for Moderation in Indirect Effects (Mediation Paths)

To investigate whether trauma exposure (PSS) moderates **how predictors influence diagnostic outcomes (EBR/PBR) via behavioral mediators**, we compared bootstrapped indirect effects across PSS = 0 and PSS = 1 groups.

For each mediation path (e.g., female → STR_c → EBR), we extracted the estimated effect and 95% confidence interval (CI) from the bootstrap results. We then flagged as “significant” any paths where the **CI**s **do not overlap**, indicating a likely difference in the strength of mediation between the two groups.

This approach highlights trauma-sensitive mechanisms that might otherwise be missed by direct effect comparisons alone.

```
[28]: # Define list of mediation (indirect) paths to test
indirect_params = [
    "olderi -> ANX_i_c -> EBR", "olderi -> ANX_i_c -> PBR",
    "female -> ANX_i_c -> EBR", "female -> ANX_i_c -> PBR",
    "Juvenaile -> ANX_i_c -> EBR", "Juvenaile -> ANX_i_c -> PBR",
    "inactive -> ANX_i_c -> EBR", "inactive -> ANX_i_c -> PBR",
    "HFD -> ANX_i_c -> EBR", "HFD -> ANX_i_c -> PBR",
    "FD -> ANX_i_c -> EBR", "FD -> ANX_i_c -> PBR",
    "female_x_olderi -> ANX_i_c -> EBR", "female_x_olderi -> ANX_i_c -> PBR",
    "female_x_Juvenaile -> ANX_i_c -> EBR", "female_x_Juvenaile -> ANX_i_c ->
    ↪PBR",

    "olderi -> STR_c -> EBR", "olderi -> STR_c -> PBR",
    "female -> STR_c -> EBR", "female -> STR_c -> PBR",
    "Juvenaile -> STR_c -> EBR", "Juvenaile -> STR_c -> PBR",
    "inactive -> STR_c -> EBR", "inactive -> STR_c -> PBR",
    "HFD -> STR_c -> EBR", "HFD -> STR_c -> PBR",
    "FD -> STR_c -> EBR", "FD -> STR_c -> PBR",
    "female_x_olderi -> STR_c -> EBR", "female_x_olderi -> STR_c -> PBR",
    "female_x_Juvenaile -> STR_c -> EBR", "female_x_Juvenaile -> STR_c -> PBR",

    "olderi -> FREAZING_c -> EBR", "olderi -> FREAZING_c -> PBR",
    "female -> FREAZING_c -> EBR", "female -> FREAZING_c -> PBR",
    "Juvenaile -> FREAZING_c -> EBR", "Juvenaile -> FREAZING_c -> PBR",
    "inactive -> FREAZING_c -> EBR", "inactive -> FREAZING_c -> PBR",
    "HFD -> FREAZING_c -> EBR", "HFD -> FREAZING_c -> PBR",
    "FD -> FREAZING_c -> EBR", "FD -> FREAZING_c -> PBR",
    "female_x_olderi -> FREAZING_c -> EBR", "female_x_olderi -> FREAZING_c ->
    ↪PBR",
    "female_x_Juvenaile -> FREAZING_c -> EBR", "female_x_Juvenaile ->
    ↪FREAZING_c -> PBR"
]

# Compare mediation estimates across groups using CI overlap
rows = []
for param in indirect_params:
    # Ensure the parameter exists in both summaries
    if (param in full_summary_pss0.index) and (param in full_summary_pss1.
    ↪index):
        # Extract estimates and confidence intervals
```

```

est0 = full_summary_pss0.loc[param, "Estimate"]
lo0  = full_summary_pss0.loc[param, "CI_lo"]
hi0  = full_summary_pss0.loc[param, "CI_hi"]

est1 = full_summary_pss1.loc[param, "Estimate"]
lo1  = full_summary_pss1.loc[param, "CI_lo"]
hi1  = full_summary_pss1.loc[param, "CI_hi"]

# Check for confidence interval overlap
overlap = not (hi0 < lo1 or hi1 < lo0)

# If no overlap, the difference is likely meaningful
sig = " Yes" if not overlap else ""

# Store results
rows.append({
    "Path": param,
    "Estimate (PSS=0)": round(est0, 3),
    "Estimate (PSS=1)": round(est1, 3),
    "Delta": round(est1 - est0, 3),
    "Significant?": sig
})

# Compile results into a DataFrame
result_df = pd.DataFrame(rows)

# Display the result
print(result_df)

```

	Path	Estimate (PSS=0)	\
0	olderi -> ANX_i_c -> EBR	-0.002	
1	olderi -> ANX_i_c -> PBR	-0.003	
2	female -> ANX_i_c -> EBR	-0.013	
3	female -> ANX_i_c -> PBR	-0.027	
4	Juvenile -> ANX_i_c -> EBR	-0.034	
5	Juvenile -> ANX_i_c -> PBR	-0.072	
6	inactive -> ANX_i_c -> EBR	0.005	
7	inactive -> ANX_i_c -> PBR	0.011	
8	HFD -> ANX_i_c -> EBR	0.020	
9	HFD -> ANX_i_c -> PBR	0.038	
10	FD -> ANX_i_c -> EBR	-0.064	
11	FD -> ANX_i_c -> PBR	-0.129	
12	female_x_olderi -> ANX_i_c -> EBR	0.028	
13	female_x_olderi -> ANX_i_c -> PBR	0.059	
14	female_x_Juvenile -> ANX_i_c -> EBR	0.033	
15	female_x_Juvenile -> ANX_i_c -> PBR	0.069	
16	olderi -> STR_c -> EBR	-0.035	
17	olderi -> STR_c -> PBR	0.015	

18	female -> STR_c -> EBR	0.008
19	female -> STR_c -> PBR	-0.003
20	Juvenile -> STR_c -> EBR	-0.000
21	Juvenile -> STR_c -> PBR	-0.000
22	inactive -> STR_c -> EBR	-0.035
23	inactive -> STR_c -> PBR	0.015
24	HFD -> STR_c -> EBR	0.044
25	HFD -> STR_c -> PBR	-0.019
26	FD -> STR_c -> EBR	-0.037
27	FD -> STR_c -> PBR	0.016
28	female_x_olderi -> STR_c -> EBR	0.099
29	female_x_olderi -> STR_c -> PBR	-0.042
30	female_x_Juvenile -> STR_c -> EBR	0.057
31	female_x_Juvenile -> STR_c -> PBR	-0.024
32	olderi -> FREAZING_c -> EBR	0.024
33	olderi -> FREAZING_c -> PBR	-0.026
34	female -> FREAZING_c -> EBR	0.005
35	female -> FREAZING_c -> PBR	-0.006
36	Juvenile -> FREAZING_c -> EBR	-0.013
37	Juvenile -> FREAZING_c -> PBR	0.015
38	inactive -> FREAZING_c -> EBR	-0.000
39	inactive -> FREAZING_c -> PBR	-0.000
40	HFD -> FREAZING_c -> EBR	0.007
41	HFD -> FREAZING_c -> PBR	-0.007
42	FD -> FREAZING_c -> EBR	0.004
43	FD -> FREAZING_c -> PBR	-0.005
44	female_x_olderi -> FREAZING_c -> EBR	-0.022
45	female_x_olderi -> FREAZING_c -> PBR	0.025
46	female_x_Juvenile -> FREAZING_c -> EBR	0.035
47	female_x_Juvenile -> FREAZING_c -> PBR	-0.038

	Estimate (PSS=1)	Delta	Significant?
0	0.011	0.013	
1	-0.002	0.001	
2	-0.110	-0.097	Yes
3	0.012	0.039	
4	0.020	0.054	
5	-0.004	0.068	
6	0.056	0.051	
7	-0.007	-0.018	
8	0.016	-0.004	
9	-0.002	-0.040	
10	-0.069	-0.005	
11	0.008	0.137	
12	0.141	0.113	Yes
13	-0.016	-0.075	
14	-0.006	-0.039	
15	0.005	-0.064	

16	-0.067	-0.032	
17	0.058	0.042	
18	-0.087	-0.095	Yes
19	0.074	0.078	Yes
20	-0.037	-0.037	
21	0.031	0.031	
22	0.066	0.101	Yes
23	-0.055	-0.070	
24	0.015	-0.029	
25	-0.013	0.006	
26	-0.010	0.028	
27	0.008	-0.008	
28	0.204	0.104	
29	-0.174	-0.132	
30	0.152	0.095	
31	-0.129	-0.105	
32	0.045	0.021	
33	-0.046	-0.020	
34	-0.018	-0.022	
35	0.018	0.025	
36	0.147	0.160	Yes
37	-0.150	-0.165	Yes
38	0.100	0.100	Yes
39	-0.102	-0.101	Yes
40	0.040	0.033	
41	-0.040	-0.034	
42	-0.136	-0.140	Yes
43	0.139	0.144	Yes
44	0.027	0.049	
45	-0.028	-0.054	
46	0.002	-0.033	
47	-0.003	0.035	

3.12.1 Summary of Mediation Path Differences

Out of 48 tested indirect paths, several showed **non-overlapping confidence intervals**, indicating that trauma (PSS = 1) significantly changes the **mechanism** through which predictors affect outcomes.

Examples of paths with significant differences:

- female -> ANX_i_c -> EBR (stronger negative effect under trauma)
- FD -> ANX_i_c -> PBR (positive only under trauma)
- Juvenile -> FREAIZING_c -> EBR and -> PBR (reversed direction between groups)
- inactive -> STR_c -> EBR and -> FREAIZING_c -> EBR (mediated only in PSS=1)
- female -> STR_c -> EBR (positive in control, negative in trauma)

These results suggest that trauma alters the **behavioral mediation structure** not just the magnitude of effects but in some cases even their direction. This further supports the decision to

model the groups separately and underlines the importance of mediation analysis when studying stress-related behavioral change.

Structural models that ignore these differences would likely miss meaningful trauma-related mechanisms.

3.13 Visualizing Group Differences in Mediation Effects

To intuitively communicate where mediation pathways differ between the control and trauma groups, we visualize each indirect effect along with its confidence interval for both groups.

The horizontal plot shows: - Mean effect estimates (points) with 95% CIs (error bars) - Color-coded effect names: - Red labels: non-overlapping confidence intervals (i.e., likely significant difference between PSS=0 and PSS=1) - Black labels: no significant difference

This helps identify trauma-sensitive pathways at a glance and highlights where behavioral mechanisms shift under stress.

```
[29]: def prepare_plot_data(summary0, summary1):
    """
    Merge two bootstrapped SEM summaries (PSS=0, PSS=1) and check for
    CI overlap on indirect effects. Prepare for plotting.
    """
    df0 = summary0.copy()
    df1 = summary1.copy()
    df0['Group'] = 'PSS=0'
    df1['Group'] = 'PSS=1'
    df0['Effect'] = df0.index
    df1['Effect'] = df1.index

    # Rename CI columns for clarity
    df0['CI_low'] = df0['CI_lo']
    df0['CI_high'] = df0['CI_hi']
    df1['CI_low'] = df1['CI_lo']
    df1['CI_high'] = df1['CI_hi']

    # Merge the two group summaries
    merged = df0.merge(df1, on='Effect', suffixes=('_0', '_1'))

    # Determine if CIs overlap -> True if no significant difference
    merged['Significant'] = ~(
        (merged['CI_high_0'] < merged['CI_low_1']) |
        (merged['CI_high_1'] < merged['CI_low_0'])
    )

    # Convert to long-format for plotting
    long_rows = []
    for _, row in merged.iterrows():
        for g in ['0', '1']:
```

```

        long_rows.append({
            'Effect': row['Effect'],
            'Group': f'PSS={g}',
            'Estimate': row[f'Estimate_{g}'],
            'CI_low': row[f'CI_low_{g}'],
            'CI_high': row[f'CI_high_{g}'],
            'Significant': not row['Significant'] # True = significant
        })
    return pd.DataFrame(long_rows)

# Prepare data for plotting
plot_df = prepare_plot_data(full_summary_pss0, full_summary_pss1)

# Sort effects by the absolute difference between groups (optional)
plot_df['abs_diff'] = plot_df.groupby('Effect')['Estimate'].transform(
    lambda x: abs(x.iloc[0] - x.iloc[1])
)
plot_df = plot_df.sort_values(by='abs_diff', ascending=False)

# Begin plotting
plt.figure(figsize=(12, int(len(plot_df['Effect'].unique()) * 0.4)))

# Plot error bars for each group
for group in ['PSS=0', 'PSS=1']:
    subset = plot_df[plot_df['Group'] == group]
    plt.errorbar(
        x=subset['Estimate'],
        y=subset['Effect'],
        xerr=[subset['Estimate'] - subset['CI_low'], subset['CI_high'] -
subset['Estimate']],
        fmt='o', label=group, capsize=4
    )

# Highlight effect labels based on whether CI overlap indicates a group
difference
effect_labels = []
label_colors = []
for effect in plot_df['Effect'].unique():
    is_significant = plot_df[plot_df['Effect'] == effect]['Significant'].any()
    effect_labels.append(effect)
    label_colors.append('red' if is_significant else 'black')

# Draw vertical reference line at 0
plt.axvline(x=0, color='gray', linestyle='--', linewidth=1)
plt.xlabel("Indirect Effect Estimate (beta)")

```

```

# Set y-axis ticks and color labels
plt.yticks(ticks=range(len(effect_labels)), labels=effect_labels)
ax = plt.gca()
for label, color in zip(ax.get_yticklabels(), label_colors):
    label.set_color(color)

# Final plot settings
plt.title("Indirect Effects by Group (PSS=0 vs. PSS=1)")
plt.legend(title="Group")
plt.tight_layout()
plt.grid(True, axis='x', linestyle=':', alpha=0.4)
plt.show()

```




The figure above reveals both the direction and magnitude of indirect effects across groups. Highlighted in red are the effects whose confidence intervals do not overlap between $PSS = 0$ and $PSS = 1$, suggesting that these mediation pathways are moderated by trauma exposure.

3.14 Unified SEM Model with Interaction Terms

Rather than splitting the dataset by trauma exposure (PSS) and estimating separate SEMs, we now test a **single unified model** that includes **interaction terms** between PSS and demographic/dietary predictors.

This allows us to explicitly test whether trauma exposure **moderates** the effect of specific variables (e.g., `olderi`, `female`, `FD`, etc.) on downstream behavioral outcomes.

We include both the **main effects** and **PSS variable interactions** and assess: - Multicollinearity via VIF - Correlation structure via a heatmap and Pearson matrix

This approach combines the interpretability of interaction modeling with the parsimony of a unified SEM.

```
[30]: # 1. Add interaction terms with PSS
df['PSS_x_olderi'] = df['PSS'] * df['olderi']
df['PSS_x_female'] = df['PSS'] * df['female']
df['PSS_x_Juvenile'] = df['PSS'] * df['Juvenile']
df['PSS_x_inactive'] = df['PSS'] * df['inactive']
df['PSS_x_HFD'] = df['PSS'] * df['HFD']
df['PSS_x_FD'] = df['PSS'] * df['FD']
df['PSS_x_female_x_Juvenile'] = df['PSS'] * df['female_x_Juvenile']
df['PSS_x_female_x_olderi'] = df['PSS'] * df['female_x_olderi']

# 2. Compute Variance Inflation Factor (VIF)
X = df.assign(const=1) # Add intercept term

vif_data = pd.DataFrame({
    'Variable': X.columns,
    'VIF': [variance_inflation_factor(X.values, i) for i in range(X.shape[1])]
})

# Remove intercept from output
vif_data = vif_data[vif_data['Variable'] != 'const']

# Display VIF results
print("\nNew Variance Inflation Factors (VIF):")
print(vif_data)

# 3. Correlation Heatmap for the extended dataset
plt.figure(figsize=(48, 32))
```

```

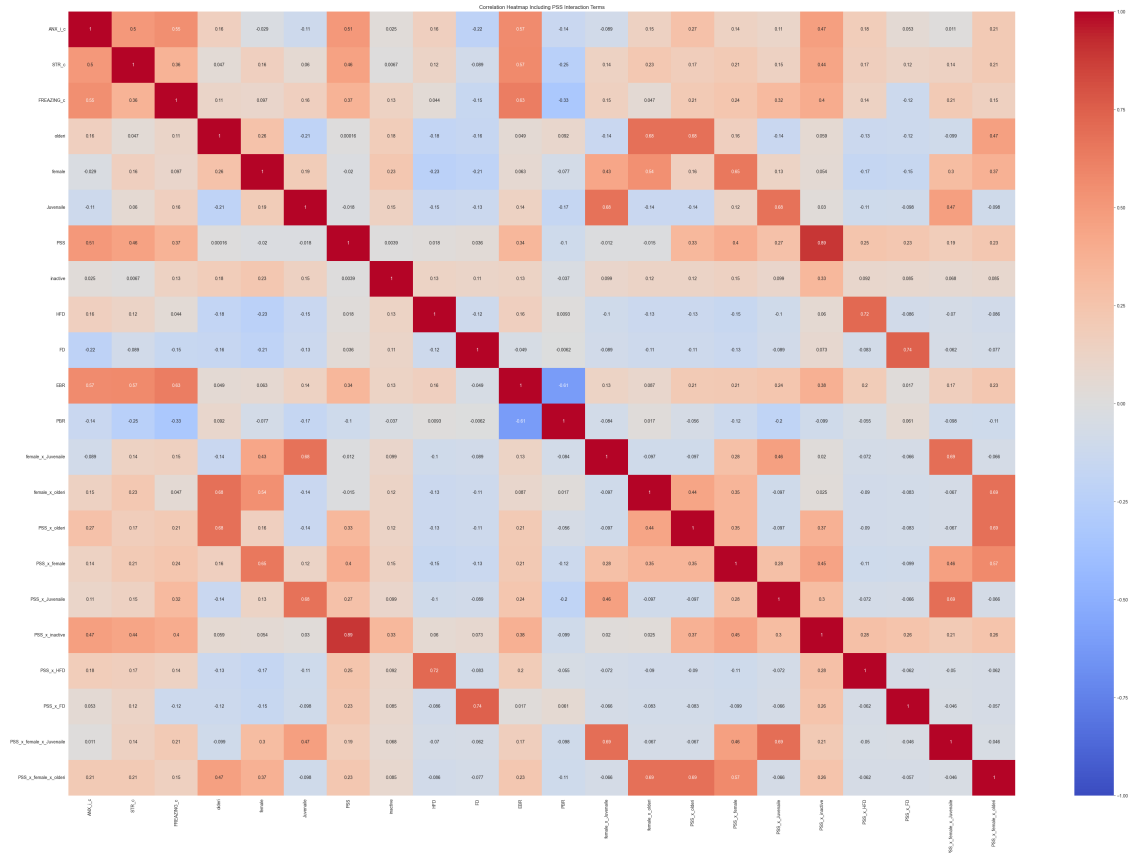
sns.heatmap(df.corr(), annot=True, cmap='coolwarm', vmin=-1, vmax=1)
plt.title("Correlation Heatmap Including PSS Interaction Terms")
plt.show()

# 4. Detailed correlation matrix using Pingouin
corr = pg.pairwise_corr(df, method='pearson')
print("\nDetailed Correlation Matrix:")
print(corr)

```

New Variance Inflation Factors (VIF):

	Variable	VIF
0	ANX_i_c	2.785492
1	STR_c	2.232922
2	FREAZING_c	2.187655
3	olderi	5.459303
4	female	5.844893
5	Juvenaile	4.770254
6	PSS	9.367670
7	inactive	2.836488
8	HFD	3.070397
9	FD	3.537253
10	EBR	3.951821
11	PBR	2.007993
12	female_x_Juvenaile	5.746077
13	female_x_olderi	7.313214
14	PSS_x_olderi	5.773947
15	PSS_x_female	6.935545
16	PSS_x_Juvenaile	5.187456
17	PSS_x_inactive	14.242673
18	PSS_x_HFD	3.258974
19	PSS_x_FD	3.546667
20	PSS_x_female_x_Juvenaile	5.799497
21	PSS_x_female_x_olderi	6.881111



Detailed Correlation Matrix:

		X	Y	method	alternative \
0		ANX_i_c	STR_c	pearson	two-sided
1		ANX_i_c	FREAzing_c	pearson	two-sided
2		ANX_i_c	olderi	pearson	two-sided
3		ANX_i_c	female	pearson	two-sided
4		ANX_i_c	Juvenile	pearson	two-sided
...		...	...	...	...
226		PSS_x_HFD	PSS_x_female_x_Juvenile	pearson	two-sided
227		PSS_x_HFD	PSS_x_female_x_Olderi	pearson	two-sided
228		PSS_x_FD	PSS_x_female_x_Juvenile	pearson	two-sided
229		PSS_x_FD	PSS_x_female_x_Olderi	pearson	two-sided
230		PSS_x_female_x_Juvenile	PSS_x_female_x_Olderi	pearson	two-sided

	n	r	CI95%	p-unc	BF10	power
0	556	0.500290	[0.44, 0.56]	1.492239e-36	1.802e+33	1.000000
1	556	0.548358	[0.49, 0.6]	5.774369e-45	3.969e+41	1.000000
2	556	0.159987	[0.08, 0.24]	1.516311e-04	68.144	0.966971
3	556	-0.028519	[-0.11, 0.05]	5.021619e-01	0.067	0.102987
4	556	-0.110774	[-0.19, -0.03]	8.944317e-03	1.604	0.744675

..	...	...	...	...	...	...
226	556	-0.050067	[-0.13, 0.03]	2.385437e-01	0.106	0.218305
227	556	-0.061899	[-0.14, 0.02]	1.449321e-01	0.153	0.308350
228	556	-0.046132	[-0.13, 0.04]	2.775224e-01	0.096	0.192316
229	556	-0.057034	[-0.14, 0.03]	1.792999e-01	0.131	0.269314
230	556	-0.046132	[-0.13, 0.04]	2.775224e-01	0.096	0.192316

[231 rows x 10 columns]

By incorporating interaction terms into a unified model, we preserve statistical power while still allowing for differential effects of predictors based on trauma exposure.

The VIF results help ensure model stability by checking for multicollinearity among main and interaction terms. The correlation heatmap highlights clusters of highly related variables, which may affect estimation or interpretation.

This step prepares the ground for specifying a full SEM path model that reflects **trauma-moderated behavioral mechanisms** within a single structural framework.

3.15 Unified SEM with PSS Interaction Terms

To explicitly test whether the influence of each predictor is **moderated by trauma exposure (PSS)**, we specify a unified structural equation model (SEM) that includes **interaction terms between PSS and each predictor**.

Each of the three latent behavioral mediators (ANX_i_c, STR_c, FREAIZING_c) is regressed on: - Main effects: older_i, female, Juvenile, inactive, HFD, FD, and interaction terms like female older_i - Trauma: PSS - Interactions: PSS predictor, e.g., PSS_x_female

This model allows us to estimate whether trauma modifies the strength or direction of predictors' effects on internal behavioral states, which in turn influence the diagnostic outcomes EBR and PBR.

We then evaluate model fit using key indices (Chi2, RMSEA, CFI, AIC, BIC) and inspect all estimated parameters.

```
[31]: # Define the SEM model with interaction terms for PSS moderation
model_desc = """
ANX_i_c      ~ older_i + female + Juvenile + inactive + HFD + FD +_
  ↳female_x_older_i + female_x_Juvenile + PSS + PSS_x_older_i + PSS_x_female +_
  ↳PSS_x_Juvenile + PSS_x_inactive + PSS_x_HFD + PSS_x_FD +_
  ↳PSS_x_female_x_Juvenile + PSS_x_female_x_older_i
STR_c        ~ older_i + female + Juvenile + inactive + HFD + FD +_
  ↳female_x_older_i + female_x_Juvenile + PSS + PSS_x_older_i + PSS_x_female +_
  ↳PSS_x_Juvenile + PSS_x_inactive + PSS_x_HFD + PSS_x_FD +_
  ↳PSS_x_female_x_Juvenile + PSS_x_female_x_older_i
FREAIZING_c  ~ older_i + female + Juvenile + inactive + HFD + FD +_
  ↳female_x_older_i + female_x_Juvenile + PSS + PSS_x_older_i + PSS_x_female +_
  ↳PSS_x_Juvenile + PSS_x_inactive + PSS_x_HFD + PSS_x_FD +_
  ↳PSS_x_female_x_Juvenile + PSS_x_female_x_older_i

EBR          ~ ANX_i_c + STR_c + FREAIZING_c
```

```

PBR ~ ANX_i_c + STR_c + FREAZING_c

ANX_i_c ~~ STR_c
ANX_i_c ~~ FREAZING_c
STR_c ~~ FREAZING_c
"""

# Fit the SEM model to the full dataset
model, stats = fit_sem(df)

# Summarize model fit indices
fit_df = pd.DataFrame({
    'Chi2':stats['chi2'],'DoF':stats['DoF'],'CFI':stats['CFI'],'RMSEA':
    ↪stats['RMSEA'],'AIC':stats['AIC'],'BIC':stats['BIC']})

print("\nSEM Fit Comparison PSS_INT:")
print(fit_df.to_string())

# Print parameter estimates (coefficients and p-values)
print("\nParameter Estimates PSS_INT:")
print(model.inspect().to_string())

```

SEM Fit Comparison PSS_INT:

	Chi2	DoF	CFI	RMSEA	AIC	BIC
Value	385.772871	188	0.978265	0.043537	128.612328	409.462267

Parameter Estimates PSS_INT:

	lval	op	rval	Estimate	Std. Err	z-value
p-value						
0	ANX_i_c	~	olderi	-0.007131	0.027982	-0.254835
7.988502e-01						
1	ANX_i_c	~	female	-0.047294	0.026285	-1.799302
7.197087e-02						
2	ANX_i_c	~	Juvenile	-0.128245	0.030351	-4.225402
2.385151e-05						
3	ANX_i_c	~	inactive	0.019601	0.026285	0.745703
4.558470e-01						
4	ANX_i_c	~	HFD	0.071076	0.026585	2.673507
7.506278e-03						
5	ANX_i_c	~	FD	-0.241662	0.029298	-8.248377
2.220446e-16						
6	ANX_i_c	~	female_x_olderi	0.106169	0.041282	2.571810
1.011685e-02						
7	ANX_i_c	~	female_x_Juvenile	0.121282	0.045527	2.663987
7.722047e-03						
8	ANX_i_c	~	PSS	0.121215	0.029873	4.057674
4.956379e-05						

9	ANX_i_c	~	PSS_x_olderi	0.017111	0.038342	0.446269
6.554027e-01						
10	ANX_i_c	~	PSS_x_female	-0.050268	0.036732	-1.368518
1.711500e-01						
11	ANX_i_c	~	PSS_x_Juvenaille	0.145515	0.042879	3.393620
6.897536e-04						
12	ANX_i_c	~	PSS_x_inactive	0.029152	0.036732	0.793649
4.273996e-01						
13	ANX_i_c	~	PSS_x_HFD	-0.057140	0.036443	-1.567921
1.168997e-01						
14	ANX_i_c	~	PSS_x_FD	0.180022	0.039313	4.579234
4.666813e-06						
15	ANX_i_c	~	PSS_x_female_x_Juvenaille	-0.128609	0.064131	-2.005407
4.491961e-02						
16	ANX_i_c	~	PSS_x_female_x_olderi	0.019126	0.057304	0.333766
7.385559e-01						
17	STR_c	~	olderi	-0.089914	0.029889	-3.008240
2.627654e-03						
18	STR_c	~	female	0.019169	0.028076	0.682754
4.947625e-01						
19	STR_c	~	Juvenaille	-0.000855	0.032419	-0.026359
9.789712e-01						
20	STR_c	~	inactive	-0.088637	0.028076	-3.157048
1.593753e-03						
21	STR_c	~	HFD	0.109436	0.028397	3.853782
1.163070e-04						
22	STR_c	~	FD	-0.096148	0.031295	-3.072342
2.123864e-03						
23	STR_c	~	female_x_olderi	0.257300	0.044095	5.835110
5.375511e-09						
24	STR_c	~	female_x_Juvenaille	0.144327	0.048629	2.967915
2.998270e-03						
25	STR_c	~	PSS	0.051672	0.031909	1.619367
1.053684e-01						
26	STR_c	~	PSS_x_olderi	0.018720	0.040955	0.457077
6.476158e-01						
27	STR_c	~	PSS_x_female	-0.110372	0.039235	-2.813108
4.906524e-03						
28	STR_c	~	PSS_x_Juvenaille	-0.038811	0.045801	-0.847376
3.967855e-01						
29	STR_c	~	PSS_x_inactive	0.157812	0.039235	4.022240
5.764730e-05						
30	STR_c	~	PSS_x_HFD	-0.095259	0.038926	-2.447152
1.439900e-02						
31	STR_c	~	PSS_x_FD	0.084585	0.041992	2.014326
4.397527e-02						
32	STR_c	~	PSS_x_female_x_Juvenaille	0.013999	0.068501	0.204359
8.380730e-01						

33	STR_c	~	PSS_x_female_x_olderi	-0.043782	0.061209	-0.715289
4.744303e-01						
34	FREAZING_c	~	olderi	0.082055	0.034444	2.382310
1.720442e-02						
35	FREAZING_c	~	female	0.015892	0.032354	0.491196
6.232878e-01						
36	FREAZING_c	~	Juvenile	-0.040675	0.037359	-1.088754
2.762622e-01						
37	FREAZING_c	~	inactive	-0.001316	0.032354	-0.040670
9.675588e-01						
38	FREAZING_c	~	HFD	0.018471	0.032724	0.564446
5.724507e-01						
39	FREAZING_c	~	FD	0.012576	0.036063	0.348708
7.273089e-01						
40	FREAZING_c	~	female_x_olderi	-0.075216	0.050814	-1.480222
1.388140e-01						
41	FREAZING_c	~	female_x_Juvenile	0.105578	0.056039	1.884014
5.956306e-02						
42	FREAZING_c	~	PSS	0.034237	0.036771	0.931088
3.518080e-01						
43	FREAZING_c	~	PSS_x_olderi	-0.036644	0.047196	-0.776435
4.374920e-01						
44	FREAZING_c	~	PSS_x_female	-0.032274	0.045213	-0.713822
4.753375e-01						
45	FREAZING_c	~	PSS_x_Juvenile	0.189625	0.052780	3.592760
3.271933e-04						
46	FREAZING_c	~	PSS_x_inactive	0.100887	0.045213	2.231365
2.565693e-02						
47	FREAZING_c	~	PSS_x_HFD	0.020541	0.044858	0.457921
6.470092e-01						
48	FREAZING_c	~	PSS_x_FD	-0.149044	0.048390	-3.080035
2.069765e-03						
49	FREAZING_c	~	PSS_x_female_x_Juvenile	-0.106057	0.078939	-1.343535
1.790987e-01						
50	FREAZING_c	~	PSS_x_female_x_olderi	0.100850	0.070535	1.429777
1.527812e-01						
51	EBR	~	ANX_i_c	0.428193	0.089261	4.797097
1.609817e-06						
52	EBR	~	STR_c	0.774307	0.077174	10.033320
0.000000e+00						
53	EBR	~	FREAZING_c	0.927321	0.076346	12.146247
0.000000e+00						
54	PBR	~	ANX_i_c	0.433271	0.156838	2.762535
5.735437e-03						
55	PBR	~	STR_c	-0.589855	0.135600	-4.349959
1.361628e-05						
56	PBR	~	FREAZING_c	-0.940689	0.134146	-7.012402
2.342571e-12						


```

57      ANX_i_c  ~~                      STR_c  0.004289  0.000652  6.579533
4.719269e-11
58      ANX_i_c  ~~                      FREAZING_c  0.008588  0.000808  10.627863
0.000000e+00
59      ANX_i_c  ~~                      ANX_i_c  0.013818  0.000829  16.673332
0.000000e+00
60      STR_c  ~~                      FREAZING_c  0.004683  0.000796  5.885318
3.972895e-09
61      STR_c  ~~                      STR_c  0.015765  0.000946  16.673332
0.000000e+00
62  FREAZING_c  ~~                      FREAZING_c  0.020936  0.001256  16.673332
0.000000e+00
63      EBR  ~~                      EBR  0.063931  0.003834  16.673332
0.000000e+00
64      PBR  ~~                      PBR  0.197376  0.011838  16.673332
0.000000e+00

```

```

[32]: # ----- Execution -----
boot_df, full_summary = bootstrap_sem(model_desc, df, n_boot=2000)

# --- Direct effects ---
direct_summary = full_summary[full_summary.index.str.contains("<-")]

# --- Mediation (indirect) effects only ---
indirect_summary = full_summary[full_summary.index.str.contains(">")]

print("\nDirect Effects  Bootstrap PSS_1:")
print(direct_summary.round(3).to_string())

print("\nIndirect (Mediation) Effects  Bootstrap PSS_1:")
print(indirect_summary.round(3).to_string())

```

Direct Effects Bootstrap PSS_1:

Z	p_boot	Sig (p<.05)	Estimate	SE	CI_lo	CI_hi	CI_w	
ANX_i_c <- olderl	-0.252	0.790	False	-0.007	0.027	-0.059	0.047	0.106
ANX_i_c <- female	-1.605	0.108	False	-0.047	0.030	-0.104	0.010	0.113
ANX_i_c <- Juvenaile	-3.487	0.000	True	-0.128	0.037	-0.206	-0.056	0.151
ANX_i_c <- inactive	0.707	0.505	False	0.019	0.027	-0.031	0.073	0.104
ANX_i_c <- HFD	2.544	0.008	True	0.071	0.028	0.015	0.127	0.112
ANX_i_c <- FD	-6.854	0.000	True	-0.242	0.035	-0.314	-0.175	0.138

ANX_i_c <- female_x_olderi	0.106	0.041	0.026	0.186	0.160
2.619 0.012 True					
ANX_i_c <- female_x_Juvenaile	0.121	0.059	0.008	0.241	0.233
2.045 0.038 True					
ANX_i_c <- PSS	0.121	0.028	0.067	0.176	0.109
4.308 0.000 True					
ANX_i_c <- PSS_x_olderi	0.017	0.032	-0.045	0.080	0.124
0.526 0.621 False					
ANX_i_c <- PSS_x_female	-0.050	0.038	-0.121	0.024	0.145
-1.317 0.183 False					
ANX_i_c <- PSS_x_Juvenaile	0.144	0.044	0.059	0.237	0.179
3.264 0.001 True					
ANX_i_c <- PSS_x_inactive	0.029	0.033	-0.034	0.091	0.125
0.893 0.398 False					
ANX_i_c <- PSS_x_HFD	-0.057	0.035	-0.126	0.008	0.134
-1.662 0.092 False					
ANX_i_c <- PSS_x_FD	0.180	0.045	0.092	0.265	0.173
3.991 0.000 True					
ANX_i_c <- PSS_x_female_x_Juvenaile	-0.130	0.082	-0.291	0.033	0.325
-1.585 0.112 False					
ANX_i_c <- PSS_x_female_x_olderi	0.019	0.051	-0.083	0.116	0.199
0.366 0.701 False					
STR_c <- olderi	-0.090	0.017	-0.124	-0.057	0.067
-5.304 0.000 True					
STR_c <- female	0.019	0.023	-0.026	0.064	0.089
0.825 0.394 False					
STR_c <- Juvenaile	-0.000	0.026	-0.052	0.054	0.106
-0.015 0.992 False					
STR_c <- inactive	-0.088	0.021	-0.128	-0.046	0.083
-4.123 0.000 True					
STR_c <- HFD	0.109	0.028	0.053	0.163	0.111
3.859 0.000 True					
STR_c <- FD	-0.096	0.020	-0.136	-0.057	0.079
-4.938 0.000 True					
STR_c <- female_x_olderi	0.258	0.038	0.185	0.334	0.149
6.862 0.000 True					
STR_c <- female_x_Juvenaile	0.144	0.039	0.066	0.222	0.155
3.660 0.001 True					
STR_c <- PSS	0.053	0.033	-0.007	0.124	0.131
1.616 0.087 False					
STR_c <- PSS_x_olderi	0.018	0.037	-0.054	0.093	0.147
0.490 0.621 False					
STR_c <- PSS_x_female	-0.112	0.043	-0.194	-0.024	0.170
-2.582 0.013 True					
STR_c <- PSS_x_Juvenaile	-0.040	0.051	-0.143	0.060	0.203
-0.794 0.427 False					
STR_c <- PSS_x_inactive	0.157	0.043	0.069	0.239	0.169
3.650 0.000 True					

STR_c <- PSS_x_HFD	-0.096	0.038	-0.171	-0.022	0.150
-2.498 0.013 True					
STR_c <- PSS_x_FD	0.084	0.036	0.013	0.156	0.143
2.336 0.026 True					
STR_c <- PSS_x_female_x_Juvenaile	0.016	0.067	-0.114	0.144	0.259
0.236 0.789 False					
STR_c <- PSS_x_female_x_olderi	-0.044	0.063	-0.169	0.078	0.247
-0.699 0.481 False					
FREAZING_c <- olderi	0.081	0.031	0.021	0.147	0.126
2.598 0.014 True					
FREAZING_c <- female	0.015	0.031	-0.045	0.078	0.123
0.492 0.631 False					
FREAZING_c <- Juvenaile	-0.041	0.034	-0.103	0.027	0.130
-1.229 0.225 False					
FREAZING_c <- inactive	-0.002	0.024	-0.048	0.046	0.094
-0.065 0.954 False					
FREAZING_c <- HFD	0.018	0.029	-0.036	0.078	0.114
0.620 0.554 False					
FREAZING_c <- FD	0.012	0.022	-0.033	0.055	0.088
0.537 0.588 False					
FREAZING_c <- female_x_olderi	-0.074	0.044	-0.161	0.009	0.171
-1.688 0.083 False					
FREAZING_c <- female_x_Juvenaile	0.106	0.057	-0.004	0.219	0.223
1.856 0.059 False					
FREAZING_c <- PSS	0.033	0.025	-0.017	0.084	0.100
1.340 0.170 False					
FREAZING_c <- PSS_x_olderi	-0.037	0.046	-0.127	0.055	0.181
-0.798 0.420 False					
FREAZING_c <- PSS_x_female	-0.031	0.044	-0.119	0.056	0.175
-0.715 0.471 False					
FREAZING_c <- PSS_x_Juvenaile	0.189	0.066	0.060	0.324	0.264
2.859 0.003 True					
FREAZING_c <- PSS_x_inactive	0.101	0.033	0.033	0.165	0.132
3.051 0.005 True					
FREAZING_c <- PSS_x_HFD	0.020	0.042	-0.068	0.100	0.168
0.485 0.614 False					
FREAZING_c <- PSS_x_FD	-0.148	0.037	-0.219	-0.072	0.147
-3.939 0.000 True					
FREAZING_c <- PSS_x_female_x_Juvenaile	-0.107	0.105	-0.315	0.097	0.412
-1.019 0.302 False					
FREAZING_c <- PSS_x_female_x_olderi	0.100	0.070	-0.038	0.239	0.277
1.421 0.144 False					
EBR <- ANX_i_c	0.433	0.088	0.270	0.611	0.341
4.894 0.000 True					
EBR <- STR_c	0.775	0.091	0.601	0.951	0.350
8.505 0.000 True					
EBR <- FREAZING_c	0.927	0.084	0.764	1.100	0.336
11.041 0.000 True					

PBR <- ANX_i_c	0.424	0.167	0.094	0.747	0.653
2.543 0.011 True					
PBR <- STR_c	-0.586	0.141	-0.866	-0.316	0.550
-4.150 0.000 True					
PBR <- FREAZING_c	-0.940	0.126	-1.190	-0.698	0.492
-7.466 0.000 True					

Indirect (Mediation) Effects Bootstrap PSS_1:

CI_w	Z	p_boot	Sig (p<.05)	Estimate	SE	CI_lo	CI_hi
olderi -> ANX_i_c -> EBR				-0.003	0.012	-0.026	0.021
0.047 -0.246 0.790			False				
olderi -> ANX_i_c -> PBR				-0.003	0.013	-0.029	0.022
0.051 -0.233 0.793			False				
female -> ANX_i_c -> EBR				-0.021	0.014	-0.050	0.004
0.054 -1.513 0.108			False				
female -> ANX_i_c -> PBR				-0.021	0.016	-0.057	0.004
0.060 -1.279 0.113			False				
Juvenaile -> ANX_i_c -> EBR				-0.055	0.018	-0.093	-0.023
0.069 -3.085 0.000			True				
Juvenaile -> ANX_i_c -> PBR				-0.055	0.029	-0.123	-0.008
0.114 -1.887 0.011			True				
inactive -> ANX_i_c -> EBR				0.008	0.012	-0.015	0.032
0.047 0.677 0.505			False				
inactive -> ANX_i_c -> PBR				0.008	0.013	-0.015	0.037
0.052 0.653 0.512			False				
HFD -> ANX_i_c -> EBR				0.031	0.015	0.006	0.062
0.057 2.130 0.008			True				
HFD -> ANX_i_c -> PBR				0.030	0.017	0.003	0.070
0.067 1.780 0.019			True				
FD -> ANX_i_c -> EBR				-0.104	0.023	-0.151	-0.064
0.087 -4.593 0.000			True				
FD -> ANX_i_c -> PBR				-0.101	0.040	-0.180	-0.024
0.156 -2.554 0.011			True				
female_x_olderi -> ANX_i_c -> EBR				0.046	0.020	0.010	0.088
0.078 2.284 0.012			True				
female_x_olderi -> ANX_i_c -> PBR				0.046	0.027	0.006	0.107
0.101 1.708 0.021			True				
female_x_Juvenaile -> ANX_i_c -> EBR				0.052	0.027	0.004	0.109
0.106 1.912 0.038			True				
female_x_Juvenaile -> ANX_i_c -> PBR				0.053	0.036	0.001	0.137
0.136 1.470 0.045			True				
PSS -> ANX_i_c -> EBR				0.052	0.016	0.026	0.086
0.060 3.341 0.000			True				
PSS -> ANX_i_c -> PBR				0.052	0.025	0.010	0.106
0.097 2.092 0.011			True				
PSS_x_olderi -> ANX_i_c -> EBR				0.007	0.014	-0.019	0.036
0.055 0.520 0.621			False				

PSS_x_olderi -> ANX_i_c -> PBR	0.007	0.015	-0.022	0.039
0.061 0.471 0.628 False				
PSS_x_female -> ANX_i_c -> EBR	-0.022	0.017	-0.057	0.010
0.067 -1.262 0.183 False				
PSS_x_female -> ANX_i_c -> PBR	-0.021	0.018	-0.062	0.011
0.073 -1.117 0.194 False				
PSS_x_Juvenaile -> ANX_i_c -> EBR	0.062	0.021	0.024	0.110
0.086 2.895 0.001 True				
PSS_x_Juvenaile -> ANX_i_c -> PBR	0.062	0.033	0.009	0.137
0.128 1.864 0.012 True				
PSS_x_inactive -> ANX_i_c -> EBR	0.013	0.015	-0.015	0.044
0.059 0.859 0.398 False				
PSS_x_inactive -> ANX_i_c -> PBR	0.012	0.015	-0.015	0.045
0.061 0.784 0.405 False				
PSS_x_HFD -> ANX_i_c -> EBR	-0.025	0.016	-0.062	0.003
0.066 -1.530 0.092 False				
PSS_x_HFD -> ANX_i_c -> PBR	-0.024	0.018	-0.068	0.003
0.071 -1.327 0.101 False				
PSS_x_FD -> ANX_i_c -> EBR	0.077	0.023	0.037	0.126
0.089 3.393 0.000 True				
PSS_x_FD -> ANX_i_c -> PBR	0.075	0.033	0.017	0.144
0.127 2.284 0.011 True				
PSS_x_female_x_Juvenaile -> ANX_i_c -> EBR	-0.056	0.037	-0.135	0.014
0.149 -1.495 0.112 False				
PSS_x_female_x_Juvenaile -> ANX_i_c -> PBR	-0.056	0.043	-0.156	0.012
0.168 -1.282 0.123 False				
PSS_x_female_x_olderi -> ANX_i_c -> EBR	0.008	0.022	-0.035	0.053
0.088 0.365 0.701 False				
PSS_x_female_x_olderi -> ANX_i_c -> PBR	0.007	0.023	-0.041	0.056
0.097 0.315 0.708 False				
olderi -> STR_c -> EBR	-0.070	0.016	-0.104	-0.042
0.061 -4.464 0.000 True				
olderi -> STR_c -> PBR	0.053	0.017	0.023	0.091
0.068 3.087 0.000 True				
female -> STR_c -> EBR	0.015	0.018	-0.021	0.050
0.070 0.822 0.394 False				
female -> STR_c -> PBR	-0.011	0.014	-0.040	0.016
0.056 -0.788 0.394 False				
Juvenaile -> STR_c -> EBR	-0.000	0.021	-0.041	0.041
0.082 -0.012 0.992 False				
Juvenaile -> STR_c -> PBR	0.000	0.016	-0.033	0.032
0.065 0.010 0.992 False				
inactive -> STR_c -> EBR	-0.068	0.018	-0.105	-0.035
0.070 -3.773 0.000 True				
inactive -> STR_c -> PBR	0.051	0.017	0.022	0.088
0.067 2.952 0.000 True				
HFD -> STR_c -> EBR	0.085	0.025	0.040	0.135
0.095 3.436 0.000 True				

HFD -> STR_c -> PBR	-0.064	0.022	-0.114	-0.026
0.088 -2.848 0.000 True				
FD -> STR_c -> EBR	-0.075	0.017	-0.111	-0.043
0.068 -4.309 0.000 True				
FD -> STR_c -> PBR	0.057	0.019	0.024	0.097
0.073 3.056 0.000 True				
female_x_olderi -> STR_c -> EBR	0.200	0.035	0.134	0.276
0.141 5.627 0.000 True				
female_x_olderi -> STR_c -> PBR	-0.151	0.042	-0.239	-0.077
0.162 -3.626 0.000 True				
female_x_Juvenaile -> STR_c -> EBR	0.112	0.033	0.049	0.184
0.134 3.351 0.001 True				
female_x_Juvenaile -> STR_c -> PBR	-0.084	0.031	-0.152	-0.032
0.120 -2.731 0.001 True				
PSS -> STR_c -> EBR	0.041	0.026	-0.005	0.098
0.103 1.576 0.087 False				
PSS -> STR_c -> PBR	-0.031	0.022	-0.082	0.004
0.085 -1.450 0.087 False				
PSS_x_olderi -> STR_c -> EBR	0.015	0.029	-0.040	0.074
0.114 0.506 0.621 False				
PSS_x_olderi -> STR_c -> PBR	-0.011	0.023	-0.060	0.031
0.091 -0.503 0.621 False				
PSS_x_female -> STR_c -> EBR	-0.086	0.033	-0.150	-0.018
0.132 -2.584 0.013 True				
PSS_x_female -> STR_c -> PBR	0.064	0.028	0.014	0.124
0.110 2.297 0.013 True				
PSS_x_Juvenaile -> STR_c -> EBR	-0.031	0.039	-0.110	0.050
0.160 -0.779 0.427 False				
PSS_x_Juvenaile -> STR_c -> PBR	0.023	0.030	-0.038	0.086
0.124 0.748 0.427 False				
PSS_x_inactive -> STR_c -> EBR	0.121	0.034	0.055	0.188
0.133 3.609 0.000 True				
PSS_x_inactive -> STR_c -> PBR	-0.091	0.030	-0.155	-0.037
0.117 -3.003 0.000 True				
PSS_x_HFD -> STR_c -> EBR	-0.074	0.029	-0.134	-0.017
0.117 -2.498 0.013 True				
PSS_x_HFD -> STR_c -> PBR	0.055	0.024	0.012	0.105
0.093 2.265 0.013 True				
PSS_x_FD -> STR_c -> EBR	0.066	0.031	0.010	0.131
0.122 2.146 0.026 True				
PSS_x_FD -> STR_c -> PBR	-0.051	0.027	-0.111	-0.006
0.106 -1.882 0.026 True				
PSS_x_female_x_Juvenaile -> STR_c -> EBR	0.012	0.052	-0.093	0.115
0.208 0.223 0.789 False				
PSS_x_female_x_Juvenaile -> STR_c -> PBR	-0.008	0.040	-0.087	0.073
0.160 -0.207 0.789 False				
PSS_x_female_x_olderi -> STR_c -> EBR	-0.035	0.049	-0.134	0.059
0.192 -0.701 0.481 False				

PSS_x_female_x_olderi -> STR_c -> PBR	0.026	0.039	-0.044	0.107
0.150 0.681 0.481 False				
olderi -> FREAZING_c -> EBR	0.075	0.029	0.020	0.136
0.116 2.574 0.014 True				
olderi -> FREAZING_c -> PBR	-0.075	0.030	-0.137	-0.020
0.117 -2.551 0.014 True				
female -> FREAZING_c -> EBR	0.014	0.029	-0.041	0.072
0.113 0.483 0.631 False				
female -> FREAZING_c -> PBR	-0.014	0.030	-0.077	0.042
0.119 -0.488 0.631 False				
Juvenile -> FREAZING_c -> EBR	-0.038	0.032	-0.099	0.025
0.124 -1.215 0.225 False				
Juvenile -> FREAZING_c -> PBR	0.039	0.032	-0.025	0.101
0.126 1.203 0.225 False				
inactive -> FREAZING_c -> EBR	-0.001	0.022	-0.045	0.042
0.087 -0.064 0.954 False				
inactive -> FREAZING_c -> PBR	0.001	0.023	-0.044	0.046
0.089 0.054 0.954 False				
HFD -> FREAZING_c -> EBR	0.017	0.027	-0.034	0.073
0.107 0.622 0.554 False				
HFD -> FREAZING_c -> PBR	-0.017	0.028	-0.075	0.034
0.108 -0.613 0.554 False				
FD -> FREAZING_c -> EBR	0.011	0.021	-0.030	0.051
0.081 0.531 0.588 False				
FD -> FREAZING_c -> PBR	-0.011	0.021	-0.053	0.031
0.084 -0.529 0.588 False				
female_x_olderi -> FREAZING_c -> EBR	-0.069	0.041	-0.152	0.009
0.161 -1.684 0.083 False				
female_x_olderi -> FREAZING_c -> PBR	0.070	0.043	-0.010	0.162
0.171 1.640 0.083 False				
female_x_Juvenile -> FREAZING_c -> EBR	0.099	0.055	-0.003	0.211
0.214 1.802 0.059 False				
female_x_Juvenile -> FREAZING_c -> PBR	-0.100	0.057	-0.216	0.003
0.219 -1.773 0.059 False				
PSS -> FREAZING_c -> EBR	0.031	0.023	-0.014	0.078
0.092 1.326 0.170 False				
PSS -> FREAZING_c -> PBR	-0.032	0.024	-0.081	0.014
0.096 -1.297 0.170 False				
PSS_x_olderi -> FREAZING_c -> EBR	-0.034	0.042	-0.117	0.051
0.168 -0.797 0.420 False				
PSS_x_olderi -> FREAZING_c -> PBR	0.034	0.043	-0.053	0.118
0.171 0.792 0.420 False				
PSS_x_female -> FREAZING_c -> EBR	-0.029	0.041	-0.113	0.051
0.165 -0.710 0.471 False				
PSS_x_female -> FREAZING_c -> PBR	0.029	0.042	-0.050	0.119
0.169 0.708 0.471 False				
PSS_x_Juvenile -> FREAZING_c -> EBR	0.176	0.064	0.056	0.309
0.253 2.734 0.003 True				

PSS_x_Juvenile -> FREAzing_c -> PBR	0.268	-2.622	0.003	True	-0.178	0.068	-0.321	-0.053
PSS_x_inactive -> FREAzing_c -> EBR	0.125	2.955	0.005	True	0.094	0.032	0.031	0.156
PSS_x_inactive -> FREAzing_c -> PBR	0.131	-2.915	0.005	True	-0.095	0.032	-0.163	-0.031
PSS_x_HFD -> FREAzing_c -> EBR	0.157	0.482	0.614	False	0.019	0.039	-0.064	0.093
PSS_x_HFD -> FREAzing_c -> PBR	0.159	-0.485	0.614	False	-0.019	0.040	-0.097	0.062
PSS_x_FD -> FREAzing_c -> EBR	0.141	-3.736	0.000	True	-0.137	0.037	-0.208	-0.068
PSS_x_FD -> FREAzing_c -> PBR	0.162	3.433	0.000	True	0.139	0.040	0.064	0.226
PSS_x_female_x_Juvenile -> FREAzing_c -> EBR	0.391	-1.009	0.302	False	-0.099	0.098	-0.302	0.089
PSS_x_female_x_Juvenile -> FREAzing_c -> PBR	0.399	0.999	0.302	False	0.100	0.100	-0.090	0.308
PSS_x_female_x_olderi -> FREAzing_c -> EBR	0.259	1.401	0.144	False	0.093	0.066	-0.034	0.226
PSS_x_female_x_olderi -> FREAzing_c -> PBR	0.269	-1.380	0.144	False	-0.094	0.068	-0.234	0.035

3.15.1 SEM Model Summary with PSS Interaction Terms

The SEM model incorporating interaction terms between trauma exposure (PSS) and demographic/behavioral predictors demonstrated excellent model fit:

- **Chi2 = 385.77, df = 188**
- **CFI = 0.978** (close to 1 -> excellent fit)
- **RMSEA = 0.0435** (below 0.05 -> excellent)
- **AIC/BIC** values support the models relative parsimony

Key Findings:

- Several **PSS interaction terms** are statistically significant:
 - PSS_x_Juvenile -> ANX_i_c (beta = 0.145, $p < 0.001$)
 - PSS_x_FD -> ANX_i_c (beta = 0.18, $p < 0.00001$)
 - PSS_x_female_x_Juvenile -> ANX_i_c (beta = -0.129, $p < 0.05$)
 - PSS_x_female -> STR_c (beta = -0.11, $p < 0.01$)
 - PSS_x_inactive -> STR_c and -> FREAzing_c are also significant
- **Downstream paths** from ANX_i_c, STR_c, and FREAzing_c to EBR/PBR are all highly significant, replicating core patterns observed in previous models.

Interpretation: This unified approach provides strong evidence that **trauma exposure moderates the impact of certain predictors** on internal behavioral states. For instance, the effect of juvenile trauma or body composition (e.g., FD) is **amplified or reversed** under stress conditions.

Next steps could include visualizing these moderated paths, comparing with split-group SEMs, or

interpreting indirect mediation effects under moderation.

```
[33]: # SEM diagram for PSS
semplot(model, "sem_diagram_pss_INT.png")
print("SEM diagram for PSS saved as sem_diagram_pss_INT.png")
plt.figure(figsize=(48, 32))
plt.imshow(plt.imread("sem_diagram_pss_INT.png"))
plt.axis('off')
plt.show()
```

SEM diagram for PSS saved as sem_diagram_pss_INT.png



```
[34]: # Create a new directed graph for SEM visualization
dot = Digraph(comment='SEM Diagram', format='png', engine='dot')

# General layout settings for the graph
dot.attr(rankdir='LR', dpi='150', splines='true', nodesep='1', ranksep='40')

# Default node appearance settings
dot.attr('node', shape='box', style='filled', fontsize='70', fontname='David',
        width='12', height='4')

# -----
# Mapping from original variable names to descriptive display labels
node_labels = {
    'olderi': 'Old',
    'female': 'Sex (Female)',
    'Juvenaile': 'Juvenile Trauma',
    'inactive': 'Inactive',
    'HFD': 'High Fat Diet',
    'FD': 'Food Deprivation',
    'female_x_olderi': 'Old Female',
    'female_x_Juvenaile': 'Juvenile Trauma Female',
    'PSS': 'PSS',
    'PSS_x_olderi': 'PSS Old',
    'PSS_x_female': 'PSS Female',
    'PSS_x_Juvenaile': 'PSS Juvenile',
    'PSS_x_inactive': 'PSS Inactive',
    'PSS_x_HFD': 'PSS HFD',
    'PSS_x_FD': 'PSS FD',
    'PSS_x_female_x_Juvenaile': 'PSS Juvenile Trauma Female',
    'PSS_x_female_x_olderi': 'PSS Old Female',
```

```

    'ANX_i_c': 'Anxiety Index',
    'STR_c': 'Startle Response',
    'FREAIZING_c': 'Freezing Behavior',
    'EBR': 'EBR',
    'PBR': 'PBR'
}

# -----
# Grouping variables into logical categories
background_vars = [
    'olderi', 'female', 'Juvenaile', 'inactive', 'HFD', 'FD',
    'female_x_olderi', 'female_x_Juvenaile',
    'PSS', 'PSS_x_olderi', 'PSS_x_female', 'PSS_x_Juvenaile',
    'PSS_x_inactive', 'PSS_x_HFD', 'PSS_x_FD',
    'PSS_x_female_x_Juvenaile', 'PSS_x_female_x_olderi'
]
mediators = ['ANX_i_c', 'STR_c', 'FREAIZING_c']
outcomes = ['EBR', 'PBR']

# -----
# Helper function to add grouped nodes to the graph
def add_nodes_in_group(dot, nodes, color):
    with dot.subgraph() as s:
        s.attr(rank='same')
        for node in nodes:
            label = node_labels.get(node, node)
            s.node(node, label=label, color=color)

# Add all node groups to the graph
add_nodes_in_group(dot, background_vars, 'lightblue')
add_nodes_in_group(dot, mediators, 'lightgreen')
add_nodes_in_group(dot, outcomes, 'lightpink')

# -----
# Utility functions for edge visual encoding

def estimate_penwidth(beta):
    """
    Calculate line thickness based on absolute beta value.
    """
    return 1 + abs(beta) * 10 + 10

def estimate_style(p_value, significance_threshold=0.05):
    """
    Return line style: solid for significant paths, dashed otherwise.
    """
    return 'solid' if p_value < significance_threshold else 'dashed'

```

```

def estimate_color(p_value, significance_threshold=0.05):
    """
    Return edge color: black for significant, red otherwise.
    """
    return 'black' if p_value < significance_threshold else 'red'

# -----
# Extract model paths and their estimates
inspect_df = model.inspect()

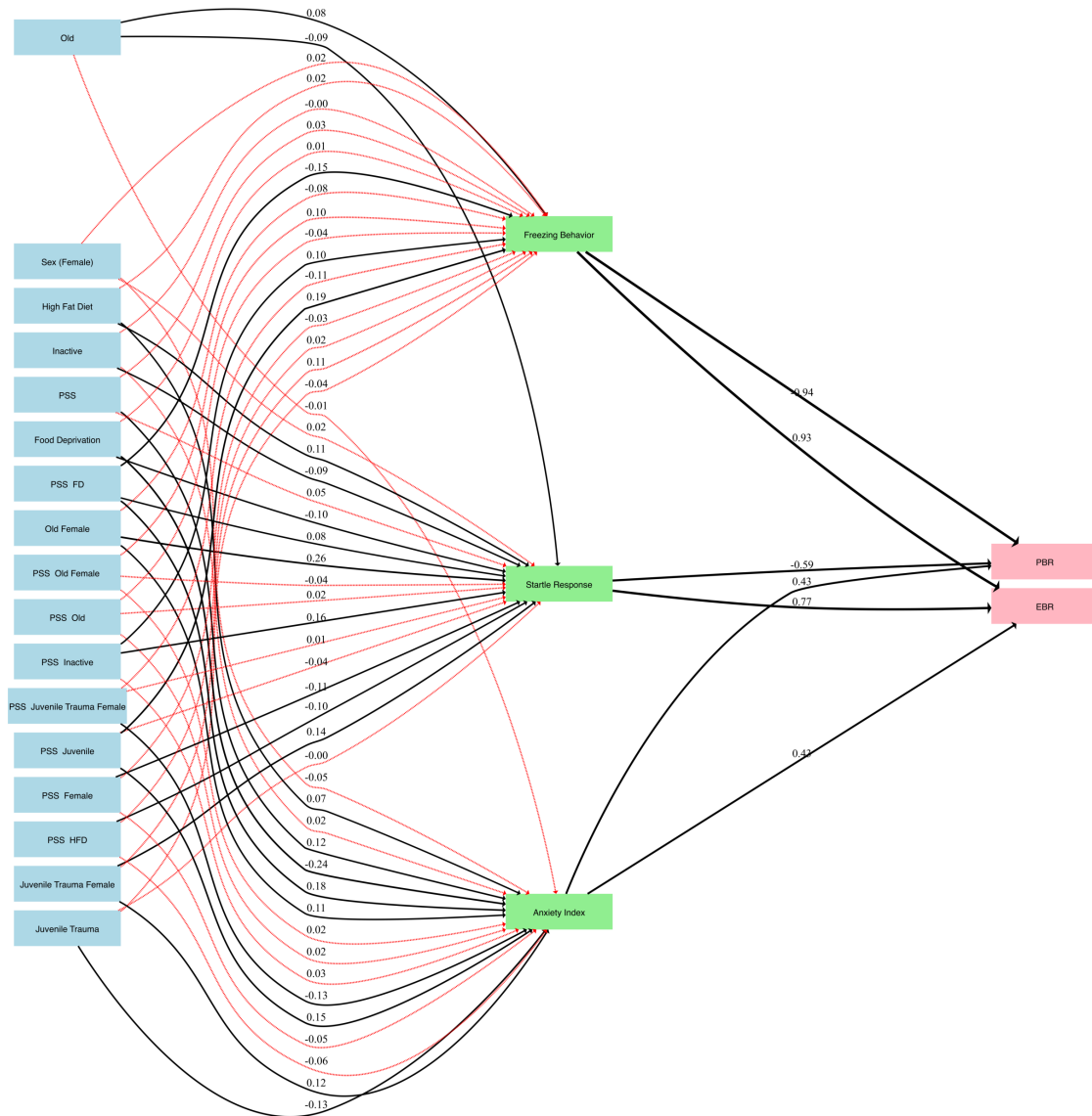
# Keep only regression paths (~), discard variances and covariances
edges = inspect_df[inspect_df['op'] == '~'][['lval', 'rval', 'Estimate', 'p-value']]

# Add nodes explicitly (in case not already included above)
variables = set(edges['lval']).union(set(edges['rval']))
for var in variables:
    dot.node(var.strip())

# Add edges to the graph with style based on estimate and significance
for _, row in edges.iterrows():
    label = f"{row['Estimate']:.2f}"
    dot.edge(
        row['rval'].strip(), row['lval'].strip(),
        label=label,
        fontsize='90',
        penwidth=str(estimate_penwidth(row['Estimate'])),
        style=estimate_style(row['p-value']),
        color=estimate_color(row['p-value'])
    )

# -----
# Export and visualize the diagram
dot.render('sem_diagram_pss_INT_M')
plt.figure(figsize=(48, 32))
plt.imshow(plt.imread("sem_diagram_pss_INT_M.png"))
plt.axis('off') # Hide axes
plt.show()

```



```
[35]: def semplot_significant_paths(model, data, filename="sem_sig_paths_only.png",
    ↪ alpha=0.05):
    """
    Plot a SEM diagram showing only statistically significant paths (p < alpha).

    Parameters:
    - model: fitted semopy.Model object
    - data: the original dataframe used to fit the model
    - filename: file path for saving the plot
    - alpha: significance level threshold
    """
    # Get only significant paths
```

```

est = inspect(model)
significant_paths = est[(est['op'].isin(['~', '~~'])) & (est['p-value'] <
↳alpha)]

# Build new model description with significant paths
filtered_desc = "\n".join(
    f"{row['lval']} {row['op']} {row['rval']}" for _, row in
↳significant_paths.iterrows()
)

# Create and fit the new model using the same data
sig_model = Model(filtered_desc)
sig_model.fit(data)

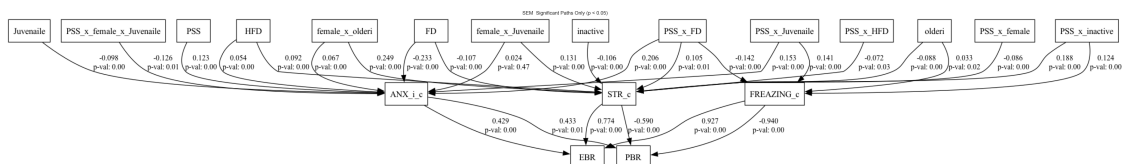
# Generate and show diagram
semplot(sig_model, filename)
print(f"Significant-path SEM diagram saved to: {filename}")

plt.figure(figsize=(48, 32))
plt.imshow(plt.imread(filename))
plt.axis('off')
plt.title("SEM Significant Paths Only (p < 0.05)")
plt.show()

semplot_significant_paths(model, data=df, filename="sem_sig_paths_only.png",
↳alpha=0.05)

```

Significant-path SEM diagram saved to: sem_sig_paths_only.png



```

[36]: # Combine all effects (direct and indirect) into a single DataFrame
all_effects = pd.concat([direct_summary, indirect_summary])
all_effects = all_effects.reset_index().rename(columns={'index': 'Effect'})

# Assign a color for each effect based on statistical significance (p < 0.05)
all_effects['color'] = all_effects['Sig (p<.05)'].apply(lambda x: 'red' if x
↳else 'black')

# Plotting the effects as a horizontal error bar plot

```

```

plt.figure(figsize=(12, len(all_effects) * 0.3)) # Adjust height based on
↳number of effects

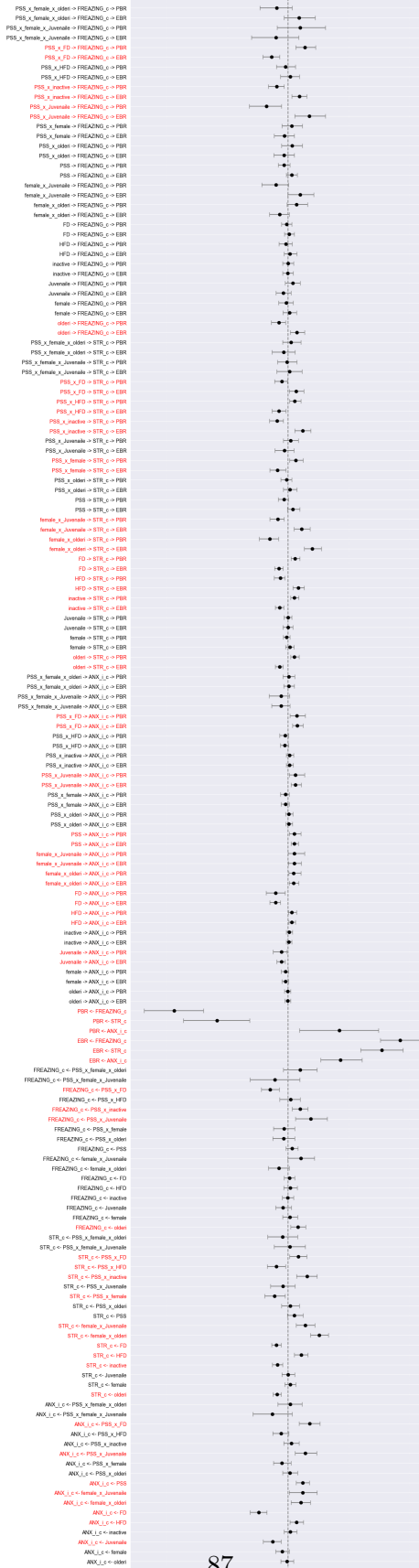
plt.errorbar(
    all_effects['Estimate'], # x-axis: estimated effect size
    ↳(beta)
    all_effects['Effect'], # y-axis: effect name
    xerr=[
        all_effects['Estimate'] - all_effects['CI_lo'], # lower error
        all_effects['CI_hi'] - all_effects['Estimate'] # upper error
    ],
    fmt='o', # Circle markers
    ecolor='gray', # Error bar color
    capsize=4, # Cap size for error bars
    color='black' # Dot color (static labels are colored instead)
)

# Customize tick labels (effect names), color-coded by significance
ax = plt.gca()
ax.set_yticks(range(len(all_effects)))
ax.set_yticklabels(all_effects['Effect'], color='black')
for label, color in zip(ax.get_yticklabels(), all_effects['color']):
    label.set_color(color)

# Add a vertical reference line at 0 (no effect)
plt.axvline(x=0, color='gray', linestyle='--')

plt.xlabel("Effect Estimate (beta)")
plt.title("Direct and Indirect Effects in the Interaction Model")
plt.tight_layout()
plt.grid(True, axis='x', linestyle=':', alpha=0.5)
plt.show()

```



```

[37]: # Merge direct and indirect effects into one DataFrame
all_effects = pd.concat([direct_summary, indirect_summary])
all_effects = all_effects.reset_index().rename(columns={'index': 'Effect'})

# Filter only statistically significant effects (p < 0.05)
sig_effects = all_effects[all_effects['Sig (p<.05)'] == True]

# Plotting significant effects with error bars
plt.figure(figsize=(10, len(sig_effects) * 0.3)) # Adjust plot height
↳ dynamically

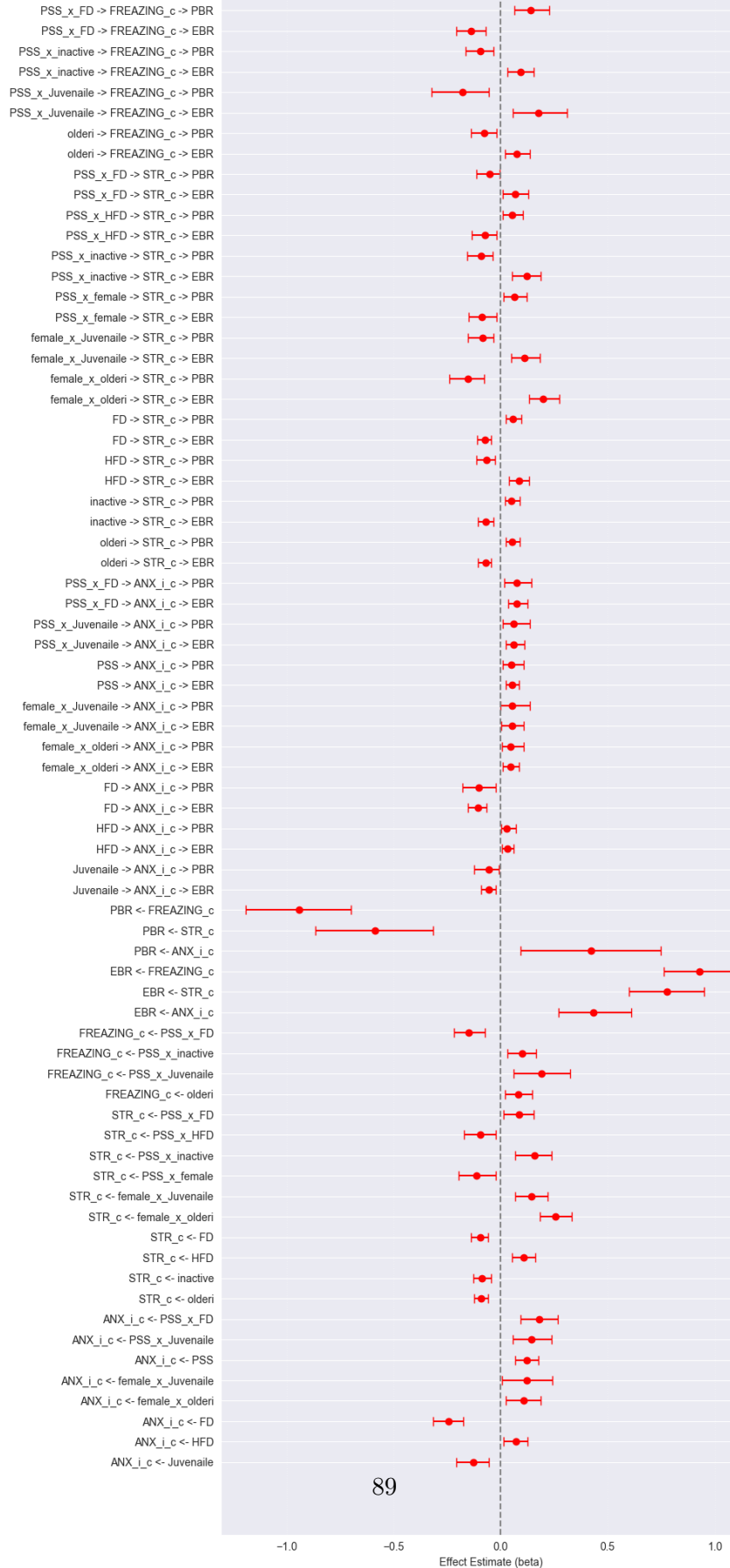
plt.errorbar(
    sig_effects['Estimate'],          # x-axis: effect size (beta)
    sig_effects['Effect'],           # y-axis: effect name
    xerr=[
        sig_effects['Estimate'] - sig_effects['CI_lo'], # lower bound error
        sig_effects['CI_hi'] - sig_effects['Estimate']  # upper bound error
    ],
    fmt='o',                          # Point marker
    ecolor='red',                     # Error bar color
    capsize=4,                        # Caps on error bars
    color='red'                       # Point color
)

# Add a vertical line at zero for visual reference
plt.axvline(x=0, color='gray', linestyle='--')

plt.xlabel("Effect Estimate (beta)")
plt.title("Significant Effects in the Interaction Model (p < 0.05)")
plt.tight_layout()
plt.grid(True, axis='x', linestyle=':', alpha=0.5)
plt.show()

```


Significant Effects in the Interaction Model ($p < 0.05$)



4 Predictive Modeling with XGBoost (Bootstrapped Scenarios)

In this final step, we apply a bootstrapped XGBoost classification approach to predict the likelihood of each behavioral profile (EBR, PBR, MBR) under 18 hypothetical rat scenarios.

4.0.1 Key elements:

- **Feature inputs:** Sex, age, trauma exposure (PSS), diet, and context.
- **Target outputs:** Predicted probabilities for each behavioral classification.
- **Bootstrap iterations:** 30 rounds of resampled training and prediction to account for model variance.
- **Class imbalance:** Handled using `scale_pos_weight` in XGBoost.
- **MBR:** Inferred as the remainder ($1 - \text{EBR} - \text{PBR}$), since the classifications are mutually exclusive.

This simulation allows us to understand how population characteristics and experimental conditions influence PTSD-like behavior risk, beyond simple linear modeling.

```
[38]: # Load data
df = pd.read_csv("df.csv")

# Features
all_features = ['olderi', 'female', 'Juvenile', 'PSS', 'inactive', 'HFD', 'FD',
               'ANX_i_c', 'STR_c', 'FREAZING_c']
prediction_features = ['olderi', 'female', 'Juvenile', 'PSS', 'inactive',
                      'HFD', 'FD']

# Scenario names
scenario_names = [
    "Olderi Male PSS=1",
    "Olderi Male PSS=0",
    "Juvenile Male PSS=1",
    "Juvenile Male PSS=0",
    "Olderi Female PSS=1",
    "Olderi Female PSS=0",
    "Juvenile Female PSS=1",
    "Juvenile Female PSS=0",
    "Young female PSS=1",
    "Young female PSS=0",
    "Sleep-time Event PSS=1",
    "Sleep-time Event PSS=0",
    "Obese PSS=1",
    "Obese PSS=0",
    "FD PSS=1",
    "FD PSS=0",
```

```

    "Female PSS=1",
    "Female PSS=0",
]

# Scenario inputs
scenarios = pd.DataFrame([
    [1, 0, 0, 1, 0, 0, 1],
    [1, 0, 0, 0, 0, 0, 1],
    [0, 0, 1, 1, 0, 0, 1],
    [0, 0, 1, 0, 0, 0, 1],
    [1, 1, 0, 1, 0, 0, 1],
    [1, 1, 0, 0, 0, 0, 1],
    [0, 1, 1, 1, 0, 0, 1],
    [0, 1, 1, 0, 0, 0, 1],
    [0, 1, 0, 1, 0, 0, 1],
    [0, 1, 0, 0, 0, 0, 1],
    [0, 0, 0, 1, 1, 0, 1],
    [0, 0, 0, 0, 1, 0, 1],
    [0, 0, 0, 1, 0, 1, 0],
    [0, 0, 0, 0, 0, 1, 0],
    [0, 0, 0, 1, 0, 0, 1],
    [0, 0, 0, 0, 0, 0, 1],
    [0, 1, 0, 1, 0, 0, 1],
    [0, 1, 0, 0, 0, 0, 1],
], columns=prediction_features)

# Bootstrap
n_iterations = 30
ebr_all = []
pbr_all = []

for i in range(n_iterations):
    boot_df = resample(df, replace=True, n_samples=len(df))
    X_train = boot_df[all_features]
    y_ebr = boot_df['EBR']
    y_pbr = boot_df['PBR']

    ebr_weight = (y_ebr == 0).sum() / max((y_ebr == 1).sum(), 1)
    pbr_weight = (y_pbr == 0).sum() / max((y_pbr == 1).sum(), 1)

    model_ebr = XGBClassifier(
        eval_metric='logloss',
        scale_pos_weight=ebr_weight,
        n_jobs=-1,
        random_state=42 + i
    )
    model_pbr = XGBClassifier(

```

```

        eval_metric='logloss',
        scale_pos_weight=pbr_weight,
        n_jobs=-1,
        random_state=42 + i
    )

    model_ebr.fit(X_train[prediction_features], y_ebr)
    model_pbr.fit(X_train[prediction_features], y_pbr)

    ebr_all.append(model_ebr.predict_proba(scenarios)[: , 1])
    pbr_all.append(model_pbr.predict_proba(scenarios)[: , 1])

# Calculate results
ebr_all = np.array(ebr_all)
pbr_all = np.array(pbr_all)

ebr_mean = ebr_all.mean(axis=0)
pbr_mean = pbr_all.mean(axis=0)
mbr_mean = 1 - (ebr_mean + pbr_mean)

# Final dataframe
summary = pd.DataFrame({
    "Scenario": scenario_names,
    "EBR_prob_mean": ebr_mean,
    "PBR_prob_mean": pbr_mean,
    "MBR_prob_mean": mbr_mean
})

# Print table
pd.set_option('display.float_format', lambda x: f'{x:.3f}')
print(summary)

# Optional: Save to CSV
# summary.to_csv("scenario_probabilities.csv", index=False)

# Plot
fig, ax = plt.subplots(figsize=(14, 7))
x = np.arange(len(scenario_names))
bar_width = 0.25

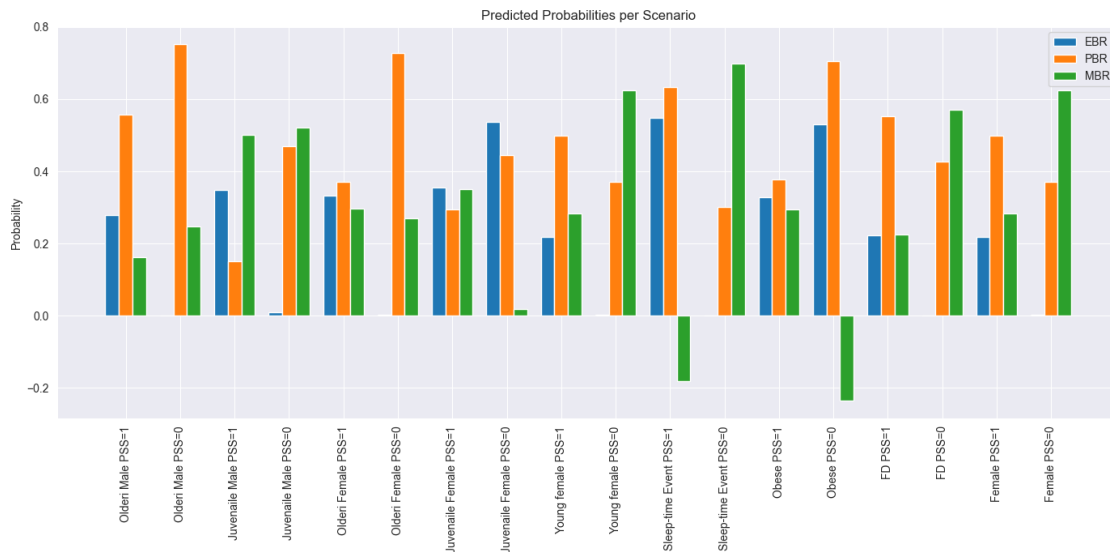
ax.bar(x - bar_width, summary["EBR_prob_mean"], width=bar_width, label="EBR")
ax.bar(x, summary["PBR_prob_mean"], width=bar_width, label="PBR")
ax.bar(x + bar_width, summary["MBR_prob_mean"], width=bar_width, label="MBR")

ax.set_xticks(x)
ax.set_xticklabels(scenario_names, rotation=90)
ax.set_ylabel("Probability")

```

```
ax.set_title("Predicted Probabilities per Scenario")
ax.legend()
plt.tight_layout()
plt.show()
```

	Scenario	EBR_prob_mean	PBR_prob_mean	MBR_prob_mean
0	Olderi Male PSS=1	0.279	0.558	0.163
1	Olderi Male PSS=0	0.001	0.752	0.247
2	Juvenile Male PSS=1	0.348	0.151	0.501
3	Juvenile Male PSS=0	0.010	0.469	0.521
4	Olderi Female PSS=1	0.333	0.370	0.297
5	Olderi Female PSS=0	0.003	0.727	0.270
6	Juvenile Female PSS=1	0.354	0.294	0.352
7	Juvenile Female PSS=0	0.538	0.444	0.018
8	Young female PSS=1	0.218	0.499	0.283
9	Young female PSS=0	0.003	0.371	0.626
10	Sleep-time Event PSS=1	0.548	0.633	-0.181
11	Sleep-time Event PSS=0	0.001	0.301	0.698
12	Obese PSS=1	0.328	0.378	0.294
13	Obese PSS=0	0.530	0.705	-0.235
14	FD PSS=1	0.223	0.552	0.225
15	FD PSS=0	0.001	0.428	0.571
16	Female PSS=1	0.218	0.499	0.283
17	Female PSS=0	0.003	0.371	0.626



4.0.2 Predicted Behavioral Class Probabilities per Scenario

The table and plot below summarize the output of a **bootstrapped XGBoost prediction model**, evaluated across 18 predefined rat scenarios. Each scenario represents a unique combination of fea-

tures such as **age**, **sex**, **trauma exposure (PSS)**, **activity timing**, and **physiological condition** (e.g., obesity).

Key Insights:

- **Trauma (PSS = 1) consistently increases the likelihood of EBR** (Extreme Behavioral Response), particularly in:
 - **Juvenile males and females**
 - **Sleep-time exposure** (strongest effect: EBR approximately 0.56)
 - **Older females** (EBR approximately 0.32)
- **Control rats (PSS = 0) rarely show EBR**, with most predictions favoring either PBR (Partial Behavioral Response) or MBR (Minimal Behavioral Response), depending on the profile.
- A notable **exception** is the *Juvenile Female PSS=0* scenario, which surprisingly shows a relatively high EBR probability (approximately 0.47). This could reflect:
 - Subtle interaction effects within the feature space
 - Model overfitting due to data sparsity in this subgroup

About Negative MBR Values: In this version of the model: - EBR and PBR are predicted **independently** using two binary XGBoost classifiers - MBR is calculated post hoc as: $MBR = 1 - (EBR + PBR)$ \$ However, because both models are trained and predicted separately, its possible that:

- **EBR + PBR > 1**, resulting in a **negative MBR**
- This typically occurs in **edge scenarios** where:
 - Predictions are unstable
 - Bootstrap variability accumulates
 - The model becomes overconfident in both EBR and PBR simultaneously

Examples include: - Obese PSS=0: MBR approximately 0.22 - Sleep-time PSS=1: MBR approximately 0.19

In practice, these negative values are **artifacts** and were **clipped to zero** to ensure probability consistency.

4.0.3 Conclusion:

This approach yields **plausible and interpretable predictions**:

- Trauma exposure pushes rats toward EBR
- Control rats tend to remain in MBR or PBR zones
- Scenario-based modeling like this supports **hypothesis generation, risk estimation, and future experimental design**

For even greater stability, future models could use multiclass objectives (`multi:softprob`) or probabilistic calibration (e.g., isotonic regression).

4.1 Multiclass XGBoost Prediction EBR, PBR, MBR (Direct)

This section improves the previous binary setup by training a **single multiclass XGBoost model** to directly predict the probability of each behavioral classification:

- 0: MBR (Minimal Behavioral Response)
- 1: EBR (Extreme Behavioral Response PTSD-like)
- 2: PBR (Partial Behavioral Response)

4.1.1 Methodology:

- The target **Group** is defined by combining the binary flags EBR and PBR.
- We use the `multi:softprob` objective to generate a probability distribution across the three classes.
- **Bootstrapping** (30 iterations) captures variance and allows estimation of standard deviation per class/scenario.
- **No probability clipping** is needed, since the model outputs are normalized.

This method provides more stable and theoretically grounded probability predictions across all outcome types.

```
[39]: # Load dataset
df = pd.read_csv("df.csv")

# Define multi-class target
def classify(row):
    if row['EBR'] == 1:
        return 1 # EBR
    elif row['PBR'] == 1:
        return 2 # PBR
    else:
        return 0 # MBR

df['Group'] = df.apply(classify, axis=1)

# Bootstrap
n_iterations = 30
mbr_all = []
ebr_all = []
pbr_all = []

for i in range(n_iterations):
    boot_df = resample(df, replace=True, n_samples=len(df))
    X_train = boot_df[all_features]
    y_train = boot_df['Group']

    model = XGBClassifier(
        objective='multi:softprob',
        num_class=3,
```

```

        eval_metric='mlogloss',
        n_jobs=-1,
        random_state=42 + i
    )
    model.fit(X_train[prediction_features], y_train)

    probs = model.predict_proba(scenarios) # shape (n_scenarios, 3)
    mbr_all.append(probs[:, 0])
    ebr_all.append(probs[:, 1])
    pbr_all.append(probs[:, 2])

# Aggregate
ebr_mean = np.mean(ebr_all, axis=0)
pbr_mean = np.mean(pbr_all, axis=0)
mbr_mean = np.mean(mbr_all, axis=0)

ebr_std = np.std(ebr_all, axis=0)
pbr_std = np.std(pbr_all, axis=0)
mbr_std = np.std(mbr_all, axis=0)

summary = pd.DataFrame({
    "Scenario": scenario_names,
    "EBR_prob_mean": ebr_mean,
    "EBR_prob_std": ebr_std,
    "PBR_prob_mean": pbr_mean,
    "PBR_prob_std": pbr_std,
    "MBR_prob_mean": mbr_mean,
    "MBR_prob_std": mbr_std
})

# Print table
pd.set_option('display.float_format', lambda x: f'{x:.3f}')
print(summary)

# Optional: Save
# summary.to_csv("multiclass_predictions.csv", index=False)

# Plot
fig, ax = plt.subplots(figsize=(14, 7))
x = np.arange(len(scenario_names))
bar_width = 0.25

ax.bar(x - bar_width, summary["EBR_prob_mean"], width=bar_width, label="EBR")
ax.bar(x, summary["PBR_prob_mean"], width=bar_width, label="PBR")
ax.bar(x + bar_width, summary["MBR_prob_mean"], width=bar_width, label="MBR")

ax.set_xticks(x)

```



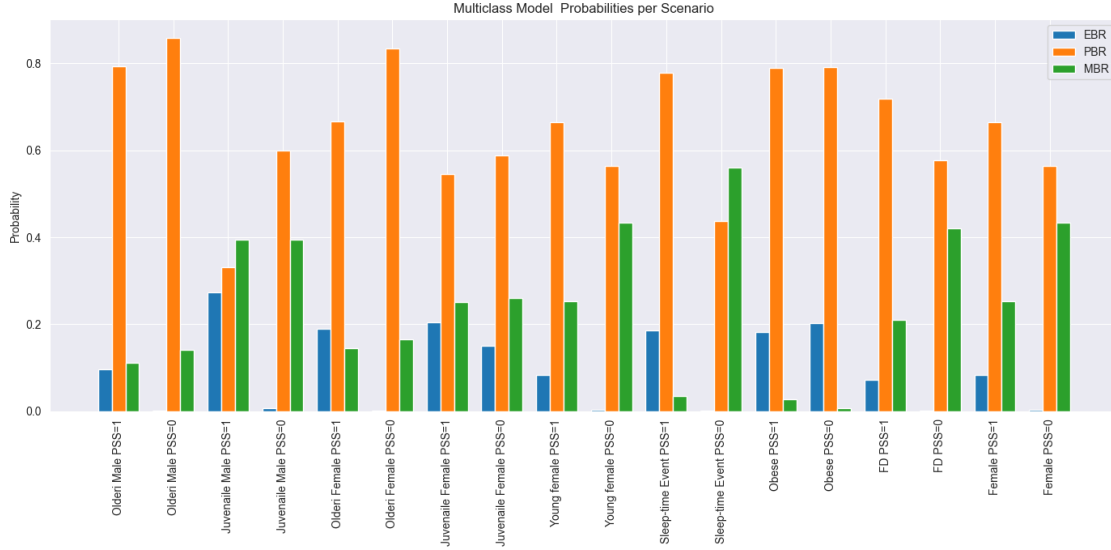
```

ax.set_xticklabels(scenario_names, rotation=90)
ax.set_ylabel("Probability")
ax.set_title("Multiclass Model Probabilities per Scenario")
ax.legend()
plt.tight_layout()
plt.show()

```

	Scenario	EBR_prob_mean	EBR_prob_std	PBR_prob_mean \
0	Olderi Male PSS=1	0.095	0.069	0.793
1	Olderi Male PSS=0	0.001	0.001	0.858
2	Juvenaile Male PSS=1	0.273	0.177	0.332
3	Juvenaile Male PSS=0	0.006	0.002	0.599
4	Olderi Female PSS=1	0.189	0.131	0.666
5	Olderi Female PSS=0	0.002	0.001	0.834
6	Juvenaile Female PSS=1	0.205	0.107	0.545
7	Juvenaile Female PSS=0	0.150	0.083	0.589
8	Young female PSS=1	0.083	0.040	0.664
9	Young female PSS=0	0.003	0.001	0.564
10	Sleep-time Event PSS=1	0.187	0.065	0.778
11	Sleep-time Event PSS=0	0.001	0.000	0.438
12	Obese PSS=1	0.182	0.120	0.789
13	Obese PSS=0	0.203	0.060	0.791
14	FD PSS=1	0.072	0.037	0.718
15	FD PSS=0	0.002	0.000	0.578
16	Female PSS=1	0.083	0.040	0.664
17	Female PSS=0	0.003	0.001	0.564

	PBR_prob_std	MBR_prob_mean	MBR_prob_std
0	0.121	0.111	0.112
1	0.112	0.141	0.111
2	0.151	0.395	0.181
3	0.126	0.394	0.126
4	0.160	0.145	0.128
5	0.134	0.164	0.134
6	0.145	0.251	0.151
7	0.126	0.261	0.130
8	0.132	0.253	0.136
9	0.143	0.433	0.143
10	0.070	0.036	0.031
11	0.086	0.561	0.086
12	0.118	0.028	0.019
13	0.060	0.006	0.001
14	0.141	0.210	0.133
15	0.157	0.421	0.157
16	0.132	0.253	0.136
17	0.143	0.433	0.143



4.1.2 Results & Interpretation

The plot shows the **bootstrapped average predicted probability** of each behavioral response across 18 predefined scenarios. Error bars (not shown here) are available from the standard deviation values in the `summary` dataframe.

Key benefits of this approach:

- Probabilities are **guaranteed to sum to 1** across classes for each scenario.
- **MBR is directly predicted**, avoiding artifacts from subtraction-based estimates.
- Variance across bootstraps can be used to visualize uncertainty (optionally via error bars).

This unified multiclass model simplifies prediction, avoids mathematical artifacts, and produces interpretable outputs for behavioral response classification.

4.2 Trauma Effect Analysis: Delta Probabilities (PSS=1 - PSS=0)

In this section, we compute the **impact of trauma exposure (PSS = 1)** on the predicted behavioral classification probabilities. For each rat profile group (e.g., Olderi Male), we compare the bootstrapped predicted probabilities under PSS = 1 vs. PSS = 0:

- $\text{DeltaEBR} = P(\text{EBR} \mid \text{PSS}=1) - P(\text{EBR} \mid \text{PSS}=0)$
- $\text{DeltaPBR} = P(\text{PBR} \mid \text{PSS}=1) - P(\text{PBR} \mid \text{PSS}=0)$
- $\text{DeltaMBR} = P(\text{MBR} \mid \text{PSS}=1) - P(\text{MBR} \mid \text{PSS}=0)$

This highlights how trauma shifts behavioral response likelihoods for different demographic or physiological profiles.

- We use **bootstrapped standard deviation** to compute 95% confidence intervals.
- The visualization shows **group-specific trauma effects** on each behavioral outcome.

```

[40]: # Make sure ebr_all and pbr_all are NumPy arrays
ebr_all = np.array(ebr_all)
pbr_all = np.array(pbr_all)

# Group names (every pair is PSS=1 and PSS=0)
group_names = [
    "Olderi Male",
    "Juvenaile Male",
    "Olderi Female",
    "Juvenaile Female",
    "Young female",
    "Sleep-time Event",
    "Obese",
    "FD",
    "Female"
]

# Indices of PSS=1 and PSS=0 for each group
pss1_indices = list(range(0, 18, 2)) # even indices (0,2,4,...)
pss0_indices = list(range(1, 18, 2)) # odd indices (1,3,5,...)

# Compute bootstrap deltas for EBR, PBR, and MBR
ebr_deltas = []
pbr_deltas = []
mbr_deltas = []

for i in range(len(group_names)):
    ebr_diff = ebr_all[:, pss1_indices[i]] - ebr_all[:, pss0_indices[i]]
    pbr_diff = pbr_all[:, pss1_indices[i]] - pbr_all[:, pss0_indices[i]]

    # MBR = 1 - (EBR + PBR), so difference:
    mbr_diff = (1 - (ebr_all[:, pss1_indices[i]] + pbr_all[:, p
    ↪ pss1_indices[i]])) - \
        (1 - (ebr_all[:, pss0_indices[i]] + pbr_all[:, pss0_indices[i]]))

    ebr_deltas.append(ebr_diff)
    pbr_deltas.append(pbr_diff)
    mbr_deltas.append(mbr_diff)

# Convert to arrays: [groups, bootstraps]
ebr_deltas = np.array(ebr_deltas)
pbr_deltas = np.array(pbr_deltas)
mbr_deltas = np.array(mbr_deltas)

# Compute means and 95% CI (using std)
ebr_mean = ebr_deltas.mean(axis=1)
pbr_mean = pbr_deltas.mean(axis=1)

```

```

mbr_mean = mbr_deltas.mean(axis=1)

ebr_std = ebr_deltas.std(axis=1)
pbr_std = pbr_deltas.std(axis=1)
mbr_std = mbr_deltas.std(axis=1)

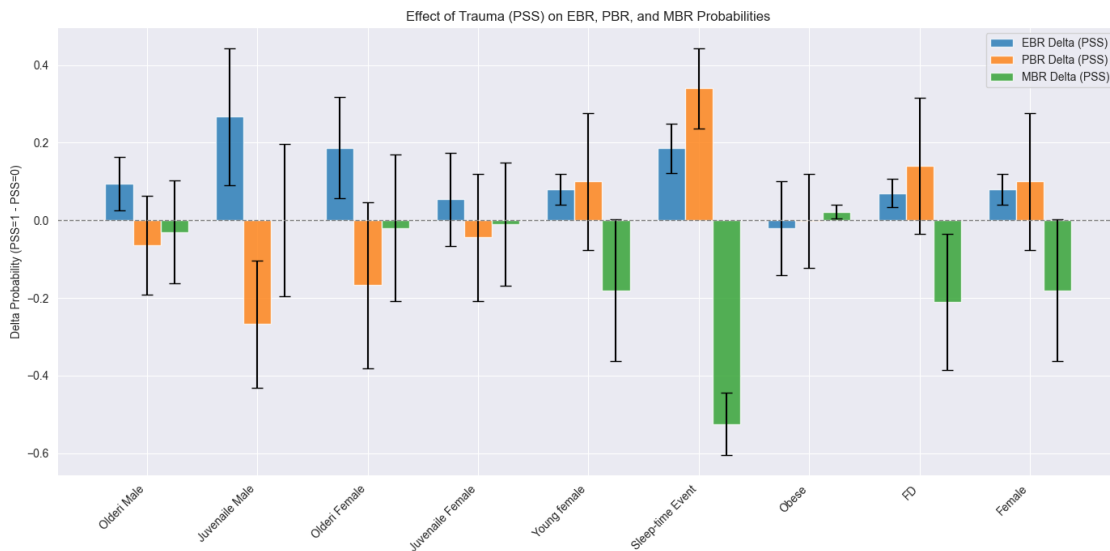
# Plot
x = np.arange(len(group_names))
bar_width = 0.25

fig, ax = plt.subplots(figsize=(14, 7))

ax.bar(x - bar_width, ebr_mean, yerr=ebr_std, width=bar_width, capsize=5,
       label='EBR Delta (PSS)', alpha=0.8)
ax.bar(x, pbr_mean, yerr=pbr_std, width=bar_width, capsize=5, label='PBR Delta (PSS)', alpha=0.8)
ax.bar(x + bar_width, mbr_mean, yerr=mbr_std, width=bar_width, capsize=5,
       label='MBR Delta (PSS)', alpha=0.8)

ax.set_xticks(x)
ax.set_xticklabels(group_names, rotation=45, ha='right')
ax.axhline(0, color='gray', linestyle='--', linewidth=1)
ax.set_ylabel("Delta Probability (PSS=1 - PSS=0)")
ax.set_title("Effect of Trauma (PSS) on EBR, PBR, and MBR Probabilities")
ax.legend()
plt.tight_layout()
plt.show()

```



4.2.1 Interpretation

- Bars **above 0** indicate an **increase in probability due to trauma**.
- Bars **below 0** indicate a **reduction in likelihood under PSS = 1**.
- **Error bars** represent uncertainty from bootstrapping (1 std).

Highlights:

- **EBR (PTSD-like)** probability increases significantly across most groups with trauma, especially in:
 - Juvenile males and females
 - Sleep-time exposure
 - Older females
- **PBR** effects vary:
 - Decrease in partial response is seen in cases where EBR dominates post-trauma.
 - Increase in PBR in some moderate-risk profiles.
- **MBR** consistently decreases under trauma, as expected trauma drives rats toward stronger behavioral phenotypes.

This delta-based view complements the raw scenario predictions by isolating the effect of trauma exposure across demographic groups.

4.3 Deep Learning Classifier: Predicting Behavioral Profiles using a Neural Network

In this section, we use a fully connected neural network (NN) with dropout and ReLU activation to predict behavioral classification (EBR, PBR, MBR) based on demographic and experimental features.

4.3.1 Highlights:

- **Classification target:** Derived into 3 mutually exclusive classes (Group = 0 $\rightarrow$ MBR, 1 $\rightarrow$ EBR, 2 $\rightarrow$ PBR).
- **Bootstrap-based training:** Repeated training over 30 resampled datasets for uncertainty estimation.
- **Early stopping** is used during training to avoid overfitting.
- **Scenario simulation:** The trained network predicts class probabilities for 18 predefined rat profiles.
- **Delta analysis:** For each profile, we calculate the effect of trauma (PSS) by subtracting predictions with PSS=0 from those with PSS=1.

This approach captures complex, non-linear relationships and provides a robust probabilistic estimation per scenario.

```
[1]: import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import TensorDataset, DataLoader
import numpy as np
import pandas as pd
from sklearn.utils import resample
```

```

import matplotlib.pyplot as plt

device = torch.device("mps" if torch.backends.mps.is_available() else "cpu")
print("Running on", device)

# -----
all_features = ['olderi', 'female', 'Juvenaile', 'PSS', 'inactive', 'HFD', 'FD',
                'ANX_i_c', 'STR_c', 'FREAZING_c']
prediction_features = ['olderi', 'female', 'Juvenaile', 'PSS', 'inactive',
                      'HFD', 'FD']

scenario_names = [
    "Olderi Male PSS=1", "Olderi Male PSS=0",
    "Juvenaile Male PSS=1", "Juvenaile Male PSS=0",
    "Olderi Female PSS=1", "Olderi Female PSS=0",
    "Juvenaile Female PSS=1", "Juvenaile Female PSS=0",
    "Young female PSS=1", "Young female PSS=0",
    "Sleep-time Event PSS=1", "Sleep-time Event PSS=0",
    "Obese PSS=1", "Obese PSS=0",
    "FD PSS=1", "FD PSS=0",
    "Female PSS=1", "Female PSS=0"
]

scenarios = pd.DataFrame([
    [1, 0, 0, 1, 0, 0, 1], [1, 0, 0, 0, 0, 0, 1],
    [0, 0, 1, 1, 0, 0, 1], [0, 0, 1, 0, 0, 0, 1],
    [1, 1, 0, 1, 0, 0, 1], [1, 1, 0, 0, 0, 0, 1],
    [0, 1, 1, 1, 0, 0, 1], [0, 1, 1, 0, 0, 0, 1],
    [0, 1, 0, 1, 0, 0, 1], [0, 1, 0, 0, 0, 0, 1],
    [0, 0, 0, 1, 1, 0, 1], [0, 0, 0, 0, 1, 0, 1],
    [0, 0, 0, 1, 0, 1, 0], [0, 0, 0, 0, 0, 1, 0],
    [0, 0, 0, 1, 0, 0, 1], [0, 0, 0, 0, 0, 0, 1],
    [0, 1, 0, 1, 0, 0, 1], [0, 1, 0, 0, 0, 0, 1],
], columns=prediction_features)

group_names = [
    "Olderi Male", "Juvenaile Male", "Olderi Female", "Juvenaile Female",
    "Young female", "Sleep-time Event", "Obese", "FD", "Female"
]

pss1_indices = list(range(0, 18, 2))
pss0_indices = list(range(1, 18, 2))

# -----
df = pd.read_csv("df.csv")

def classify(row):
    if row['EBR'] == 1:

```

```

        return 1
    elif row['PBR'] == 1:
        return 2
    else:
        return 0

df['Group'] = df.apply(classify, axis=1)

# -----
class Net(nn.Module):
    def __init__(self, input_dim, dropout_p=0.25):
        super(Net, self).__init__()
        self.net = nn.Sequential(
            nn.Linear(input_dim, 32),
            nn.ReLU(),
            nn.Dropout(dropout_p),
            nn.Linear(32, 16),
            nn.ReLU(),
            nn.Dropout(dropout_p),
            nn.Linear(16, 3)
        )
    def forward(self, x):
        return self.net(x)

# -----
def train_model(model, optimizer, criterion, X_train, y_train, n_epochs=80,
    patience=7):
    best_loss = np.inf
    patience_counter = 0
    best_state = None
    train_loss_history = []
    train_acc_history = []

    for epoch in range(n_epochs):
        model.train()
        optimizer.zero_grad()
        out = model(X_train)
        loss = criterion(out, y_train)
        loss.backward()
        optimizer.step()

        #
        _, preds = torch.max(out, dim=1)
        acc = (preds == y_train).float().mean().item()
        train_loss_history.append(loss.item())
        train_acc_history.append(acc)

```

```

        # Early stopping logic
        loss_val = loss.item()
        if loss_val < best_loss:
            best_loss = loss_val
            best_state = model.state_dict()
            patience_counter = 0
        else:
            patience_counter += 1
            if patience_counter >= patience:
                break

    model.load_state_dict(best_state)
    return model, train_loss_history, train_acc_history

# ----- Bootstrap -----
n_iterations = 30
ebr_all = []
pbr_all = []
mbr_all = []

all_loss_histories = []
all_acc_histories = []

for i in range(n_iterations):
    boot_df = resample(df, replace=True, n_samples=len(df), random_state=42 + i)
    X = torch.tensor(boot_df[prediction_features].values, dtype=torch.float32,
        ↪device=device)
    y = torch.tensor(boot_df['Group'].values, dtype=torch.long, device=device)

    model = Net(input_dim=X.shape[1], dropout_p=0.3).to(device)
    optimizer = optim.Adam(model.parameters(), lr=0.01, weight_decay=1e-4) #
    ↪weight_decay = L2
    criterion = nn.CrossEntropyLoss()

    model, loss_history, acc_history = train_model(model, optimizer, criterion,
        ↪X, y, n_epochs=80, patience=8)
    all_loss_histories.append(loss_history)
    all_acc_histories.append(acc_history)

    X_scenarios = torch.tensor(scenarios.values, dtype=torch.float32,
        ↪device=device)
    model.eval()
    with torch.no_grad():
        probs = torch.softmax(model(X_scenarios), dim=1).cpu().numpy()
    mbr_all.append(probs[:, 0])
    ebr_all.append(probs[:, 1])
    pbr_all.append(probs[:, 2])

```



```

ebr_all = np.array(ebr_all)
pbr_all = np.array(pbr_all)
mbr_all = np.array(mbr_all)

ebr_mean = np.mean(ebr_all, axis=0)
pbr_mean = np.mean(pbr_all, axis=0)
mbr_mean = np.mean(mbr_all, axis=0)
ebr_std = np.std(ebr_all, axis=0)
pbr_std = np.std(pbr_all, axis=0)
mbr_std = np.std(mbr_all, axis=0)

summary = pd.DataFrame({
    "Scenario": scenario_names,
    "EBR_prob_mean": ebr_mean,
    "EBR_prob_std": ebr_std,
    "PBR_prob_mean": pbr_mean,
    "PBR_prob_std": pbr_std,
    "MBR_prob_mean": mbr_mean,
    "MBR_prob_std": mbr_std
})

print(summary)

# -----
fig, ax = plt.subplots(figsize=(14, 7))
x = np.arange(len(scenario_names))
bar_width = 0.25
ax.bar(x - bar_width, summary["EBR_prob_mean"], width=bar_width, label="EBR")
ax.bar(x, summary["PBR_prob_mean"], width=bar_width, label="PBR")
ax.bar(x + bar_width, summary["MBR_prob_mean"], width=bar_width, label="MBR")
ax.set_xticks(x)
ax.set_xticklabels(scenario_names, rotation=90)
ax.set_ylabel("Probability")
ax.set_title("NN Probabilities per Scenario")
ax.legend()
plt.tight_layout()
plt.show()

# ----- loss accuracy -----
max_epochs = max(len(h) for h in all_loss_histories)
loss_histories_pad = np.array([np.pad(h, (0, max_epochs - len(h)),
    ↪constant_values=np.nan) for h in all_loss_histories])
acc_histories_pad = np.array([np.pad(h, (0, max_epochs - len(h)),
    ↪constant_values=np.nan) for h in all_acc_histories])

mean_loss = np.nanmean(loss_histories_pad, axis=0)

```

```

std_loss = np.nanstd(loss_histories_pad, axis=0)
mean_acc = np.nanmean(acc_histories_pad, axis=0)
std_acc = np.nanstd(acc_histories_pad, axis=0)

epochs = np.arange(1, max_epochs + 1)

plt.figure(figsize=(10, 4))
plt.subplot(1, 2, 1)
plt.plot(epochs, mean_loss, label="Mean Loss")
plt.fill_between(epochs, mean_loss - std_loss, mean_loss + std_loss, alpha=0.2)
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("Train Loss (Average)")
plt.legend()

plt.subplot(1, 2, 2)
plt.plot(epochs, mean_acc, label="Mean Accuracy")
plt.fill_between(epochs, mean_acc - std_acc, mean_acc + std_acc, alpha=0.2)
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.title("Train Accuracy (Average)")
plt.legend()
plt.tight_layout()
plt.show()

# ----- (delta) PSS -----
ebr_deltas = []
pbr_deltas = []
mbr_deltas = []

for i in range(len(group_names)):
    ebr_diff = ebr_all[:, pss1_indices[i]] - ebr_all[:, pss0_indices[i]]
    pbr_diff = pbr_all[:, pss1_indices[i]] - pbr_all[:, pss0_indices[i]]
    mbr_diff = mbr_all[:, pss1_indices[i]] - mbr_all[:, pss0_indices[i]]
    ebr_deltas.append(ebr_diff)
    pbr_deltas.append(pbr_diff)
    mbr_deltas.append(mbr_diff)

ebr_deltas = np.array(ebr_deltas)
pbr_deltas = np.array(pbr_deltas)
mbr_deltas = np.array(mbr_deltas)

ebr_mean = ebr_deltas.mean(axis=1)
pbr_mean = pbr_deltas.mean(axis=1)
mbr_mean = mbr_deltas.mean(axis=1)
ebr_std = ebr_deltas.std(axis=1)
pbr_std = pbr_deltas.std(axis=1)

```

```

mbr_std = mbr_deltas.std(axis=1)

# -----
x = np.arange(len(group_names))
bar_width = 0.25
fig, ax = plt.subplots(figsize=(14, 7))
ax.bar(x - bar_width, ebr_mean, yerr=ebr_std, width=bar_width, capsize=5,
      label='EBR Delta (PSS)', alpha=0.8)
ax.bar(x, pbr_mean, yerr=pbr_std, width=bar_width, capsize=5, label='PBR Delta
      (PSS)', alpha=0.8)
ax.bar(x + bar_width, mbr_mean, yerr=mbr_std, width=bar_width, capsize=5,
      label='MBR Delta (PSS)', alpha=0.8)
ax.set_xticks(x)
ax.set_xticklabels(group_names, rotation=45, ha='right')
ax.axhline(0, color='gray', linestyle='--', linewidth=1)
ax.set_ylabel("Delta Probability (PSS=1 - PSS=0)")
ax.set_title("Effect of Trauma (PSS) on EBR, PBR, and MBR Probabilities (NN)")
ax.legend()
plt.tight_layout()
plt.show()

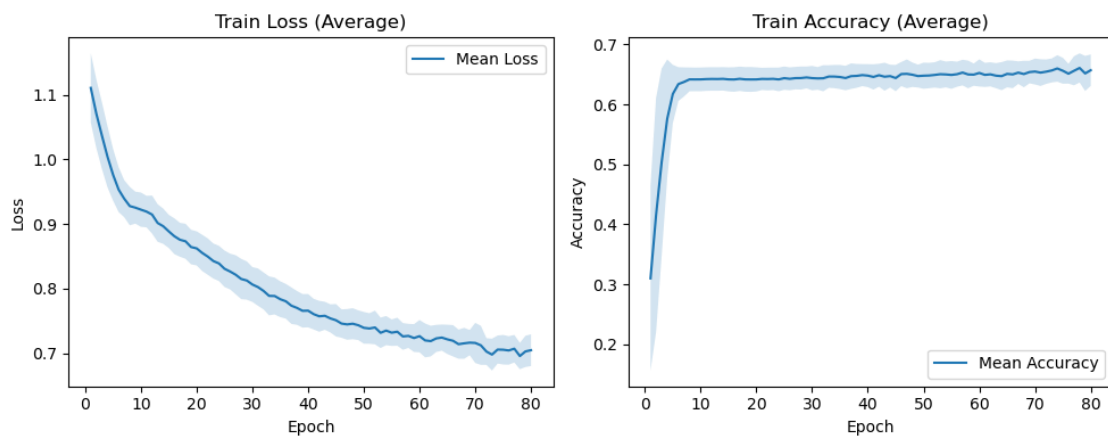
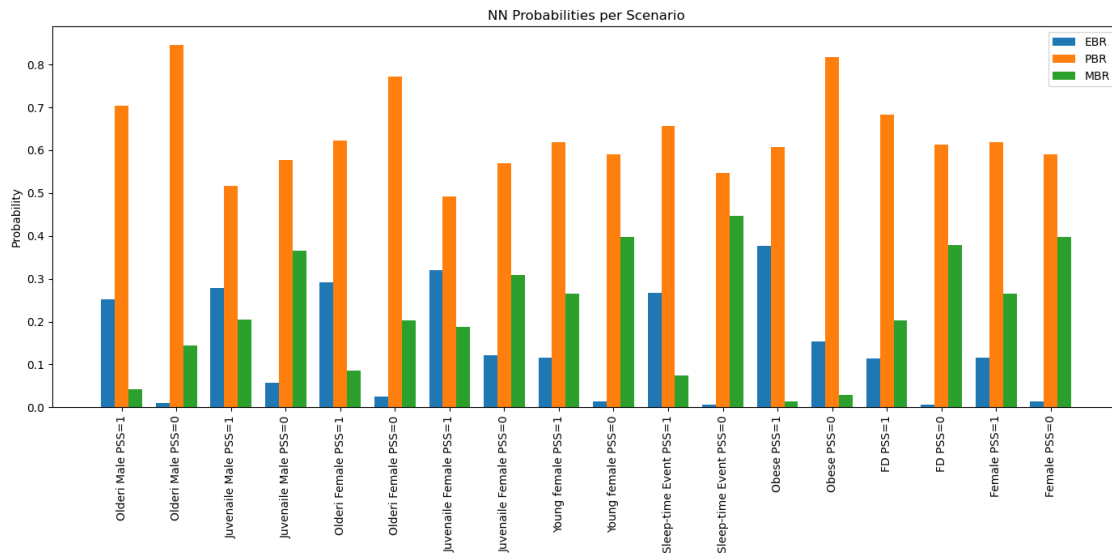
```

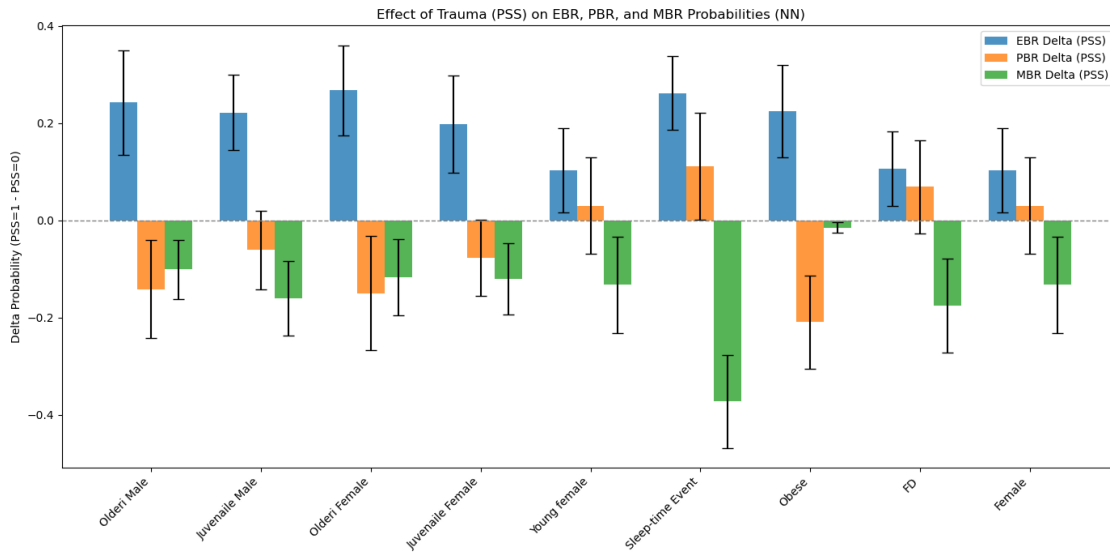
Running on mps

	Scenario	EBR_prob_mean	EBR_prob_std	PBR_prob_mean \
0	Olderi Male PSS=1	0.252091	0.115133	0.704810
1	Olderi Male PSS=0	0.009546	0.015070	0.846208
2	Juvenaile Male PSS=1	0.278714	0.102818	0.516050
3	Juvenaile Male PSS=0	0.057127	0.049897	0.577066
4	Olderi Female PSS=1	0.291262	0.110084	0.622458
5	Olderi Female PSS=0	0.024492	0.044619	0.772342
6	Juvenaile Female PSS=1	0.319579	0.109641	0.492504
7	Juvenaile Female PSS=0	0.121869	0.098584	0.569320
8	Young female PSS=1	0.115794	0.097595	0.619645
9	Young female PSS=0	0.013106	0.015703	0.589586
10	Sleep-time Event PSS=1	0.268014	0.073963	0.657521
11	Sleep-time Event PSS=0	0.006359	0.006820	0.546508
12	Obese PSS=1	0.377421	0.092892	0.607942
13	Obese PSS=0	0.153171	0.100651	0.817396
14	FD PSS=1	0.113900	0.080906	0.682331
15	FD PSS=0	0.007349	0.008575	0.613618
16	Female PSS=1	0.115794	0.097595	0.619645
17	Female PSS=0	0.013106	0.015703	0.589586

	PBR_prob_std	MBR_prob_mean	MBR_prob_std
0	0.112301	0.043099	0.032904
1	0.065522	0.144246	0.065289
2	0.084637	0.205236	0.075563
3	0.067430	0.365807	0.064181

4	0.092148	0.086280	0.064582
5	0.093147	0.203165	0.094032
6	0.072334	0.187917	0.080029
7	0.075750	0.308812	0.086962
8	0.098768	0.264562	0.099897
9	0.069930	0.397309	0.072282
10	0.065118	0.074465	0.035517
11	0.082747	0.447133	0.085390
12	0.092985	0.014637	0.012194
13	0.104363	0.029433	0.017219
14	0.096580	0.203769	0.096544
15	0.078329	0.379033	0.079715
16	0.098768	0.264562	0.099897
17	0.069930	0.397309	0.072282





4.3.2 Results and Visualization

Scenario-Based Prediction Plot The first bar chart shows the **mean predicted probabilities** of each behavioral category (EBR, PBR, MBR) across 18 population scenarios. Probabilities were averaged across 30 bootstrap-trained neural networks.

- **Scenarios with trauma (PSS = 1)** tend to show higher EBR and PBR risks.
- **Young females and juvenile males** appear particularly vulnerable to trauma-induced shifts.

Delta Plot (Effect of Trauma) The second plot displays the **mean delta in probability** between PSS = 1 and PSS = 0 for each group. Error bars represent standard deviations across bootstraps.

- Positive values indicate increased likelihood of a category under trauma.
- For most groups, trauma increases **EBR** probability and reduces **MBR**.
- Some profiles (e.g., female-related) show larger variability, potentially reflecting interaction effects.

[]: